



Technical University of Denmark

Written examination, August 20, 2016

Page 1 of 10 pages

Course name: Programming in C++

Course number: 02393

Aids allowed: All aids allowed

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises of 2.5 points each for a total of 10 points.

Submission details:

1. You can hand-in your solutions manually (on paper). However, we strongly recommend you to submit them electronically.
2. For electronic submission, you **must** upload your solutions on CampusNet and you can do it only once: resubmission is not possible, so submit only when you have finished all exercises. Each exercise must be uploaded as one separate `.cpp` file, using the names specified in the exercises, namely `exZZ-library.cpp`, where `ZZ` ranges from `01` to `04`. The files must be handed in separately (not as a zip-file) and must have these exact filenames. Feel free to add comments to your code.
3. You *can also* upload your solutions individually on CodeJudge under **Re-Exam** at <https://dtu.codejudge.net/02393-f16/assignment>. When you hand in a solution on CodeJudge, the test example given in the assignment description will be run on your solution. Consider that additional tests may be run on your solutions after the exam. You can upload to CodeJudge as many times as you like during the exam.

EXERCISE 1. REDUCING MATRICES (2.5 POINTS)

Alice needs to perform some computations on square matrices. She has already implemented part of the code but she is not sure about its correctness, and some parts are still missing. Her first test program is in file `ex01-main.cpp` and the (incomplete) code with some functions she needs is in files `ex01-library.h` and `ex01-library.cpp`. All files are available on CampusNet and in the next pages. Help Alice by solving the following tasks:

- (a) Check the implementation of function `void display(double ** A, unsigned int n)` and correct it if necessary. The function should correctly display the $n \times n$ matrix `A`. For a 3×3 square matrix with all 0s the expected output is

```
0 0 0
0 0 0
0 0 0
```

Notice that Alice has decided to represent an $n \times n$ square matrix as an array of arrays (each of size n). Recall that in such a representation the element at row i and column j in a matrix `A` is accessed by `A[i][j]`.

- (b) Implement function `reset(double ** A, unsigned int n, double x)`. The function should set all cells in the $n \times n$ square matrix `A` to value `x`.
- (c) Implement function `vector<double> sumRows(double ** A, unsigned int n)`. This function should take as input an $n \times n$ square matrix `A` and should return a vector that contains the sums of values of each row in `A`. For the first matrix below in task (e), the vector would be (1, 5, 4).
- (d) Implement function `vector<double> sumCols(double ** A, unsigned int n)`. This function should take as input an $n \times n$ square matrix `A` and should return a vector that contains the sums of values of each column in `A`. For the first matrix below in task (e), the vector would be (2, 5, 3).
- (e) Implement the function `reduce(double ** A, unsigned int n)`. This function should take as input an $n \times n$ square matrix `A` and should update all elements of the matrix according to the following idea: each cell should take the sum of all *adjacent* cells. Adjacent cells are cells that are above, below, leftwards or rightwards. Let $a_{i,j}$ be the cell in row i and column j . For $a_{0,0}$ the adjacent cells are $a_{0,1}$ and $a_{1,0}$ only, since there is no cell leftwards or above $a_{0,0}$. As an example, reducing this matrix:

0 1 0		3 0 4
2 0 3	Should update it to	0 10 0
0 4 0		6 0 7

Exercise follows in next page...

File ex01-main.cpp

```

#include <iostream>
#include <string>
#include "ex01-library.h"

using namespace std;

int main(void){

    // I am building my initial matrix here
    unsigned int n = 3;
    double * * A = new double *[n];
    for(unsigned int i = 0; i<n; i++){
        A[i] = new double[n];
    }

    // Setting all values to 0
    reset(A,n,0);
    // Setting some values in the matrix
    A[0][1] = 1;
    A[1][0] = 2;
    A[1][2] = 3;
    A[2][1] = 4;
    display(A,n);
    cout << endl;

    // Summing up rows and values
    vector<double> v;
    v = sumRows(A,n);
    print(v);
    v = sumCols(A,n);
    print(v);
    cout << endl;

    // Reducing the matrix
    reduce(A,n);
    display(A,n);

    for(unsigned int i = 0; i<n; i++){
        delete [] A[i];
    }
    delete [] A;

    return 0;
}

```

File ex01-library.h

```

#ifndef __ex01_library__
#define __ex01_library__

#include <vector>

using namespace std;

void display(double * * A, unsigned int n);
void reset(double * * A, unsigned int n, double x);
void reduce(double * * A, unsigned int n);
vector<double> sumRows(double * * A, unsigned int n);
vector<double> sumCols(double * * A, unsigned int n);
void print(vector<double> & v);

#endif

```

File ex01-library.cpp

```

#include <iostream>
#include <vector>
#include "ex01-library.h"

using namespace std;

// Exercise 1 (a)
// Check and correct if necessary
void display(double * * A, unsigned int n){
    for(unsigned int i = 0; i <= n; i++){
        for(unsigned int j = 0; j <= n; j++){
            cout << A[i][j] << " ";
        }
        cout << endl;
    }
}

// Exercise 1 (b)
// Implement this function
void reset(double * * A, unsigned int n, double x){
    // Put your code here
}

// Exercise 1 (c)
// Implement this function
vector<double> sumRows(double * * A, unsigned int n){
    // Put your code here
}

// Exercise 1 (d)
// Implement this function
vector<double> sumCols(double * * A, unsigned int n){
    // Put your code here
}

// Exercise 1 (e)
// Implement this function
void reduce(double * * A, unsigned int n){
    // Put your code here
}

// Do not modify
void print(vector<double> & v){
    for(unsigned int i=0; i<v.size(); i++){
        cout << v[i] << " ";
    }
    cout << endl;
}

```

EXERCISE 2. MATCHING SEQUENCES (2.5 POINTS)

Bob works for a bioinformatics lab and needs to perform a complex operation to match sequences of elements related to DNA and other biological data. Given two sequences of elements $u = u_1, u_2, \dots, u_k$ and $v = v_1, v_2, \dots, v_l$ the *match* function returns a new sequence and is recursively defined as follows

$$\text{match}(u, v) = \begin{cases} \epsilon & \text{if } u = \epsilon \text{ or } v = \epsilon \\ \text{match}(p(u), p(v)), t(u) & \text{if } t(u) = t(v) \\ \text{match}(u, p(v)) & \text{if } |\text{match}(u, p(v))| \geq |\text{match}(p(u), v)| \\ \text{match}(p(u), v) & \text{otherwise} \end{cases}$$

where

- ϵ denotes the empty sequence;
- $|w|$ denotes the length of a sequence w ;
- concatenation of sequences is denoted with a comma “,”;
- $t(w)$ denotes the last element of a non-empty sequence, i.e. $t(w_1, w_2, w_3, \dots, w_n) = w_n$;
- $p(w)$ denotes the sequence obtained by removing the last element of w , that is $p(w_1, w_2, w_3, \dots, w_{n-1}, w_n) = w_1, w_2, w_3, \dots, w_{n-1}$. If w is empty or has just one element then $p(w)$ is just ϵ ;

As an example you can easily check that matching the sequences X, Y, Z and X, Z yields

$$\begin{aligned} & \text{match}((X, Y, Z), (X, Z)) \\ = & \text{match}((X, Y), (X)), Z && \text{since } t(X, Y, Z) = t(X, Z) = Z \\ = & \text{match}((X), (X)), Z && \text{since } |\text{match}((X), (X))| \geq |\text{match}((X, Y), (\epsilon))| \\ = & (\text{match}((\epsilon), (\epsilon), X)), Z && \text{since } t(X) = t(X) = X \\ = & X, Z && \text{since } \text{match}(\epsilon, \epsilon) = \epsilon \text{ and } u, \epsilon = u \text{ for all sequences } u \end{aligned}$$

Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. All files are available on CampusNet and in the next pages.

Exercise follows in next page...

02393 Programming in C++

As you can see Bob has decided to represent elements with strings and sequences of elements with vectors of strings. He has already implemented some functions to read the sequences from `stdin` and print a sequence in `stdout`. What he is missing is the implementation of function `match` in file `ex02-library.cpp`. Help Bob implementing such function.

File ex02-main.cpp

```
#include <iostream>
#include <vector>
#include <string>
#include "ex02-library.h"

int main(void){

    // Read two sequences of strings
    // end of sequence is denote by "STOP"
    vector<string> u = read_until("STOP");
    vector<string> v = read_until("STOP");

    // Match the sequences
    vector<string> w = match(u,v);

    // Display the result
    display(w);

    return 0;
}
```

File ex02-library.h

```
#ifndef __ex02_library__
#define __ex02_library__

#include <vector>
#include <string>

using namespace std;

vector<string> match(vector<string> & u,
                    vector<string> & v);

vector<string> read_until(string stop);

void display(vector<string> & u);

#endif
```

File ex02-library.cpp

```
#include <iostream>
#include <vector>
#include <string>
#include "ex02-library.h"

using namespace std;

// Exercise 2
vector<string> match(vector<string> & u,
                    vector<string> & v){

    // Put your code here

}

// Do not modify
vector<string> read_until(string stop){
    vector<string> u;
    string e;
    while(true){
        cin >> e;
        if(cin.fail() || e == stop) break;
        u.push_back(e);
    }
    return u;
}

// Do not modify
void display(vector<string> & u){
    for(unsigned int i=0; i<u.size(); i++)
        cout << u[i] << " " ;
    cout << endl;
}
```

EXERCISE 3. LOST IN TRANSLATION (2.5 POINTS)

Claire wants to implement a class `Dictionary` to support some basic translation functionalities, like translating a word between languages or obtaining a word's synonyms in a given language. Her first test program is in file `ex03-main.cpp` and the (incomplete) code with some functions he needs is in files `ex03-library.h` and `ex03-library.cpp`. All files are available on CampusNet and in the next pages. Help Claire by implementing the class `Dictionary` in file `ex03-library.cpp`.

Claire does not know how to implement the methods but she has been told that the `map` containers of the standard library already provide a lot of the functionalities she needs. So she has decided to use the following internal (`private`) representation for the library:

- `map<string,string> english2danish`: A mapping from strings (representing an english word) into strings (representing its direct translation into danish).
- `map<string,string> danish2english`: A mapping from strings (representing a danish word) into strings (representing its direct translation into english).
- `map<string,set<string> > english_synonyms`: A mapping from strings (representing an english word) into sets of strings (representing a set of synonyms).
- `map<string,set<string> > danish_synonyms`: A mapping from strings (representing a danish word) into sets of strings (representing a set of synonyms).

Help Claire by performing the following tasks:

- (a) Check the implementation of method `void insert_words(string u, string v)` and correct it if necessary. The method should record that the english word `u` and the danish word `v` are the direct translations of each other, provided that neither `u` nor `v` have direct translations already.
- (b) Check the implementation of method `string get_word(string lang, string u)` and correct it if necessary. The method should return the direct translation of the word `u` in language `lang` (either `"english"` or `"danish"`), or the string `"#unknown#"` if there is no direct translation.
- (c) Implement method `void insert_synonym(string lang, string u, string v)`. This method inserts a synonym `v` and `u` as synonyms in language `lang`. If `lang` is neither `"english"` nor `"danish"` the method should do nothing.
- (d) Implement method `set<string> get_synonyms(string lang, string u)`. Given a language `lang` and a word `u` the method should return the set of synonyms of `u`.
Exercise follows in next page...

- (e) Implement method `set<string> translate(string lang, string u)`. Given a language `lang` and a word `u` the method should return the set of possible translations of `u`. These should include not only the direct translation of `u` but also the synonyms of the direct translation of `u`. For example, in the test in file `ex03-main.cpp`, Claire expects to get the output `car` (because it is the direct translation of “bil”) and `auto` (because it is a synonym of “car”).

Hints about using maps:

- A key `k` in a map `m` can be updated (mapped) to `v` with `m[k] = v`;
- The value mapped to a key `k` in a map `m` is obtained with `m[k]`;
- The above two methods create the entry for the key if it is not present in the map. To check if the key is present you can use the test `m.find(k) != m.end()`.

File ex03-main.cpp

```
#include <iostream>
#include <string>
#include <set>
#include "ex03-library.h"

using namespace std;

int main(void){
    Dictionary d;
    set<string> s;

    d.insert_words("car","bil");
    cout << d.get_word("english","car") << endl;
    cout << d.get_word("danish","bil") << endl ;

    d.insert_synonym("english","car","auto");

    s = d.get_synonyms("english","auto");
    for(set<string>::iterator it = s.begin();
        it != s.end(); it++)
        cout << *it << " ";
    cout << endl;

    s = d.translate("danish","bil");
    for(set<string>::iterator it = s.begin();
        it != s.end(); it++)
        cout << *it << " ";
    cout << endl;

    return 0;
}
```

File ex03-library.h

```
#ifndef __ex03_library__
#define __ex03_library__

#include <map>
#include <set>
#include <string>

using namespace std;

class Dictionary {

public:
    void insert_words(string u, string v);
    string get_word(string lang, string u);
    void insert_synonym(string lang, string u, string v);
    set<string> get_synonyms(string lang, string u);
    set<string> translate(string lang, string u);

private:
    map<string,string> english2danish;
    map<string,string> danish2english;
    map<string,set<string> > english_synonyms;
    map<string,set<string> > danish_synonyms;

};

#endif
```

File ex03-library.cpp

```
#include <iostream>
#include <map>
#include <set>
#include <vector>
#include "ex03-library.h"

using namespace std;

// Exercise 3(b)
// Check and correct if necessary
void Dictionary::insert_words(string u, string v){
    english2danish[u] = v;
    danish2english[u] = v;
}

// Exercise 3(b)
// Check and correct if necessary
string Dictionary::get_word(string lang, string u){
    return english2danish[u];
}

// Exercise 3(c)
void Dictionary::insert_synonym(string lang, string u,
                                string v){
    // Put your code here
}

// Exercise 3(d)
set<string> Dictionary::get_synonyms(string lang, string u){
    // Put your code here
}

// Exercise 3(e)
set<string> Dictionary::translate(string lang, string u){
    // Put your code here
}
```


EXERCISE 4. FUN WITH MONOIDS (2.5 POINTS)

A monoid expression is either a constant or the composition of two monoid expressions. Hugo is a fan of monoid expressions and their many applications and is implementing a C++ library for supporting them. He has prepared a test program in file `ex04-main.cpp`, the declaration of the class `Monoid` for monoid expressions in file `ex04-library.h` and a sketch of its implementation in file `ex04-library.cpp`. All files are available on CampusNet and in the next pages. Hugo has decided to use the following internal (`private`) representation for monoid expressions:

- `C constant`: A value of class `C` to store constant value expressions. If the expression is the binary composition of two expressions then the value of `constant` is irrelevant.
- `Monoid * m1`: A pointer to a monoid. If the monoid expression is a constant, `m1` is a `nullptr`, otherwise (the monoid expression is a binary composition) it points to the left operand.
- `Monoid * m2`: A pointer to a monoid. If the monoid expression is a constant, `m2` is a `nullptr`, otherwise it points to the right operand.

Hugo has already implemented several functions, including constructors, an operator `*` to compose monoids, and an assignment operator `=`. Help Hugo finish the implementation of class `Monoid` in file `ex04-library.cpp`. Your tasks are:

- (a) Check the class destructor and correct it if necessary.
- (b) Implement method `int constants(void)`. This method should return the number of constants in a monoid expression. In the example of the test program, the expected result is 3.
- (c) Implement the method `void commute(void)`. If invoked for a monoid expression with two operands, it should reverse the order of the operands. For example, an expression $x * y$ should become $y * x$.
- (d) Implement the method `void associate_left(void)`. If invoked for a monoid expression with two operands, such that the right operand has at least two operands, the expression should be re-arranged so that the operands are associated to the left. For example, an expression $x * (y * z)$ should become $(x * y) * z$.
- (e) Implement the method `void associate_right(void)` (symmetric case for task (d)).

Exercise follows in next page...

File ex04-main.cpp

```
#include <iostream>
#include <string>
#include "ex04-library.h"

using namespace std;

int add(int u, int v){ return u + v; }
int subtract(int u, int v){ return u - v; }

int main(void){

    Monoid<int> u(1);
    Monoid<int> v(2);
    Monoid<int> w(3);

    Monoid<int> x((u * v) * w);
    x.print(); cout << endl;
    cout << x.constants() << endl;

    x.commute();
    x.print(); cout << endl;

    x.associate_left();
    x.print(); cout << endl;

    x.associate_right();
    x.print(); cout << endl;

    return 0;
}
```

File ex04-library.h

```
#ifndef __ex04_library__
#define __ex04_library__

#include <string>
using namespace std;

template <class C>
class Monoid {
public:
    Monoid(C constant);
    Monoid(Monoid<C> & m1);
    Monoid(Monoid<C> & m1, Monoid<C> & m2);
    Monoid<C> & operator=(Monoid<C> & m);
    Monoid<C> & operator*(Monoid<C> & m);
    ~Monoid(void);
    C eval(C (*f)(C,C));
    int constants(void);
    void commute(void);
    void associate_left(void);
    void associate_right(void);
    void print(void);
private:
    C constant;
    Monoid<C> * m1;
    Monoid<C> * m2;
};

#endif
```

File ex04-library.cpp

```
#include <iostream>
#include "ex04-library.h"

using namespace std;

// Exercise 4(a)
template <class C>
Monoid<C>::~Monoid(void){
    if(m1 != nullptr) delete m1;
    if(m2 != nullptr) delete m2;
}

// Exercise 4(b)
template <class C>
int Monoid<C>::constants(void){
    // Put your code here
}

// Exercise 4(c)
template <class C>
void Monoid<C>::commute(void){
    // Put your code here
}

// Exercise 4(d)
template <class C>
void Monoid<C>::associate_left(void){
    // Put your code here
}

// Exercise 4(e)
template <class C>
void Monoid<C>::associate_right(void){
    // Put your code here
}

// Some code follows...
// You don't need to understand it
// but it could give you some hints
```



Technical University of Denmark

Written examination, May 17, 2016

Page 1 of 11 pages

Course name: Programming in C++

Course number: 02393

Aids allowed: All aids allowed

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises of 2.5 points each for a total of 10 points.

Submission details:

1. You can hand-in your solutions manually (on paper). However, we strongly recommend you to submit them electronically.
2. For electronic submission, you **must** upload your solutions on CampusNet. Each assignment must be uploaded as one separate `.cpp` file, using the names specified in the exercises, namely `exZZ-library.cpp`, where `ZZ` ranges from `01` to `04`. The files must be handed in separately (not as a zip-file) and must have these exact filenames. Feel free to add comments to your code.
3. You *can also* upload your solutions individually on CodeJudge under the **Exam** at <https://dtu.codejudge.net/02393-f16/assignment>. When you hand in a solution on CodeJudge, the test example given in the assignment description will be run on your solution. Consider that further tests may be run on your solutions, also after the exam. You can upload to CodeJudge as many times as you like during the exam.

EXERCISE 1. REDUCING MATRICES (2.5 POINTS)

Alice needs to perform some computations on square matrices. She has already implemented part of the code she needs but she is not sure about its correctness, and some parts are still missing. Her first test program is in file `ex01-main.cpp` and the (incomplete) code with some functions she needs is in files `ex01-library.h` and `ex01-library.cpp`. All files are available on CampusNet and in the next pages. Help Alice by solving the following tasks:

- (a) Check the implementation of function `void display(double * A, unsigned int n)` and correct it if necessary. The function should correctly display the $n \times n$ matrix `A`. For a 3×3 square matrix with all 0s the expected output is

```
0 0 0
0 0 0
0 0 0
```

Notice that Alice has decided to represent $n \times n$ square matrices with single index arrays of size $n \times n$ (as we did in several examples during the course).

- (b) Implement function `reset(double * A, unsigned int n, double x)`. The function should set all cells in the $n \times n$ square matrix `A` to value `x`.
- (c) Implement the function `reduce(double * A, unsigned int n)`. This function should take as input an $n \times n$ square matrix `A` and should update all elements of the matrix according to the following idea: each cell should take the sum of all *adjacent* cells. Adjacent cells are cells that can be found above, below, leftwards or rightwards. Let $a_{i,j}$ denote the cell in row i and column j . For $a_{0,0}$ the adjacent cells are $a_{0,1}$ and $a_{1,0}$ only, since there is no cell leftwards or above $a_{0,0}$. As an example, reducing this matrix:

0 1 0		3 0 4
2 0 3	Should update it to	0 10 0
0 4 0		6 0 7

- (d) Implement function `vector<double> sumRows(double * A, unsigned int n)`. This function should take as input an $n \times n$ square matrix `A` and should return a vector that contains the sums of values of each row in `A`. For the last matrix above, the vector would be (7, 10, 13).
- (e) Implement function `vector<double> sumCols(double * A, unsigned int n)`. This function should take as input an $n \times n$ square matrix `A` and should return a vector that contains the sums of values of each column in `A`. For the last matrix above, the vector would be (9, 10, 11).

Exercise follows in next page...

File ex01-main.cpp

```
#include <iostream>
#include <string>
#include "ex01-library.h"

using namespace std;

int main(void){

    // I am building my initial matrix here
    unsigned int n = 3;
    double * A = new double[n*n];

    // I am setting all values to 0
    reset(A,n,0);
    // Setting some values in the matrix
    A[0*3+1] = 1;
    A[1*3+0] = 2;
    A[1*3+2] = 3;
    A[2*3+1] = 4;
    display(A,n);
    cout << endl;

    // Reducing the matrix
    reduce(A,n);
    display(A,n);
    cout << endl;

    // Finally, I am summing up rows and values
    vector<double> v;
    v = sumRows(A,n);
    print(v);
    v = sumCols(A,n);
    print(v);

    return 0;
}
```

File ex01-library.h

```
#ifndef __ex01_library__
#define __ex01_library__

#include <vector>

using namespace std;

void display(double * A, unsigned int n);

void reset(double * A, unsigned int n, double x);

void reduce(double * A, unsigned int n);

vector<double> sumRows(double * A, unsigned int n);

vector<double> sumCols(double * A, unsigned int n);

void print(vector<double> & v);

#endif
```

File ex01-library.cpp

```
#include <iostream>
#include <vector>
#include "ex01-library.h"

using namespace std;

// Exercise 1 (a)
// Check and correct if necessary
void display(double *A, unsigned int n){
    for(unsigned int i = 1; i <= n; i++){
        for(unsigned int j = 1; j <= n; j++){
            cout << A[i*n+j] << " ";
        }
        cout << endl;
    }
}

// Exercise 1 (b)
// Implement this function
void reset(double *A, unsigned int n, double x){
    // Put your code here
}

// Exercise 1 (c)
// Implement this function
void reduce(double * A, unsigned int n){
    // Put your code here
}

// Exercise 1 (d)
// Implement this function
vector<double> sumRows(double * A, unsigned int n){
    // Put your code here
}

// Exercise 1 (e)
// Implement this function
vector<double> sumCols(double * A, unsigned int n){
    // Put your code here
}

// Do not modify
void print(vector<double> & v){
    for(unsigned int i=0; i<v.size(); i++){
        cout << v[i] << " ";
    }
    cout << endl;
}
```

EXERCISE 2. MATCHING SEQUENCES (2.5 POINTS)

Bob works for a bioinformatics lab and needs to perform a complex operation to match sequences of elements related to DNA and other biological data. Given two sequences of elements $u = u_1, u_2, \dots, u_k$ and $v = v_1, v_2, \dots, v_l$ the *match* function returns a new sequence and is recursively defined as follows

$$\text{match}(u, v) = \begin{cases} \epsilon & \text{if } u = \epsilon \text{ or } v = \epsilon \\ h(u), \text{match}(t(u), t(v)) & \text{if } h(u) = h(v) \\ \text{match}(u, t(v)) & \text{if } |\text{match}(u, t(v))| \geq |\text{match}(t(u), v)| \\ \text{match}(t(u), v) & \text{otherwise} \end{cases}$$

where

- ϵ denotes the empty sequence;
- $|w|$ denotes the length of a sequence w ;
- concatenation of sequences is denoted with a comma “,”;
- $h(w)$ denotes the first element of a non-empty sequence, i.e. $h(w_1, w_2, w_3, \dots) = w_1$;
- $t(w)$ denotes the sequence obtained by removing the first element of w , that is $h(w_1, w_2, w_3, \dots) = w_2, w_3, \dots$. If w is empty or has just one element then $t(w)$ is just ϵ ;

As an example you can easily check that matching the sequences A, B, C and A, C yields

$$\begin{aligned} & \text{match}((A, B, C), (A, C)) \\ = & A, \text{match}((B, C), (C)) && \text{since } h(A, B, C) = h(A, C) = A \\ = & A, \text{match}((C), (C)) && \text{since } |\text{match}((C), (C))| \geq |\text{match}((B, C), (\epsilon))| \\ = & A, C, \text{match}((\epsilon), (\epsilon)) && \text{since } h(C) = h(C) = C \\ = & A, C && \text{since } \text{match}(\epsilon, \epsilon) = \epsilon \text{ and } u, \epsilon = u \text{ for all sequences } u \end{aligned}$$

Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. All files are available on CampusNet and in the next pages.

Exercise follows in next page...

02393 Programming in C++

As you can see Bob has decided to represent elements with strings and sequences of elements with vectors of strings. He has already implemented some functions to read the sequences from `stdin` and print a sequence in `stdout`. What he is missing is the implementation of function `match` in file `ex02-library.cpp`. Help Bob implementing such function.

File `ex02-main.cpp`

```
#include <iostream>
#include <vector>
#include <string>
#include "ex02-library.h"

int main(void){

    // Read two sequences of strings
    // end of sequence is denote by "STOP"
    vector<string> u = read_until("STOP");
    vector<string> v = read_until("STOP");

    // Match the sequences
    vector<string> w = match(u,v);

    // Display the result
    display(w);

    return 0;
}
```

File `ex02-library.h`

```
#ifndef __ex02_library__
#define __ex02_library__

#include <vector>
#include <string>

using namespace std;

vector<string> match(vector<string> & u,
                    vector<string> & v);

vector<string> read_until(string stop);

void display(vector<string> & u);

#endif
```

File `ex02-library.cpp`

```
#include <iostream>
#include <vector>
#include <string>
#include "ex02-library.h"

using namespace std;

// Exercise 2
vector<string> match(vector<string> & u,
                    vector<string> & v){

    // Put your code here

}

// Do not modify
vector<string> read_until(string stop){
    vector<string> u;
    string e;
    while(true){
        cin >> e;
        if(cin.fail() || e == stop) break;
        u.push_back(e);
    }
    return u;
}

// Do not modify
void display(vector<string> & u){
    for(unsigned int i=0; i<u.size(); i++)
        cout << u[i] << " " ;
    cout << endl;
}
```

EXERCISE 3. LOST IN TRANSLATION (2.5 POINTS)

Claire wants to implement a class `Dictionary` to support some basic translation functionalities, like translating a word between languages or obtaining a word's synonyms in a given language. Her first test program is in file `ex03-main.cpp` and the (incomplete) code with some functions he needs is in files `ex03-library.h` and `ex03-library.cpp`. All files are available on CampusNet and in the next pages. Help Claire by implementing the class `Dictionary` in file `ex03-library.cpp`.

Claire does not know how to implement the methods but she has been told that the `map` containers of the standard library already provide a lot of the functionalities she needs. So she has decided to use the following internal (`private`) representation for the library:

- `vector<map<string,string> > words`: A vector of mappings from strings (representing a word) into strings (representing its translation). The idea is that the vector will have size 2: one for each possible direction of the translation. The first language of the dictionary will be denoted and indexed with 0 and the second one by 1. So the idea is that if Claire introduces the pair of English/Danish words ("car", "bil") these will be stored as follows: `words[0]["car"]` will be mapped to "bil" and `words[1]["bil"]` will be mapped to "car".
- `vector<map<string,set<string> > > synonyms`: A vector of mappings from strings (representing a word) into sets of strings (representing a set of synonyms). Again, the idea is that the vector will have size 2: one for each possible language. For example, if Claire introduces the synonym "auto" for "car" then `synonyms[0]["car"]` will be a set containing "auto" (and possibly other synonyms introduced before).

Help Claire by performing the following tasks:

- (a) Implement method `Dictionary(void)`. This is the constructor of the dictionary.
- (b) Check the implementation of methods `void insert_words(string u, string v)` and `string get_word(int lang, string u)` and correct them if necessary. The first method inserts a word `u` from the first language and its translation `v` in the second language in the dictionary. The second method should return the direct translation of the `lang` word `u`, or the empty string if there is no direct translation. Hint: see how Claire uses these two functions in her test program.
- (c) Implement method `void insert_synonym(int lang, string u, string v)`. This method inserts a synonym `v` for word `u` of language `lang`.
- (d) Implement method `set<string> get_synonyms(int lang, string u)`. Given a language `lang` and a word `u` the method should return the set of synonyms of `u`.

Exercise follows in next page...

- (e) Implement method `set<string> translate(int lang, string u)`. Given a language `lang` and a word `u` the method should return the set of possible translations of `u`. These should include not only the direct pair of words introduced with `insert_words` but also those words obtained through synonyms, in particular the method should return the translation of `u` and of the synonyms of `u`, and their respective synonyms. For example, in the test in file `ex03-main.cpp`, Claire expects to get the output `car` (because the direct translation of “bil” is in the dictionary), `auto` (because it is a synonym of “car”) , `wagon` (because it is the direct translation of “vogn”, which is a synonym of “bil”) and `van` (because it is a synonym of “wagon”).

Hints about using maps:

- A key `k` in a map `m` can be updated (mapped) to `v` with `m[k] = v`;
- The value mapped to a key `k` in a map `m` is obtained with `m[k]`;
- The above two methods create the entry for the key if it is not present in the map. To check if the key is present you can use the test `m.find(k) != m.end()`.

File ex03-main.cpp

```
#include <iostream>
#include <string>
#include <set>
#include "ex03-library.h"

using namespace std;

int main(void){

    Dictionary d;

    d.insert_words("car","bil");
    cout << d.get_word(0,"car") << endl;
    cout << d.get_word(1,"bil") << endl ;

    d.insert_words("wagon","vogn");
    d.insert_synonym(0,"car","auto");
    d.insert_synonym(1,"bil","vogn");
    d.insert_synonym(0,"wagon","van");

    set<string> s = d.translate(1,"bil");
    for(set<string>::iterator it = s.begin();
        it != s.end(); it++)
        cout << *it << " ";
    cout << endl;

    return 0;
}
```

File ex03-library.h

```
#ifndef __ex03_library__
#define __ex03_library__

#include <map>
#include <set>
#include <vector>
#include <string>

using namespace std;

class Dictionary {
public:
    Dictionary(void);
    void insert_words(string u, string v);
    string get_word(int lang, string u);
    void insert_synonym(int lang, string u, string v);
    set<string> get_synonyms(int lang, string u);
    set<string> translate(int lang, string u);

private:
    vector<map<string,string> > words;
    vector<map<string,set<string> > > synonyms;

};

#endif
```

File ex03-library.cpp

```
#include <iostream>
#include <map>
#include <set>
#include <vector>
#include "ex03-library.h"

using namespace std;

// Exercise 3(a)
Dictionary::Dictionary(void){
    // Put your code here
}

// Exercise 3(b)
// Check and correct if necessary
void Dictionary::insert_words(string u, string v){
    words[0][u] = v;
}

// Exercise 3(b)
// Check and correct if necessary
string Dictionary::get_word(int lang, string u){
    return words[lang][u];
}

// Exercise 3(c)
void Dictionary::insert_synonym(int lang, string u,
                                string v){
    // Put your code here
}

// Exercise 3(d)
set<string> Dictionary::get_synonyms(int lang, string u){
    // Put your code here
}

// Exercise 3(e)
set<string> Dictionary::translate(int lang, string u){
    // Put your code here
}
```

EXERCISE 4. FUN WITH MONOIDS (2.5 POINTS)

A monoid is an algebraic structure with a single associative binary operation and an identity element. Typical examples are sequences with concatenation as binary operation and the empty string as identity element, or natural numbers with addition as binary operation and 0 as identity element. Hugo is a fan of monoids and their many applications and wants to implement a C++ library for supporting monoid expressions. A monoid expression can be either a constant or the composition of two monoid expressions. Hugo has prepared a test program in file `ex04-main.cpp`, the declaration of the class `Monoid` for monoid expressions in file `ex04-library.h` and a sketch of its implementation in file `ex04-library.cpp`. All files are available on CampusNet and in the next pages.

Hugo is unsure about the implementation but he has decided to use the following internal (`private`) representation for monoid expressions:

- **C constant**: A value of class `C` to store constant value expressions. If the expression is the binary composition of two expressions then the value of `constant` is irrelevant.
- **Monoid * m1**: A pointer to a monoid. If the monoid expression is a constant, `m1` is a `nullptr`, otherwise (the monoid expression is a binary composition) it points to the left operand.
- **Monoid * m2**: A pointer to a monoid. If the monoid expression is a constant, `m2` is a `nullptr`, otherwise it points to the right operand.

Help Hugo implementing the class `Monoid` in file `ex04-library.cpp`. Your tasks are:

- (a) Implement the constructor `Monoid(C constant)`. This is a constructor of constant monoid expressions. The constructed monoid expression should contain the value `constant`. See for example how Hugo builds the monoid expression for constant "Hello" with `Monoid<string> a("Hello")`.
- (b) Implement the constructor `Monoid(Monoid<C> & m1)`. This is a *copy* constructor. It builds a monoid expression which is a new copy of `m1`.
- (c) Implement the constructor `Monoid(Monoid<C> & m1, Monoid<C> & m2)`. This constructor takes two monoid expressions as arguments and builds a new expression (representing a binary composition) with new copies of `m1` and `m2` as left and right arguments, respectively. See for example how Hugo builds the monoid expression that results from combining monoid expressions `a` and `b` with `Monoid<string> d(a,b)`.
- (d) Implement method `Monoid<C> operator*(Monoid<C> & m)`. This is a binary operation. It takes two monoid expressions (the current object and `m`) and returns a new monoid expression with a new copy of the the current object as left operand and a new copy of `m` as right operand. Note the similarity with the binary constructor. Indeed, `Monoid<string> d(a,b)` and `a * b` build identical monoid expressions.

Exercise follows in next page...

- (e) Implement method `C eval(C (*f)(C,C))`. This method takes as parameter a function `f` (that returns a value of type `C` when applied to two arguments of type `C`), and should return the result of evaluating the monoid expression using `f` as interpretation of the monoid operator. That is, if the monoid expression is a constant it should just return the value of the constant, otherwise it should return the result of applying `f` to the evaluation of the two operands (`m1`, `m2`) of the monoid expression.

For example, in Hugo's test program, he expects to get the following output (as the result of invoking `eval`)

```
Hello world!  
Hello world!  
6
```

Note: we have not seen how to pass functions as parameters in the course, but all you need here is to use `f` within the method just as it would be a normal function (i.e. one not passed as argument), e.g. `f(x,y)` applies `f` with arguments `x` and `y`.

Exercise follows in next page...

File ex04-main.cpp

```
#include <iostream>
#include <string>
#include "ex04-library.h"

using namespace std;

string concat(string u, string v){
    return u + v;
}

unsigned int add(unsigned int u, unsigned int v){
    return u + v;
}

int main(void){
    Monoid<string> a("Hello");
    Monoid<string> b("");
    Monoid<string> c("world!");

    Monoid<string> d(a,b);
    Monoid<string> e(d,c);

    cout << e.eval(concat) << endl;
    cout << (a * b * c).eval(concat) << endl;

    Monoid<unsigned int> u(1);
    Monoid<unsigned int> v(2);
    Monoid<unsigned int> w(3);

    cout << (u * v * w).eval(add) << endl;

    return 0;
}
```

File ex04-library.h

```
#ifndef __ex04_library__
#define __ex04_library__

#include <string>

using namespace std;

template <class C>
class Monoid {

public:
    Monoid(C constant);
    Monoid<C> operator*(Monoid<C> & m);
    Monoid(Monoid<C> & m1, Monoid<C> & m2);
    C eval(C (*f)(C,C));
    void print(void);

private:
    Monoid(Monoid<C> & m1);
    C constant;
    Monoid<C> * m1;
    Monoid<C> * m2;

};

#endif
```

File ex04-library.cpp

```
#include <iostream>
#include "ex04-library.h"

using namespace std;

// Exercise 4 (a)
template <class C>
Monoid<C>::Monoid(C constant){
    // Put your code here
}

// Exercise 4 (b)
template <class C>
Monoid<C>::Monoid(Monoid<C> & m1, Monoid<C> & m2){
    // Put your code here
}

// Exercise 4 (c)
template <class C>
Monoid<C>::Monoid(Monoid & m){
    // Put your code here
}

// Exercise 4 (d)
template <class C>
Monoid<C> Monoid<C>::operator*(Monoid<C> & m){
    // Put your code here
}

// Exercise 4 (e)
template <class C>
C Monoid<C>::eval(C (*f)(C,C)){
    // Put your code here
}

// Do not modify
template <class C>
void Monoid<C>::print(void){
    if (m1 == nullptr) cout << constant ;
    else {
        m1->print();
        cout << " * ";
        m2->print();
    }
}

// Do not modify
template class Monoid<string>;
template class Monoid<unsigned int>;
```



Technical University of Denmark

Written examination, December 10, 2017

Page 1 of 16 pages

Course name: Programming in C++

Course number: 02393

Aids allowed: All aids allowed

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises of 2.5 points each for a total of 10 points.

Submission details:

1. You can hand-in your solutions manually (on paper). However, we strongly recommend you to submit them electronically.
2. For electronic submission, you **must** upload your solutions on CampusNet and you can do it only once: resubmission is not possible, so submit only when you have finished all exercises. Each exercise must be uploaded as one separate `.cpp` file, using the names specified in the exercises, namely `exZZ-library.cpp`, where `ZZ` ranges from `01` to `04`. The files must be handed in separately (not as a zip-file) and must have these exact filenames. Feel free to add comments to your code.
3. You can also upload your solutions on CodeJudge under **Exam December 2017** at <https://dtu.codejudge.net/02393-e17/exercisegroup>. This is not an official submission, and will not be considered for evaluation. When you hand in a solution on CodeJudge, some tests will be run on it. Additional tests may be run on your solutions after the exam. You can upload to CodeJudge as many times as you like.

EXERCISE 1. GRAPHICS EDITOR (2.5 POINTS)

Alice needs to implement a library to manipulate black/white pictures, usually stored as matrices with one entry per pixel. Each entry takes integer values from 0 to 255: 0 corresponds to black, 255 to white, and intermediate values to tonalities of grey.

Alice needs to perform some computations on such matrices. She has already implemented part of the code but she is not sure about its correctness, and some parts are still missing. Her first test program is in file `ex01-main.cpp` and the (incomplete) code with some functions she needs is in files `ex01-library.h` and `ex01-library.cpp`. All files are available on CampusNet (with additional comments) and in the next pages. Help Alice by solving the following tasks:

- (a) Check the implementation of function

```
int ** createMatrix(unsigned int n, unsigned int m)
```

and correct it if necessary. The function should correctly create an $n \times m$ matrix, and return it. The function should only allocate the required memory. Initialization of the entries of the matrix is not required.

Notice that Alice has decided to represent a $n \times m$ matrix as an array (of size n) of arrays (each of size m). Recall that in such a representation the element at row r and column c in a matrix A is accessed by `A[r][c]`.

- (b) Implement function

```
int ** duplicateMatrix(int ** A, unsigned int n, unsigned int m);
```

The function should create and return a copy of the $n \times m$ matrix A .

- (c) Implement function

```
void initMatrix(int ** A, unsigned int n, unsigned int m);
```

The function should set to 0 all entries of the $n \times m$ matrix A .

- (d) Implement function

```
void deallocateMatrix(int ** A, unsigned int n);
```

This function is the dual of `createMatrix`. It should deallocate all memory allocated for the matrix A .

Exercise follows in next page...

(e) Implement function

```
int ** makeBitonal(int ** A, unsigned int n, unsigned int m, int threshold);
```

This function makes a picture *bi-tonal* according to a given *threshold*: each pixel gets either black (0) or white (255). The function takes as input an $n \times m$ matrix **A**, and a threshold. This function should scan all elements of **A**, and replace all values

- below **threshold** with value 0.
- equal to **threshold** with value 255.
- above **threshold** with value 255.

Hence, this filter sets to white all grey entries clearer than the given threshold, and to black all the ones darker than the given threshold.

Notes:

- The function has to apply the filter on a copy, let's say **B**, of **A**. Such modified copy **B** has to be returned.
- The function must not modify **A**.

For example, for threshold 100:

- all entries with value smaller than 100 get value 0.
- all entries with value equal to 100 get value 255.
- all entries with value greater than 100 get value 255.

Exercise follows in next page...

File ex01-main.cpp

```
#include <iostream>
#include <string>
#include "ex01-library.h"

using namespace std;

int main(void){

    // Matrix representing a picture
    unsigned int n = 4;
    unsigned int m = 4;
    int **A = createMatrix(n,m);

    // Setting all values to 0
    initMatrix(A,n,m);
    // Setting some values in the matrix
    for(int i=0;i<n-1;i++){
        for(int j=0;j<m;j++){
            A[i][j]=i*n+j;
        }
    }
    A[1][0]=126;
    A[1][1]=127;
    A[1][2]=128;
    printMatrix(A,n,m,"main_matrix");
    cout << endl;

    int **B = duplicateMatrix(A,n,m);
    printMatrix(B,n,m,"copy");

    //I change B, and I print B and A
    for(int j=0;j<m;j++){
        B[0][j]=B[0][j]+3;
    }

    printMatrix(B,n,m,"modified_copy");
    printMatrix(A,n,m,"main_matrix");

    //I deallocate B
    deallocateMatrix(B,n);

    int ** C = makeBitonal(A,n,m,127);
    printMatrix(C,n,m,"bi-tonal_copy");

    deallocateMatrix(A,n);
    deallocateMatrix(C,n);
    return 0;
}
```

File ex01-library.h

```
#ifndef EX01_LIBRARY_H_
#define EX01_LIBRARY_H_

#include <vector>
#include <string>
using namespace std;

int ** createMatrix(unsigned int n, unsigned int m);
int ** duplicateMatrix(int ** A, unsigned int n, unsigned int m);
void initMatrix(int ** A, unsigned int n, unsigned int m);
void deallocateMatrix(int ** A, unsigned int n);
int ** makeBitonal(int ** A, unsigned int n, unsigned int m,
    int threshold);
void printMatrix(int ** A, unsigned int n, unsigned int m,
    string description);

#endif /* EX01_LIBRARY_H_ */
```

File ex01-library.cpp

```
#include <iostream>
#include <vector>
#include <iomanip>
#include "ex01-library.h"
using namespace std;

//Exercise 1 (a) Check and correct if necessary
int ** createMatrix(unsigned int n, unsigned int m){
    int ** A = new int *[n];
    for(unsigned int i = 0; i<=n; i++){
        A[i] = new int[n];
    }
}

//Exercise 1 (b) Implement this function
int ** duplicateMatrix(int ** A, unsigned int n, unsigned int m){
    //Put your code here
}

//Exercise 1 (c) Implement this function
void initMatrix(int ** A, unsigned int n, unsigned int m){
    //Put your code here
}

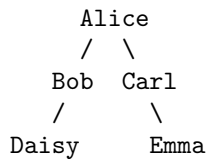
//Exercise 1 (d) Implement this function
void deallocateMatrix(int ** A, unsigned int n){
    //Put your code here
}

//Exercise 1 (e) Implement this function
int ** makeBitonal(int ** A, unsigned int n, unsigned int m,
    int threshold){
    //Put your code here
}

//Do not modify
void printMatrix(int ** A, unsigned int n, unsigned int m,
    string description){
    cout<< "Printing:_" << description << endl;
    for(unsigned int i = 0; i < n; i++){
        for(unsigned int j = 0; j < m; j++){
            cout << setw(5) << A[i][j] << "_";
        }
        cout << endl;
    }
}
```

EXERCISE 2. GENEALOGY (2.5 POINTS)

Bob is interested in studying the genealogy of its family. So far he has built the following family tree:



Alice had two sons: Bob and Carl, which in turn had a daughter each, Daisy and Emma, respectively. Daisy and Emma currently have no descendants.

Bob would like to automate the computation of simple queries on family trees, like the computation of the *descendants* of an individual, or the computation of all individuals that had children (i.e., that are *parents*). Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. All files are available on CampusNet (with additional comments) and in the next pages.

As you can see Bob has decided to represent family trees using binary trees, where each node can have up to two children. Each node of the tree is stored in a struct `Node` containing:

- **name**: the name of the corresponding individual;
- **left** and **right**: pointers to other `Node` structures, representing the children of the corresponding individual. No particular order is assumed among the two pointers.

Note that such representation allows to represent families in which each individual had at most two children. In case the individual had no children, then both pointers have value `nullptr`. In case the individual had one children, then either of the two pointers will refer to the `Node` representing the child, while the other pointer will have value `nullptr`.

Help Bob by solving the following tasks:

- (a) Check the implementation of function

```
void computeParentNodes(Node *n, set<string> & parents)
```

and correct it if necessary. A *parent node* is a node with at least one child. This function should correctly compute all *parent nodes* met while navigating the tree starting from node `n` (including `n` itself, in case it has children). In particular, the function should add the name of the current node `n` to the set `parents` if `n` has at least one child, and then recursively call the function on the children of `n`. For instance, the parent nodes found starting from Alice's node are: Alice, Bob and Carl. Instead, the parent nodes met starting from Bob's node is just Bob itself.

Exercise follows in next page...

(b) Implement function

```
void computeMembersOfSubTree(Node * n, set<string> & members)
```

This function should compute the set of names of all members of the sub-tree starting from node `n` (including `n` itself). Such names have to be added to the set `members`. For instance, the names of the members of the sub-tree starting from Alice's node are: Alice, Bob, Carl, Daisy, and Emma. Instead, the sub-tree starting from Daisy's node contains only Daisy.

Exercise follows in next page...

File ex02-main.cpp

```

#include <iostream>
#include "ex02-library.h"
using namespace std;

void experimentParentNodes(Node * n){
    set<string> parents;
    computeParentNodes(n,parents);
    cout<<"The nodes with children starting from "
        << n->name << " are:\n";
    printSet(parents);
}

void experimentSubtree(Node * n){
    set<string> members;
    computeMembersOfSubTree(n,members);
    cout<<"The sub-tree with root in " << n->name
        << " contains:\n";
    printSet(members);
}

int main() {
    /* Bob's family tree from text */
    Node *Alice = new Node;
    Alice->name="Alice";
    Node *Bob = new Node;
    Bob->name="Bob";
    Node *Carl = new Node;
    Carl->name="Carl";
    Node *Daisy = new Node;
    Daisy->name="Daisy";
    Node *Emma = new Node;
    Emma->name="Emma";
    Alice->left=Bob;
    Alice->right=Carl;
    Bob->left=Daisy;
    Bob->right=nullptr;
    Carl->right=Emma;
    Carl->left=nullptr;
    Daisy->left=nullptr;
    Daisy->right=nullptr;
    Emma->left=nullptr;
    Emma->right=nullptr;

    cout << "Experiments about parent nodes\n";
    experimentParentNodes(Alice);
    experimentParentNodes(Bob);
    experimentParentNodes(Carl);
    experimentParentNodes(Daisy);
    experimentParentNodes(Emma);
    cout << "Experiment about sub-tree\n";
    experimentSubtree(Alice);
    experimentSubtree(Bob);
    experimentSubtree(Carl);
    experimentSubtree(Daisy);
    experimentSubtree(Emma);
    return 0;
}

```

File ex02-library.h

```

#ifndef EX02_LIBRARY_H_
#define EX02_LIBRARY_H_

#include <set>
#include <string>
using namespace std;

struct Node{
    std::string name;
    Node * left;
    Node * right;
};

void printSet(set<string> s);
void computeParentNodes(Node* n, set<string> & parents);
void computeMembersOfSubTree(Node* n, set<string> & members);
#endif

```

File ex02-library.cpp

```

#include "ex02-library.h"
#include <iostream>

//Do not modify
void printSet(set<string> s){
    if(s.size()==0){
        cout << "No nodes\n";
    }
    else{
        set<string>::iterator it;
        for (it=s.begin(); it!=s.end(); ++it){
            cout << ' ' << *it << "\n";
        }
    }
    cout << "\n";
}

//Exercise 2 (a) Check and correct if necessary
void computeParentNodes(Node *n, set<string> & parents){
    if(n->left != nullptr || n->right != nullptr){
        parents.insert(n->name);
    }
    computeParentNodes(n->left,parents);
    computeParentNodes(n->right,parents);
}

//Exercise 2 (b) Implement this function
void computeMembersOfSubTree(Node * n, set<string> & members){
    //Put your code here
}

```

EXERCISE 3. CURRENCY CONVERTER (2.5 POINTS)

Claire wants to implement a class `CurrencyConverter` to support some basic currency conversion functionalities, like converting from Danish krone (DKK) to EURO (EUR), and vice versa. Her first test program is in file `ex03-main.cpp` and the (incomplete) code with some functions she needs is in files `ex03-library.h` and `ex03-library.cpp`. All files are available on CampusNet (with extra comments) and in the next pages. Help Claire by implementing the class `CurrencyConverter` in file `ex03-library.cpp`.

Claire does not know how to implement the methods but she has been told that the `map` containers of the standard library already provide a lot of the functionalities she needs. So she has decided to use the following internal (`private`) representation for the library:

- `set<string> currencies`: The set of currently supported currencies. We store the code (e.g., DKK for Danish krone, EUR for EURO, and USD for US dollars).
- `map<string,double> currencyToExchangeRate`: A mapping from strings (representing the code of a currency) into doubles (representing the exchange rate from the currency to DKK).

The idea is that this currency converter is based on DKK. Everytime we want to add a new currency, we have to provide its exchange rate in DKK. In other words:

- `OTHER→DKK`: by multiplying an amount in a given currency by its exchange rate we get the corresponding amount in DKK
(intuitively, $DKK = exchangeRate * OTHER$).
- `DKK→OTHER`: by dividing an amount in DKK by the exchange rate of a given currency we get the corresponding amount in the given currency
(intuitively, $OTHER = DKK / exchangeRate$).

Claire already implemented the default constructor of `CurrencyConverter` where the converter is initialized with one currency only, DKK, with exchange rate 1. Claire also provided a method

```
bool supportsCurrency(string currencyCode)
```

which returns true if the currency with code `currencyCode` is currently supported, and false otherwise. Help Claire by performing the following tasks:

- (a) Check the implementation of method

```
void print()
```

and correct it if necessary. The method should correctly print information on the supported currencies. For each supported currency it should print a line (starting with an empty space) of this form:

Exercise follows in next page...

```
currency code has exchange rate exchangeRate
```

where `code` and `exchangeRate` are, respectively, the code and exchange rate of the considered currency.

(b) Check the implementation of method

```
bool addCurrency(string currencyCode, double exchangeRateToDKK)
```

and correct it if necessary. The method should add a new currency, together with its exchange rate to DKK. Special cases:

- If the currency is already supported, ignore the request, and return false.
- If the exchange rate is not positive, ignore the request, and return false.

If the request succeeds, the method should return true.

(c) Implement method

```
bool updateExchangeRate(string currencyCode, double newExchangeRate)
```

This method should update the exchange rate of an existing currency, and return true if none of the following special cases apply:

- If the currency is not supported, ignore the request, and return false.
- If the new exchange rate is not positive, ignore the request, and return false.
- If `currencyCode` is DKK, ignore the request and return false.

If the request succeeds, the method should return true.

(d) Implement method

```
double convertToDKK(double amount, string currencyCodeOfSource)
```

This method should convert (and return) `amount` from `currencyCodeOfSource` to DKK. Special cases:

- If the source currency is not supported, return -1.
- If the amount is not positive, return -1.

(e) Implement method

```
double convertFromDKK(double amountDKK, string currencyCodeOfTarget);
```

This method should convert (and return) `amount` from DKK to `currencyCodeOfTarget`. Special cases:

Exercise follows in next page...

- If the target currency is not supported, return -1.
- If the amount is not positive, return -1.

Hints about using maps:

- A key `k` in a map `m` can be updated (mapped) to `v` with `m[k] = v`;
- The value mapped to a key `k` in a map `m` is obtained with `m[k]`;
- The operator `m[k]` creates an entry for the key `k` if it is not present in the map `m`.
To check if the key is present you can use the test `m.find(k) != m.end()`.

Exercise follows in next page...

02393 Programming in C++

File ex03-main.cpp

```
#include <iostream>
#include "ex03-library.h"

int main() {
    CurrencyConverter cc;
    cout << "\naddCurrency(\"EUR\",7.44416);\n";
    cc.addCurrency("EUR",7.44416);
    cc.print();

    cout << "\naddCurrency(\"EUR\",17.44416);\n";
    cc.addCurrency("EUR",17.44416);
    cc.print();

    cout << "\naddCurrency(\"USD\",16.31708);\n";
    cc.addCurrency("USD",16.31708);
    cc.print();

    cout << "\nupdateExchangeRate(\"USD\",6.31708);\n";
    cc.updateExchangeRate("USD",6.31708);
    cc.print();

    cout << "\n\n";

    double amountDKK=100;
    double amountEUR=cc.convertFromDKK(amountDKK,"EUR");
    cout << amountDKK << "DKK=" << amountEUR << "EUR\n";

    amountDKK = cc.convertToDKK(amountEUR,"EUR");
    cout << amountEUR << "EUR=" << amountDKK << "DKK\n";

    return 0;
}
```

File ex03-library.h

```
#ifndef EX03_LIBRARY_H_
#define EX03_LIBRARY_H_

#include <string>
#include <map>
#include <set>
using namespace std;

class CurrencyConverter {
private:
    set<string> currencies;
    map<string,double> currencyToExchangeRate;
public:
    CurrencyConverter();
    bool supportsCurrency(string currencyCode);
    void print();
    bool addCurrency(string currencyCode, double exchangeRateToDKK);
    bool updateExchangeRate(string currencyCode, double newExchangeRate);
    double convertToDKK(double amount,string currencyCodeOfSource);
    double convertFromDKK(double amountDKK,string currencyCodeOfTarget);
};

#endif /* EX03_LIBRARY_H_ */
```

Exercise follows in next page...

02393 Programming in C++

File ex03-library.cpp

```
#include <iostream>
#include "ex03-library.h"

//Do not modify
CurrencyConverter::CurrencyConverter() {
    currencies.insert("DKK");
    currencyToExchangeRate["DKK"]=1;
}

//Do not modify
bool CurrencyConverter::supportsCurrency(string currencyCode){
    if(currencies.find(currencyCode) != currencies.end()){
        //I have the currency.
        return true;
    }
    else{
        return false;
    }
}

//Exercise 3 (a) Check and correct if necessary
void CurrencyConverter::print(){
    cout << "The converter supports the following currencies:"<<endl;
    for (map<string,double>::iterator it=currencyToExchangeRate.begin(); it!=currencyToExchangeRate.end(); ++it){
        cout << ' ' << "currency_" << it->second << " has exchange rate_" << it->first << endl;
    }
}

//Exercise 3 (b) Check and correct if necessary
bool CurrencyConverter::addCurrency(string currencyCode,double exchangeRateToDKK) {
    if(supportsCurrency(currencyCode)){
        //I already have this element. Hence I return false
        return false;
    }
    else if(exchangeRateToDKK <= 0){
        //Exchange rates must be positive
        return false;
    }
    else{
        currencies.insert(currencyCode);
        currencyToExchangeRate[exchangeRateToDKK]=currencyCode;
        return true;
    }
}

//Exercise 3 (c) Implement this function
bool CurrencyConverter::updateExchangeRate(string currencyCode,double newExchangeRate) {
    //Put your code here
}

//Exercise 3 (d) Implement this function
double CurrencyConverter::convertToDKK(double amount, string currencyCodeOfSource) {
    //Put your code here
}

//Exercise 3 (e) Implement this function
double CurrencyConverter::convertFromDKK(double amountDKK,string currencyCodeOfTarget) {
    //Put your code here
}
```

EXERCISE 4. PARAMETRIC DOUBLE-ENDED QUEUE (2.5 POINTS)

Daisy attended the course 02393 Programming in C++ at DTU. Taking inspiration from some of the examples seen in class, she decided to implement a queue (i.e. a list) in which it is possible to add elements both at the end (`push_back`, like for vectors), and at the beginning (`push_front`). Data structures with these functionalities are known as double-ended queues, or **deques**.

Daisy is implementing a C++ library for manipulating parametric deques. She has prepared a test program (`ex04-main.cpp`), the declaration of class `mydeque` (`ex04-library.h`) and a sketch of its implementation (`ex04-library.cpp`). All files are available on CampusNet (with further comments) and in the next pages. Daisy has decided to represent an entry of the deque using the parametric struct `Node` containing two fields:

- `T content`: the element of (parametric) type `T` in that position of the deque.
- `Node<T> * next`: pointer to the `Node` storing the next element of the deque.

Daisy has decided to use the following internal (`private`) representation for deques:

- `Node<T> * first`: pointer to the first element of the deque.
- `Node<T> * last`: pointer to the last element of the deque.
- `int size`: the current number of elements stored in the deque.

If the deque is empty, then both `first` and `last` have value `nullptr`. If the deque has size one, then `first` and `last` point to the same `Node`.

Daisy has already implemented several functions, including:

- a constructor and a destructor.
- a method `push_when_empty(T v)` to insert elements in an empty deque.¹
- public methods to print the value stored in the first (`print_front()`) and last position of the deque (`print_back()`), and to print information on the list (`print()`).

Help Daisy implementing class `mydeque` in file `ex04-library.cpp`. Your tasks are:

- (a) Check the class destructor and correct it if necessary. This method should deallocate all memory used by each of its `Node` structs.
- (b) Implement the method `push_back(T v)`. This method should add an element `v` at the end of the deque. Hence `v` becomes the last element of the deque. Special cases:
 - If the method is invoked on a empty deque, then this method just invokes the method `push_when_empty(T v)`

Exercise follows in next page...

¹This method should be private, but we made it public to ease the development of tests.

- (c) Implement the method `push_front(T v)`. This method should add an element `v` at the beginning of the deque. Hence `v` becomes the first element of the deque. Special cases:

- If the method is invoked on an empty deque, then this method just invokes the method `push_when_empty(T v)`

- (d) Check method `print_back()`, and correct it if necessary. The method should print the last element of the deque (if any). Similarly to method `print`, if the method is invoked on an empty deque, then it should print on screen using command:

```
cout << "The deque is empty.\n";
```

- (e) Complete the implementation of method `pop_front()`. This method should remove the first element in the deque, effectively reducing the deque size by one. The memory allocated to store the element in the deque should be deallocated. The method should return `true` if the deque contained at least an element, meaning that the pop succeeded. Instead, it should return `false` if the deque was empty, and hence the pop failed. Special cases:

- If the method is invoked on an empty deque, nothing is done, and the value `false` is returned.
- If the method is invoked on a deque with one element, the deque becomes empty, and the value `true` is returned.

Daisy already wrote the code necessary for the special cases. Help Daisy by providing the code for the general case of deques with 2 or more elements.

Exercise follows in next page...

02393 Programming in C++

File ex04-main.cpp

```
#include <iostream>
#include "ex04-library.h"

using namespace std;

int main() {
    mydeque<int> deque;

    deque.push_front(2);
    deque.push_front(1);
    deque.push_back(3);

    deque.print();

    cout << "\n";
    cout << "The first element is: ";
    deque.print_front();
    cout << "The last element is: ";
    deque.print_back();

    do{
        cout << "\n";
        cout << "pop_front(): ";
        cout << deque.pop_front() << "\n";
        deque.print();
    }while(deque.getSize()>0);

    cout << "\n";
    cout << "pop_front(): ";
    cout << deque.pop_front() << "\n";

    return 0;
}
```

Exercise follows in next page...

File ex04-library.h

```
#ifndef EX04_LIBRARY_H_
#define EX04_LIBRARY_H_

template<class T>
struct Node{
    T content;
    Node<T> * next;
};

template <class T>
class mydeque {
private:
    Node<T> * first;
    Node<T> * last;
    int size;
public:
    mydeque();
    void push_when_empty(T v);
    ~mydeque();
    int getSize();
    void print_front();
    void print_back();
    void print();
    void push_back(T v);
    void push_front(T v);
    bool pop_front();
};
#endif
```

File ex04-library.cpp

```

#include "ex04-library.h"
#include <iostream>

using namespace std;

//Do not modify
template<class T>
mydeque<T>::mydeque() {
    size=0;
    first=nullptr;
    last=nullptr;
}

//Do not modify
template<class T>
int mydeque<T>::getSize() {
    return size;
}

//Do not modify
template<class T>
void mydeque<T>::print_front() {
    if(size==0){
        cout << "The deque is empty.\n";
    }
    else{
        cout << first->content << "\n";
    }
}

//Do not modify
template<class T>
void mydeque<T>::print() {
    if(size==0){
        cout << "The deque is empty.\n";
    }
    else{
        cout << "The deque has size " << size << ":\n";
        Node<T> * current = first;
        while(current!=nullptr){
            cout << " " << current->content << "\n";
            current = current->next;
        }
    }
}

//Do not modify
template<class T>
void mydeque<T>::push_when_empty(T v) {
    Node<T> * node = new Node<T>;
    node->content=v;
    node->next=nullptr;
    first=node;
    last=node;
    size=1;
}

//Exercise 4 (a) Check and correct if necessary
template<class T>
mydeque<T>::~mydeque() {
    Node<T> * current = first;
    while(current!=nullptr){
        delete current;
        Node<T> * next = current->next;
        current = next;
    }
    cout << "Destructor completed\n";
}

//Exercise 4 (b) Implement this function
template<class T>
void mydeque<T>::push_back(T v) {
    //Put your code here
}

//Exercise 4 (c) Implement this function
template<class T>
void mydeque<T>::push_front(T v) {
    //Put your code here
}

//Exercise 4 (d) Check and correct if necessary
template<class T>
void mydeque<T>::print_back() {
    if(size!=0){
        cout << "The deque is empty.\n";
    }
    else{
        cout << first->content << "\n";
    }
}

//Exercise 4 (e) Complete body of last else
template<class T>
bool mydeque<T>::pop_front() {
    if(size==0){
        //Cannot pop from an empty deque. I return false;
        return false;
    }
    else if(size==1){
        //Since size is 1, the deque becomes empty
        delete first;
        first=nullptr;
        last=nullptr;
        size=0;
        return true;
    }
    else{
        //The deque has at least 2 elements.
        //Put your code here
    }
}

//Do not modify
template class mydeque<int>;

```



Technical University of Denmark

Written examination, December 09, 2018

Page 1 of 15 pages

Course name: Programming in C++

Course number: 02393

Aids allowed: All aids allowed

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises of 2.5 points each for a total of 10 points.

Submission details:

1. You can hand-in your solutions manually (on paper). However, we strongly recommend you to submit them electronically.
2. For electronic submission, you **must** upload your solutions on CampusNet and you can do it only once: resubmission is not possible, so submit only when you have finished all exercises. Each exercise must be uploaded as one separate `.cpp` file, using the names specified in the exercises, namely `exZZ-library.cpp`, where `ZZ` ranges from `01` to `04`. The files must be handed in separately (not as a zip-file) and must have these exact filenames. Feel free to add comments to your code.
3. You can also upload your solutions on CodeJudge under **Exam December 2018** at <https://dtu.codejudge.net/02393-e18/exercises>. This is not an official submission, and will not be considered for evaluation. When you hand in a solution on CodeJudge, some tests will be run on it. Additional tests may be run on your solutions after the exam. You can upload to CodeJudge as many times as you like.

EXERCISE 1. TEMPERATURE SCALES CONVERTER (2.5 POINTS)

Alice needs to implement a library to manipulate a data set of temperatures registered in Copenhagen. This is stored as an array where each entry takes a double value corresponding to a temperature in Celsius scale (e.g., water freezes at 0°C , and boils at 100°C in standard pressure conditions).

Alice needs to perform some computations on such data. She has already implemented part of the code but she is not sure about its correctness, and some parts are still missing. Her first test program is in file `ex01-main.cpp` and the (incomplete) code with some functions she needs is in files `ex01-library.h` and `ex01-library.cpp`. All files are available on CampusNet and in the next pages. Help Alice by solving the following tasks:

- (a) Check the implementation of function

```
double * createAndInitArray(unsigned int n, double value)
```

and correct it if necessary. The function should correctly create an array of doubles of length `n`, and return it. The function should allocate the required memory, and initialize each entry of the array to `value`.

- (b) Implement function

```
double * duplicateArray(double * A, unsigned int n);
```

The function should create and return a copy of the array `A` of length `n`.

- (c) Implement function

```
void deallocateArray(double * A);
```

This function is the dual of `createArray`. It should deallocate all memory allocated for the array `A`.

Exercise follows in next page...

(d) Implement function

```
double * toFahrenheit(double * A, unsigned int n);
```

This function transforms the data set from the Celsius scale to the Fahrenheit scale. The function takes as input an array A of length n , and returns a new array of same length containing the measurements in A converted in Fahrenheit scale. The function should not modify A . Each value v_C in Celsius scale has to be transformed in the corresponding value v_F in Fahrenheit scale computed as follows:

$$v_F = v_C * 1.8 + 32$$

Notes:

- The function has to apply the conversion on a copy, let's say B , of A . Such modified copy B has to be returned.
- The function must not modify A .

For example, the data set $A = [-10, 0, 5]$ in Celsius scale is converted in the data set B in Fahrenheit scale:

$$B = [-148, 32, 122]$$

because

$$(-10) * 1.8 + 32 = -148$$

$$0 * 1.8 + 32 = 32$$

$$5 * 1.8 + 32 = 122$$

Exercise follows in next page...

File ex01-main.cpp

```

#include <iostream>
#include <string>
#include "ex01-library.h"

using namespace std;

int main(void){

    // Array containing Celsius
    // temperatures initialized to 100
    unsigned int n = 4;
    double *A = createAndInitArray(n,100);

    // Setting some values in the array
    unsigned int m = 3;
    for(int i=0;i<n-1;i++){
        for(int j=0;j<m-1;j++){
            A[i]+=i*n+j*3;
        }
    }
    printArray(A,n,"main_array");

    double * B = duplicateArray(A,n);
    printArray(B,n,"copy");

    //I change B, and I print B and A
    B[0] = B[0]+3;
    B[1] = B[0]+3;
    B[2] = B[1]+3;

    printArray(B,n,"modified_copy");
    printArray(A,n,"main_array");

    //I deallocate B
    deallocateArray(B);

    //I convert to Fahrenheit
    double * C = toFahrenheit(A,n);
    printArray(C,n,"Fahrenheit_copy");
    printArray(A,n,"main_array_not_modified");

    //I deallocate A and C
    deallocateArray(A);
    deallocateArray(C);

    cout << "Completed"<<endl;
    return 0;
}

```

File ex01-library.h

```

#ifndef EX01_LIBRARY_H_
#define EX01_LIBRARY_H_

#include <vector>
#include <string>
using namespace std;

double * createAndInitArray(unsigned int n, double value);
double * duplicateArray(double * A, unsigned int n);
void deallocateArray(double * A);
double * toFahrenheit(double * A, unsigned int n);
void printArray(double * A, unsigned int n, string descr);

#endif /* EX01_LIBRARY_H_ */

```

File ex01-library.cpp

```

#include <iostream>
#include "ex01-library.h"

using namespace std;

//Exercise 1 (a) Check and correct if necessary
double * createAndInitArray(unsigned int n, double value){
    double * A = new double;
    for(unsigned int i = 0; i<=value+1; i++){
        A[i] = value;
    }
    return A;
}

//Exercise 1 (b) Implement this function
double * duplicateArray(double * A, unsigned int n){
    //put your code here
}

//Exercise 1 (c) Implement this function
void deallocateArray(double * A){
    //put your code here
}

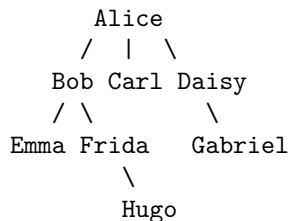
//Exercise 1 (d) Implement this function
double * toFahrenheit(double * A, unsigned int n){
    //put your code here
}

//Do not modify
void printArray(double * A, unsigned int n, string descr){
    cout<< "Printing:\n" << descr << endl;
    for(unsigned int i = 0; i < n; i++){
        cout << A[i] << '\n';
    }
    cout << "\n\n";
}

```

EXERCISE 2. GENEALOGY (2.5 POINTS)

Bob is interested in studying the genealogy of his family. So far he has built the following family tree:



Alice had three children: Bob, Carl, and Daisy. Bob had two children: Emma, and Frida, while Daisy had one, Gabriel, and Frida had one, Hugo. Finally, Carl, Emma, Gabriel, and Hugo had no children.

Bob would like to automate the computation of simple queries on family trees, such as the computation of all individuals that had children (i.e., that are *parents*), or the computation of the *descendants* of an individual. Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. All files are available on CampusNet and in the next pages.

Bob has decided to represent family trees using tree data structures, where each node can have any number of children. Each node of the tree is stored in an object of class `Node` containing:

- **name:** the name of the corresponding individual;
- **children:** a vector of pointers to other `Node` objects, representing the children of the corresponding individual. The vector is empty if the individual had no children.

Help Bob by solving the following tasks:

(a) Implement the constructor

```
Node(string name)
```

which just sets the given parameter, `string name`, as the name of the node. Remember that you don't need to initialize vectors because they are initialized automatically. Implement the getter method

```
string getName()
```

which just returns the name of the node.

(b) Implement method

```
void addChild(Node * child)
```

which adds `child` to the vector `children` of the node.

Exercise follows in next page...

- (c) Check the implementation of method

```
void printParentNodes()
```

and correct it if necessary. A *parent node* is a node with at least one child. This method should correctly print all parent nodes met while navigating the tree starting from node **n** on which the method has been invoked (including **n** itself, in case it has children). In particular, if the node has at least one child the method should:

- print the name of the current node, followed by a blank space (the char '␣')
- after this, the method is recursively invoked on the children of the current node, following the order in which they appear in the vector **children**.

For instance, invoking **printParentNodes** on Alice's node we get:

```
Alice Bob Frida Daisy
```

Instead, invoking the method on Bob's node we get:

```
Bob Frida
```

- (d) Implement method

```
void printMembersOfSubTree(int generation=0)
```

This method should print the names of all members of the sub-tree starting from node **n** on which the method has been invoked (including **n** itself). Also, for each considered node it should compute the *generation* of the node with respect to the node **n** on which the method has been originally invoked. E.g., if we invoke the method on **Alice**, then she is in generation 0, while her sons Bob, Carl and Daisy are in generation 1, her grandsons are in generation 2, and so on.

In particular, for each considered node, the method should print the name of the node, followed by a blank space (i.e., the char '␣'), followed by the generation of the node with respect to the node **n** on which the method has been originally invoked, followed by another blank space. After this, the method should be recursively invoked on the children of the current node, following the order in which they appear in the vector **children**. Note that the parameter **generation** has default value 0, meaning that invoking **printMembersOfSubTree()** is equivalent to **printMembersOfSubTree(0)**.

For instance, invoking **printMembersOfSubTree** on Alice's node we get:

```
Alice 0 Bob 1 Emma 2 Frida 2 Hugo 3 Carl 1 Daisy 1 Gabriel 2
```

Instead, invoking **printMembersOfSubTree** on Bob's node we get:

```
Bob 0 Emma 1 Frida 1 Hugo 2
```

Exercise follows in next page...

File ex02-main.cpp

```

#include <iostream>
#include "ex02-library.h"
using namespace std;

int main() {
    /* Bob's family tree from text */
    Node *Alice = new Node("Alice");
    Node *Bob = new Node("Bob");
    Node *Carl = new Node("Carl");
    Node *Daisy = new Node("Daisy");
    Node *Emma = new Node("Emma");
    Node *Frida = new Node("Frida");
    Node *Gabriel = new Node("Gabriel");
    Node *Hugo = new Node("Hugo");

    cout << "Alice's_name:_" << Alice->getName();
    cout << endl << endl;

    Alice->addChild(Bob);
    Alice->addChild(Carl);
    Alice->addChild(Daisy);
    Bob->addChild(Emma);
    Bob->addChild(Frida);
    Daisy->addChild(Gabriel);
    Frida->addChild(Hugo);

    cout << "Experiments_about_parent_nodes";
    cout << endl << "Alice:_" ;
    Alice->printParentNodes();
    cout << endl << "Bob:_" ;
    Bob->printParentNodes();
    cout << endl << "Carl:_" ;
    Carl->printParentNodes();
    cout << endl << "Daisy:_" ;
    Daisy->printParentNodes();
    cout << endl << "Emma:_" ;
    Emma->printParentNodes();
    cout << endl << "Frida:_" ;
    Frida->printParentNodes();
    cout << endl << endl;

    cout << "Experiment_about_sub-tree";
    cout << endl << "Alice:_" ;
    Alice->printMembersOfSubTree();
    cout << endl << "Bob:_" ;
    Bob->printMembersOfSubTree();
    cout << endl << "Carl:_" ;
    Carl->printMembersOfSubTree();
    cout << endl << "Daisy:_" ;
    Daisy->printMembersOfSubTree();
    cout << endl << "Emma:_" ;
    Emma->printMembersOfSubTree();
    cout << endl << "Frida:_" ;
    Frida->printMembersOfSubTree();
    cout << endl << "Gabriel:_" ;
    Gabriel->printMembersOfSubTree();
    cout << endl << "Hugo:_" ;
    Hugo->printMembersOfSubTree();
    cout << endl;

    return 0;
}

```

File ex02-library.h

```

#ifndef EX02_LIBRARY_H_
#define EX02_LIBRARY_H_

#include <vector>
#include <string>

using namespace std;

class Node{
private:
    string name;
    vector<Node *> children;
public:
    int countChildren();
    Node(string name);
    string getName();
    void addChild(Node * child);
    void printParentNodes();
    void printMembersOfSubTree(int generation=0);
};

#endif

```

File ex02-library.cpp

```

#include "ex02-library.h"
#include <iostream>

//Do not modify
int Node::countChildren(){
    return children.size();
}

//Exercise 2 (a) Implement the constructor and getName()
Node::Node(string name){
    //put your code here
}

string Node::getName(){
    //put your code here
}

//Exercise 2 (b) Implement this method
void Node::addChild(Node * child){
    //put your code here
}

//Exercise 2 (c) Check and correct if necessary
void Node::printParentNodes(){
    for(int i=0; i<children.size()-1;i++){
        children[i]->printParentNodes();
    }
    cout << this->name << '_';
}

//Exercise 2 (d) Implement this method
void Node::printMembersOfSubTree(int generation){
    //put your code here
}

```

EXERCISE 3. TEMPERATURES SCALES CONVERTER 2 (2.5 POINTS)

Claire wants to implement a class `TemperatureScalesConverter` to support the storing of temperature measurements in different temperature scales: Celsius (C), and Fahrenheit (F). Her first test program is in file `ex03-main.cpp` and the (incomplete) code with some functions she needs is in files `ex03-library.h` and `ex03-library.cpp`. All files are available on CampusNet and in the next pages. Help Claire by implementing the class `TemperatureScalesConverter` in file `ex03-library.cpp`.

Claire decided to use the `vector` containers of the standard library. So she has decided to use the following internal (`private`) representation for the library:

- `vector<double> CTemperatures`: A vector containing the temperature measurements in Celsius scale.
- `vector<double> FTemperatures`: A vector containing the temperature measurements in Fahrenheit scale.

The general idea is that the converter supports two scales: Celsius and Fahrenheit. Users can add new measurements in any of the two scales. Each measurement is converted to the remaining scale and added to both vectors. For example, if the user adds a Celsius measurement, then (i) it will be added to `CTemperatures`, (ii) it will be converted to Fahrenheit, and (iii) it will be added to `FTemperatures`.

Claire already implemented the default constructor of `TemperatureScalesConverter`, which adds a default measurement of 0 Celsius, equivalent to 32 Fahrenheit. Help Claire by performing the following tasks:

- (a) Check the implementation of method

```
void print()
```

and correct it if necessary. The method should correctly print information on the added measurements in this form:

```
n Celsius measurements: Cmeasurement1 Cmeasurement2 ... Cmeasurementn
n Fahrenheit measurements: Fmeasurement1 Fmeasurement2 ... Fmeasurementn
```

where `n` is the number of measurements added, `Cmeasurementi` is the `i`-th Celsius measurement, and `Fmeasurementi` is the `i`-th Fahrenheit measurement. Note that each measurement is followed by a blank space.

Exercise follows in next page...

02393 Programming in C++

(b) Implement method

```
double convertToF(double CTemperature)
```

which should convert the Celsius measurement `CTemperature` in Fahrenheit scale and return it.

A Celsius measurement v_C has to be converted to the corresponding value v_F in Fahrenheit scale as follows:

$$v_F = v_C * 1.8 + 32$$

(c) Implement method

```
double convertToC(double FTemperature)
```

which should convert the Fahrenheit measurement `FTemperature` in Celsius scale and return it.

A Fahrenheit measurement v_F has to be converted¹ to the corresponding value v_C in Celsius scale as follows:

$$v_C = (v_F - 32) * 0.56$$

(d) Implement method

```
bool addMeasurement(string scale, double temperature)
```

where `scale` denotes a temperature scale, i.e. `"C"` for Celsius and `"F"` for Fahrenheit, and `temperature` is a measurement in the scale denoted by the first parameter. This method should add `temperature` to the correct vector. Then, it should convert `temperature` in the other scale, and add it to the corresponding vector. Special cases:

- If `scale` is neither `"C"` nor `"F"`, the method should do nothing and return false. Otherwise the method should do what required, and return true.

Exercise follows in next page...

¹The actual formula involves 0.55, here rounded to 0.56

02393 Programming in C++

File ex03-main.cpp

```
#include <iostream>
#include "ex03-library.h"

int main() {
    TemperatureScalesConverter tsc;
    cout << "Default measurement of 0C = 32F:" << endl;
    tsc.print();
    cout << endl;

    double f = tsc.convertToF(20);
    cout << "20Celsius = " << f << "Fahrenheit" << endl;
    double c = tsc.convertToC(30);
    cout << "30Fahrenheit = " << c << "Celsius" << endl;

    bool res = tsc.addMeasurement("C", 20);
    if(!res){
        cout << "tsc.addMeasurement(\"C\", 20); FAILED" << endl;
    }
    res = tsc.addMeasurement("F", 30);
    if(!res){
        cout << "tsc.addMeasurement(\"F\", 30); FAILED" << endl;
    }
    tsc.print();
    cout << endl;

    res = tsc.addMeasurement("K", 50);
    if(!res){
        cout << "tsc.addMeasurement(\"K\", 50); FAILED" << endl;
    }
    tsc.print();

    return 0;
}
```

File ex03-library.h

```
#ifndef EX03_LIBRARY_H_
#define EX03_LIBRARY_H_

#include <string>
#include <vector>
using namespace std;

class TemperatureScalesConverter {
private:
    vector<double> CTemperatures;
    vector<double> FTemperatures;
public:
    TemperatureScalesConverter();
    void print();
    double convertToF(double CTemperature);
    double convertToC(double FTemperature);
    bool addMeasurement(string scale, double temperature);
};

#endif /* EX03_LIBRARY_H_ */
```

Exercise follows in next page...

02393 Programming in C++

File ex03-library.cpp

```
#include <iostream>
#include "ex03-library.h"

//Do not modify
TemperatureScalesConverter::TemperatureScalesConverter() {
    //By default we add a measurement of 0 C which corresponds to 32 F.
    CTemperatures.push_back(0);
    FTemperatures.push_back(32);
}

//Exercise 3 (a) Check and correct this method
void TemperatureScalesConverter::print(){
    cout << CTemperatures.size() << "\nCelsius measurements:";
    for(int i = 0; i < CTemperatures.size() - 1; i++){
        cout<< ' ' << CTemperatures[i];
    }
    cout << endl;

    cout << FTemperatures.size() << "\nFahrenheit measurements:";
    for(int i = CTemperatures.size() - 1; i > 0; i--){
        cout<< ' ' << CTemperatures[i];
    }
    cout << endl;
}

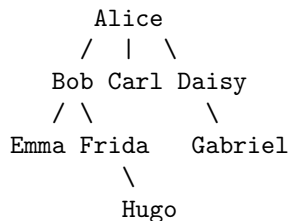
//Exercise 3 (b) Implement this method
double TemperatureScalesConverter::convertToF(double CTemperature){
    //put your code here
}

//Exercise 3 (c) Implement this method
double TemperatureScalesConverter::convertToC(double FTemperature){
    //put your code here
}

//Exercise 3 (d) Implement this method
bool TemperatureScalesConverter::addMeasurement(string scale, double temperature){
    //put your code here
}
```


EXERCISE 4. GENEALOGY 2 (2.5 POINTS)

Bob is interested in studying the genealogy of his family. So far he has built the following family tree:



Alice had three children: Bob, Carl, and Daisy. Bob had two children: Emma, and Frida, while Daisy had one, Gabriel, and Frida had one, Hugo. Finally, Carl, Emma, Gabriel, and Hugo had no children.

Bob has already written some code. His first test program is in file `ex04-main.cpp` and the (incomplete) code with some functions he needs is in files `ex04-library.h` and `ex04-library.cpp`. All files are available on CampusNet and in the next pages.

Bob would like to automate the computation of queries on family trees, like the computation of all individuals that had no children (i.e., *leaf* nodes), or counting of the number of *descendants* of an individual. For each individual, Bob wants to store his name. In order to make the code more general, Bob also wants to enrich each individual with a second information, of parametric type (omitted in the above diagram, but shown later in file `ex04-main.cpp`). This can be done using C++ generic programming, i.e. using templates.

Bob decided to represent family trees using trees, where each node can have any number of children. Each node is stored in an object of parametric class `Node<T>` containing:

- **string name:** the name of the corresponding individual;
- **T value:** a second information of parametric type `T` of the corresponding individual;
- **children:** a vector of pointers to `Node<T>` objects. These are the children of the corresponding individual. This vector is empty if the individual had no children.

Help Bob by solving the following tasks:

- (a) Check and correct the implementation of method

```
Node(string name,T value)
```

which sets the **name** and **value** given as parameters as the name and value of the node, respectively. Remember that you don't need to initialize vectors because they are initialized automatically.

Exercise follows in next page...

- (b) Implement the methods `getName`, `getValue`, and `addChild`. The method

```
string getName()
```

returns the name of the node. Instead,

```
T getValue()
```

returns the value of the node. While

```
void addChild(Node<T> * child)
```

adds `child` to the vector `children` of the node.

- (c) Implement method

```
void printLeafNodes()
```

A *leaf node* is a node without children. This method should print all leaf nodes met while navigating the tree starting from node `n` on which the method has been invoked (including `n` itself, in case it has no children). When invoked on non-leaf nodes, the method should call recursively itself on the children of the current node, following the order in which they appear in the vector `children`. Instead, when invoked on leaf nodes, the method should print the **name** of the current node, followed by a blank space (i.e., the char `'␣'`), followed by **value**, followed by another blank space.

For instance, invoking `printParentNodes` on Alice's node we get:

```
Emma 01/01/1998 Hugo 01/01/2018 Carl 01/01/1978 Gabriel 01/02/1998
```

Instead, invoking the method on Carl's node we get:

```
Carl 01/01/1978
```

As you can see, in this example the **value** of each node is the birth date of the corresponding individual, as shown in `ex04-main.cpp` in the next page.

- (d) Implement method

```
int countMembersOfSubTree()
```

This method should count the members of the sub-tree starting from node `n` on which the method has been invoked (including `n` itself). For instance, invoking `countMembersOfSubTree` on Alice's and Bob's node we get 8 and 4, respectively.

Exercise follows in next page...

02393 Programming in C++

File ex04-main.cpp

```
#include <iostream>
#include "ex04-library.h"

using namespace std;

int main() {
    /* Bob's family tree from text */
    Node<string> *Alice = new Node<string>("Alice", "01/01/1957");
    Node<string> *Bob = new Node<string>("Bob", "01/01/1977");
    Node<string> *Carl = new Node<string>("Carl", "01/01/1978");
    Node<string> *Daisy = new Node<string>("Daisy", "01/12/1978");
    Node<string> *Emma = new Node<string>("Emma", "01/01/1998");
    Node<string> *Frida = new Node<string>("Frida", "01/12/1998");
    Node<string> *Gabriel = new Node<string>("Gabriel", "01/02/1998");
    Node<string> *Hugo = new Node<string>("Hugo", "01/01/2018");

    cout << "Alice's_name_is:_" << Alice->getName() << endl;
    cout << "Alice's_value_is:_" << Alice->getValue() << endl;

    Alice->addChild(Bob);
    Alice->addChild(Carl);
    Alice->addChild(Daisy);
    Bob->addChild(Emma);
    Bob->addChild(Frida);
    Daisy->addChild(Gabriel);
    Frida->addChild(Hugo);

    cout << endl;
    cout << "Experiments_about_leaf_nodes";
    cout << endl << "Alice:_" ;
    Alice->printLeafNodes();
    cout << endl << "Bob:_" ;
    Bob->printLeafNodes();
    cout << endl << "Carl:_" ;
    Carl->printLeafNodes();
    cout << endl << "Daisy:_" ;
    Daisy->printLeafNodes();
    cout << endl << "Emma:_" ;
    Emma->printLeafNodes();
    cout << endl << "Frida:_" ;
    Frida->printLeafNodes();
    cout << endl;

    cout << endl;
    cout << "Experiment_about_sub-tree";
    cout << endl << "Alice:_" ;
    cout << Alice->countMembersOfSubTree();
    cout << endl << "Bob:_" ;
    cout << Bob->countMembersOfSubTree();
    cout << endl << "Carl:_" ;
    cout << Carl->countMembersOfSubTree();
    cout << endl << "Daisy:_" ;
    cout << Daisy->countMembersOfSubTree();
    cout << endl << "Emma:_" ;
    cout << Emma->countMembersOfSubTree();
    cout << endl << "Frida:_" ;
    cout << Frida->countMembersOfSubTree();
    cout << endl;

    return 0;
}
```

Exercise follows in next page...

File ex04-library.h

```

#ifndef EX04_LIBRARY_H_
#define EX04_LIBRARY_H_

#include <vector>
#include <string>

using namespace std;

template<class T>
class Node{
private:
    string name;
    T value;
    vector<Node *> children;
public:
    void printNode();
    Node(string name, T value);
    string getName();
    T getValue();
    void addChild(Node * child);
    void printLeafNodes();
    int countMembersOfSubTree();
};

#endif

```

File ex04-library.cpp

```

#include "ex04-library.h"
#include <iostream>

using namespace std;

//Do not modify
template<class T>
void Node<T>::printNode(){
    cout << "Name:_" << name << ",_value:_" << value ;
    cout << ",_children:_" << children.size() << endl;
}

//Exercise 4 (a) Check and correct if necessary
template<class T>
Node<T>::Node(string name, T value){
}

//Exercise 4 (b) Implement getName, getValue, and addChild
template<class T>
string Node<T>::getName(){
    //put your code here
}

template<class T>
T Node<T>::getValue(){
    //put your code here
}

template<class T>
void Node<T>::addChild(Node<T> * child){
    //put your code here
}

//Exercise 4 (c) Implement this method
template<class T>
void Node<T>::printLeafNodes(){
    //put your code here
}

//Exercise 4 (d) Implement this method
template<class T>
int Node<T>::countMembersOfSubTree(){
    //put your code here
}

//Do not modify
template class Node<string>;

```



Technical University of Denmark

Written examination, May 14, 2018

Page 1 of 16 pages

Course name: Programming in C++

Course number: 02393

Aids allowed: All aids allowed

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises of 2.5 points each for a total of 10 points.

Submission details:

1. You can hand-in your solutions manually (on paper). However, we strongly recommend you to submit them electronically.
2. For electronic submission, you **must** upload your solutions on CampusNet and you can do it only once: resubmission is not possible, so submit only when you have finished all exercises. Each exercise must be uploaded as one separate `.cpp` file, using the names specified in the exercises, namely `exZZ-library.cpp`, where ZZ ranges from 01 to 04. The files must be handed in separately (not as a zip-file) and must have these exact filenames. Feel free to add comments to your code.
3. You can additionally upload your solutions on CodeJudge under **ReExam May 2018** at <https://dtu.codejudge.net/02393-e17/exercisegroup>. This is not an official submission, and will not be considered for evaluation. When you hand in a solution on CodeJudge, some tests will be run on it. Additional tests may be run on your solutions after the exam. You can upload to CodeJudge as many times as you like.

EXERCISE 1. GRAPHICS EDITOR (2.5 POINTS)

Alice needs to implement a library to manipulate black/white pictures, stored as matrices with one entry per pixel. Each entry takes integer values from 0 to 255:

- 0 corresponds to black,
- 255 corresponds to white
- intermediate values correspond to tonalities of grey.

Alice needs to perform some computations on such matrices. She has already implemented part of the code but she is not sure about its correctness, and some parts are still missing. Her first test program is in file `ex01-main.cpp` and the (incomplete) code with some functions she needs is in files `ex01-library.h` and `ex01-library.cpp`. All files are available on CampusNet (with additional comments) and in the next pages. Help Alice by solving the following tasks:

(a) Check the implementation of function

```
unsigned int ** createMatrix(int n, int m);
```

and correct it if necessary. The function should correctly create an $n \times m$ matrix of `unsigned int` elements, and return it. The function should only allocate the required memory. Initialization of the entries of the matrix is not required.

Notice that Alice has decided to represent a $n \times m$ matrix as an array (of size `n`) of arrays (each of size `m`) of `unsigned int` elements. Recall that in such a representation the element at row `r` and column `c` in a matrix `A` is accessed by `A[r][c]`.

(b) Check the implementation of function

```
void deallocateMatrix(unsigned int ** A, int n);
```

This function is the dual of `createMatrix`. It should deallocate all memory allocated for the matrix `A`.

(c) Implement function

```
void fillWhite(unsigned int ** A, int n, int m);
```

The function should set to white (i.e., to value 255) all entries of the $n \times m$ matrix `A`.

Exercise follows in next page...

(d) Implement function

```
void lighten(unsigned int ** A, int n, int m);
```

This filter makes the picture brighter: each pixel's value gets incremented by 1 (preserving the range 0-255). This function should scan all elements of the matrix **A** given as parameter, and

- increase by one all values smaller than 255.
- leave unchanged all values equal to 255.

(e) Implement function

```
void darken(unsigned int ** A, int n, int m);
```

This filter makes the picture darker: each pixel's value gets decremented by 1 (preserving the range 0-255). This function should scan all elements of the matrix **A** given as parameter, and

- decrease by one all values greater than 0.
- leave unchanged all values equal to 0.

Exercise follows in next page...

File ex01-main.cpp

```
#include <iostream>
#include <string>
#include "ex01-library.h"

using namespace std;

int main(void){

    // Matrix representing a picture
    int n = 2;
    int m = 3;
    unsigned int **A = createMatrix(n,m);

    // Setting all values to 255 (white)
    fillWhite(A,n,m);
    // Setting some values in the matrix
    for(int i=0;i<n-1;i++){
        for(int j=0;j<m;j++){
            A[i][j]=i*n+j;
        }
    }

    // Printing the matrix
    printMatrix(A,n,m,"original_matrix");
    cout << endl;

    //Making the figure brighter
    for(int i=1;i<3;i++){
        lighten(A,n,m);
        printMatrix(A,n,m,"brighter_matrix_" +
            to_string(i)+"");
        cout << endl;
    }

    //Making the figure darker
    for(int i=1;i<4;i++){
        darken(A,n,m);
        printMatrix(A,n,m,"darker_matrix_" +
            to_string(i)+"");
        cout << endl;
    }

    //Deallocating A
    deallocateMatrix(A,n);

    return 0;
}
```

File ex01-library.h

```
#ifndef EX01_LIBRARY_H_
#define EX01_LIBRARY_H_

#include <vector>
#include <string>
using namespace std;

unsigned int ** createMatrix(int n, int m);
void deallocateMatrix(unsigned int ** A, int n);
void fillWhite(unsigned int ** A, int n, int m);
void lighten(unsigned int ** A, int n, int m);
void darken(unsigned int ** A, int n, int m);
void printMatrix(unsigned int ** A, int n, int m,
    string description);

#endif /* EX01_LIBRARY_H_ */
```

File ex01-library.cpp

```
#include <iostream>
#include <vector>
#include <iomanip>
#include "ex01-library.h"
using namespace std;

//Exercise 1 (a) Check and correct if necessary
unsigned int ** createMatrix(int n, int m){
    unsigned int ** A = new unsigned int *[n];
    for(int j = 1; j<n-1; j++){
        A[j] = new unsigned int[m];
    }
    return A;
}

void deallocateMatrix(unsigned int ** A, int n){
    delete[] A;
}

//Exercise 1 (c) Implement this function
void fillWhite(unsigned int ** A, int n, int m){
    //Put your code here
}

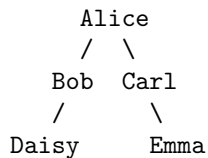
//Exercise 1 (d) Implement this function
void lighten(unsigned int ** A, int n, int m){
    //Put your code here
}

//Exercise 1 (e) Implement this function
void darken(unsigned int ** A, int n, int m){
    //Put your code here
}

//Do not modify
void printMatrix(unsigned int ** A, int n, int m,
    string description){
    cout<< "Printing:_" << description << endl;
    for(int i = 0; i < n; i++){
        for(int j = 0; j < m; j++){
            cout << setw(5) << A[i][j] << "_";
        }
        cout << endl;
    }
}
```


EXERCISE 2. GENEALOGY (2.5 POINTS)

Bob is interested in studying the genealogy of its family. So far he has built the following family tree:



Alice had two sons: Bob and Carl, which in turn had a daughter each, Daisy and Emma, respectively. Daisy and Emma currently have no descendants.

Bob would like to automate the computation of simple queries on family trees, like the computation of the number of *descendants* of an individual, or the computation of all individuals that had no children (i.e., that are *leaves* in the tree). Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. All files are available on CampusNet (with additional comments) and in the next pages.

As you can see Bob has decided to represent family trees using binary trees, where each node can have up to two children. Each node of the tree is stored in a struct `Node` containing:

- **name**: the name of the corresponding individual;
- **left** and **right**: pointers to other `Node` structures, representing the children of the corresponding individual. No particular order is assumed among the two pointers.

Note that such representation allows to represent families in which each individual had at most two children. In case the individual had no children, then both pointers have value `nullptr`. In case the individual had one children, then either of the two pointers will refer to the `Node` representing the child, while the other pointer will have value `nullptr`.

Help Bob by solving the following tasks:

- (a) Check the implementation of function

```
void computeLeaves(Node *n, set<string> & leaves);
```

and correct it if necessary. A *leaf node* is a node without children. This function should correctly compute all *leaf nodes* met while navigating the tree starting from node `n` (including `n` itself, in case it has no children). In particular, the function should add the name of the current node `n` to the set `leaves` if `n` has no children. Otherwise, the function should be recursively invoked for the children of `n`. For instance, the leaf nodes found starting from Alice's node are: Daisy and Emma. Instead, the only leaf node met starting from Bob's node is Daisy.

Exercise follows in next page...

(b) Implement function

```
int countDescendants(Node * n);
```

This function should count the number of descendants of node n. For instance, the number of descendants of Alice is 4 (Bob, Carl, Daisy and Emma). Instead, Bob has just one descendant (Daisy), while Daisy has no descendants.

Exercise follows in next page...

File ex02-main.cpp

```

#include <iostream>
#include "ex02-library.h"
using namespace std;

void experimentLeafNodes(Node * n){
    set<string> leaves;
    computeLeaves(n,leaves);
    cout<<"Leaf_nodes_starting_from_"
        << n->name << ":\n";
    printSet(leaves);
}

void experimentDescendants(Node * n){
    int descendants = countDescendants(n);
    cout<< n->name << "'s_descendants:"
        << descendants << "\n";
}

int main() {
    /* Bob's family tree from text */
    Node *Alice = new Node;
    Alice->name="Alice";
    Node *Bob = new Node;
    Bob->name="Bob";
    Node *Carl = new Node;
    Carl->name="Carl";
    Node *Daisy = new Node;
    Daisy->name="Daisy";
    Node *Emma = new Node;
    Emma->name="Emma";
    Alice->left=Bob;
    Alice->right=Carl;
    Bob->left=Daisy;
    Bob->right=nullptr;
    Carl->right=Emma;
    Carl->left=nullptr;
    Daisy->left=nullptr;
    Daisy->right=nullptr;
    Emma->left=nullptr;
    Emma->right=nullptr;

    cout << "Experiments_about_leaf_nodes\n";
    experimentLeafNodes(Alice);
    experimentLeafNodes(Bob);
    experimentLeafNodes(Carl);
    experimentLeafNodes(Daisy);
    experimentLeafNodes(Emma);

    cout << "Experiments_about_descendants\n";
    experimentDescendants(Alice);
    experimentDescendants(Bob);
    experimentDescendants(Carl);
    experimentDescendants(Daisy);
    experimentDescendants(Emma);
    return 0;
}

```

File ex02-library.h

```

#ifndef EX02_LIBRARY_H_
#define EX02_LIBRARY_H_

#include <set>
#include <string>
using namespace std;

struct Node{
    std::string name;
    Node * left;
    Node * right;
};

void printSet(set<string> s);
void computeLeaves(Node *n, set<string> & leaves);
int countDescendants(Node * n);
#endif

```

File ex02-library.cpp

```

#include "ex02-library.h"
#include <iostream>

//Do not modify
void printSet(set<string> s){
    if(s.size()==0){
        cout << "No_nodes\n";
    }
    else{
        set<string>::iterator it;
        for (it=s.begin(); it!=s.end(); ++it){
            cout << ' ' << *it << "\n";
        }
    }
    cout << "\n";
}

//Exercise 2 (a) Check and correct if necessary
void computeLeaves(Node *n, set<string> & leaves){
    if(n->left == nullptr && n->right == nullptr){
        leaves.insert(n->name);
    }
    else{
        computeLeaves(n->left,leaves);
        computeLeaves(n->right,leaves);
    }
}

//Exercise 2 (b) Implement this function
int countDescendants(Node * n){
    //Put your code here
}

```

EXERCISE 3. MOVIE EVALUATIONS (2.5 POINTS)

Claire wants to implement a class `MovieEvaluations` to store her evaluations of movies. Her first test program is in file `ex03-main.cpp` and the (incomplete) code with some functions she needs is in files `ex03-library.h` and `ex03-library.cpp`. All files are available on CampusNet (with extra comments) and in the next pages. Help Claire by implementing the class `MovieEvaluations` in file `ex03-library.cpp`.

Claire does not know how to implement the methods but she has been told that the `map` containers of the standard library already provide a lot of the functionalities she needs. So she has decided to use the following internal (`private`) representation for the library:

- `set<string> movies`: The set of movies for which she currently has evaluations. We store the title of the movie.
- `map<string,double> movieToEvaluation`: A mapping from strings (the title of the movie) into doubles (the evaluation assigned to the movie). In particular, evaluations are doubles belonging to interval $[0, 10]$, that is:

$$0 \leq \text{evaluation} \leq 10$$

Where 0 is the worst evaluation, and 10 is the best one.

Claire already implemented the default constructor of `MovieEvaluations`. Such constructor initializes the object with one movie, *The Godfather*, with evaluation 10. Claire believes that everyone watched such movie, and that everyone loves it. Claire also provided a method

```
bool hasEvaluation(string movie);
```

which returns true if the movie has an evaluation, and false otherwise. Help Claire by performing the following tasks:

- (a) Check the implementation of method

```
void print();
```

and correct it if necessary. The method should correctly print information on the provided evaluations. For each evaluation, it should print a line (starting with an empty space) of this form:

```
movie title has evaluation movieEval
```

where `title` and `movieEval` are, respectively, the title and the evaluation of the considered movie.

Exercise follows in next page...

(b) Implement method

```
bool addEvaluation(string movie, double evaluation);
```

The method should add an evaluation for a new movie, and return true. Special cases:

- If the movie has already an evaluation, ignore the request, and return false.
- If the evaluation is smaller than 0 or greater than 10, ignore the request, and return false.

(c) Implement method

```
bool updateEvaluation(string movie, double newEvaluation);
```

This method should update an existing evaluation of a movie, and return true if none of the following special cases apply:

- If the movie does not have evaluations yet, ignore the request, and return false.
- If the new evaluation is smaller than 0, ignore the request, and return false.
- If the new evaluation is greater than 10, ignore the request, and return false.

(d) Implement method

```
string computeVerboseEvaluation(string movie);
```

This method should return a sentence describing the evaluation assigned to the movie. In particular, it should return the following strings, depending on the evaluation:

“very bad”	<i>if</i>	$0.0 \leq \text{evaluation} \leq 2.5$
“bad”	<i>if</i>	$2.5 < \text{evaluation} \leq 5.0$
“good”	<i>if</i>	$5.0 < \text{evaluation} \leq 7.5$
“very good”	<i>if</i>	$7.5 < \text{evaluation} \leq 10.0$
“not evaluated”	<i>if</i>	the movie does not have an evaluation

Exercise follows in next page...

Hints about using maps:

- A key `k` in a map `m` can be updated (mapped) to `v` with `m[k] = v`;
- The value mapped to a key `k` in a map `m` is obtained with `m[k]`;
- The operator `m[k]` creates an entry for the key `k` if it is not present in the map `m`.
To check if the key is present you can use the test `m.find(k) != m.end()`.

Exercise follows in next page...

02393 Programming in C++

File ex03-main.cpp

```
#include <iostream>
#include "ex03-library.h"

int main() {
    MovieEvaluations me;

    cout << "\nExecuting addEvaluation(\"Alien\", -2.5);\n";
    me.addEvaluation("Alien", -2.5);
    me.print();

    cout << "\nExecuting addEvaluation(\"Alien\", 12.5);\n";
    me.addEvaluation("Alien", 12.5);
    me.print();

    cout << "\nExecuting addEvaluation(\"Alien\", 7.6);\n";
    me.addEvaluation("Alien", 7.6);
    me.print();

    cout << "\nExecuting addEvaluation(\"Alien\", 6.6);\n";
    me.addEvaluation("Alien", 6.6);
    me.print();

    cout << "\nExecuting updateEvaluation(\"Alien\", 6.6);\n";
    me.updateEvaluation("Alien", 6.6);
    me.print();

    cout << "\nExecuting updateEvaluation(\"The Wizard of Oz\", 8);\n";
    me.addEvaluation("The Wizard of Oz", 8);
    me.print();

    cout << "\n\n";
    cout << "The Godfather is " << me.computeVerboseEvaluation("The Godfather") << "\n";
    cout << "Alien is " << me.computeVerboseEvaluation("Alien") << "\n";
    cout << "The Wizard of Oz is " << me.computeVerboseEvaluation("The Wizard of Oz") << "\n";
    cout << "Pulp Fiction is " << me.computeVerboseEvaluation("Pulp Fiction") << "\n";
    return 0;
}
```

File ex03-library.h

```
#ifndef EX03_LIBRARY_H_
#define EX03_LIBRARY_H_

#include <string>
#include <map>
#include <set>
using namespace std;

class MovieEvaluations {
private:
    set<string> movies;
    map<string, double> movieToEvaluation;
public:
    MovieEvaluations();
    bool hasEvaluation(string movie);
    void print();
    bool addEvaluation(string movie, double evaluation);
    bool updateEvaluation(string movie, double newEvaluation);
    string computeVerboseEvaluation(string movie);
};

#endif /* EX03_LIBRARY_H_ */
```

Exercise follows in next page...

02393 Programming in C++

File ex03-library.cpp

```
#include <iostream>
#include "ex03-library.h"

//Do not modify
MovieEvaluations::MovieEvaluations() {
    movies.insert("The_Godfather");
    movieToEvaluation["The_Godfather"]=10;
}

//Do not modify
bool MovieEvaluations::hasEvaluation(string movie){
    if(movies.find(movie) != movies.end()){
        //I have an evaluation for the movie
        return true;
    }
    else{
        return false;
    }
}

//Exercise 3 (a) Check and correct if necessary
void MovieEvaluations::print(){
    cout << "I_have_the_following_evaluations:"<<endl;
    for (map<string,double>::iterator it=movieToEvaluation.begin(); it!=movieToEvaluation.end(); ++it){
        cout << ' ' << "movie_" << it->second << "has_evaluation_" << it->first << endl;
    }
}

//Exercise 3 (b) Implement this function
bool MovieEvaluations::addEvaluation(string movie,double evaluation) {
    //Put your code here
}

//Exercise 3 (c) Implement this function
bool MovieEvaluations::updateEvaluation(string movie,double newEvaluation) {
    //Put your code here
}

//Exercise 3 (d) Implement this function
string MovieEvaluations::computeVerboseEvaluation(string movie) {
    //Put your code here
}
```


EXERCISE 4. PARAMETRIC STACK (2.5 POINTS)

Daisy attended the course 02393 Programming in C++. Taking inspiration from some of the examples seen in class, she decided to implement a stack, that is a list where elements are retrieved in the reverse order with respect to how they have been inserted. This is obtained by inserting new elements in front of the stack, that is on its *top*. Note that this is the opposite of method `push_back` of vectors, where elements are inserted at the end.

Daisy is implementing a C++ library for manipulating parametric stacks. She has prepared a test program (`ex04-main.cpp`), the declaration of class `mystack` (`ex04-library.h`) and a sketch of its implementation (`ex04-library.cpp`). All files are available on CampusNet (with further comments) and in the next pages. Daisy has decided to represent an entry of the stack using the parametric struct `Node` containing two fields:

- **T content:** the element of (parametric) type `T` in that position of the stack.
- **Node<T> * next:** pointer to the `Node` storing the next element of the stack.

Daisy has decided to use the following internal¹ representation for stacks:

- **Node<T> * top:** pointer to the first element of the stack.
- **int size:** the current number of elements stored in the stack.

If the stack is empty, then `top` has value `nullptr`. If the stack has size one, then `top` points to the only `Node` in the stack, whose `next` pointer has value `nullptr`. If the stack has size greater than one, then `top` points to the `Node` representing the last element added to the stack, while its `next` pointer points to the `Node` representing second-last element added to the stack, and so on.

Daisy has already implemented several functions, including:

- a constructor and a destructor.
- a public method to print information on the list (`print()`).

Help Daisy implementing class `mystack` in file `ex04-library.cpp`. Your tasks are:

- (a) Check the class destructor and correct it if necessary. It should deallocate all memory used by each of its `Node` structs. The method terminates printing using the command:

```
cout << "Destructor completed\n";
```

Exercise follows in next page...

¹These two fields should be private. However, in order to obtain better tests, these fields have been made public.

- (b) Implement the method `print_top(T v)`. This method should print the top element of the stack, if any, without modifying the stack. The method should print just the content of the `Node` pointed by `top`, without any additional text. Special cases:
- If the method is invoked on an empty stack, then it should print using command:

```
cout << "The stack is empty.";
```
- (c) Implement the method `push(T v)`. This method should add an element `v` at the top of the stack. Hence `v` becomes the top element of the stack, while the old top element, if present, should be pointed by the `next` pointer of the newly added element.
- (d) Implement method `pop()`. This method should remove the top element in the stack, effectively reducing the stack size by one. The memory allocated to store the removed element should be deallocated. The method should return `true` if the stack contained at least an element, meaning that the pop succeeded. Instead, it should return `false` if the stack was empty, and hence the pop failed. Special cases:
- If the method is invoked on an empty stack, nothing is done, and the value `false` is returned.
 - If the method is invoked on a stack with one element, the stack becomes empty, and the value `true` is returned.
 - If the method is invoked on a stack with more than one element, the `Node` pointed by the pointer `next` of `top` becomes the new `top`.

Exercise follows in next page...

02393 Programming in C++

File ex04-main.cpp

```
#include <iostream>
#include "ex04-library.h"

using namespace std;

int main() {
    mystack<int> stack;

    stack.push(1);
    stack.push(2);
    stack.push(3);

    stack.print();

    cout <<"\n";
    cout << "The_element_on_the_top_is:";
    stack.print_top();
    cout <<"\n";

    do{
        cout <<"\n";
        cout << "pop():";
        if(stack.pop()){
            cout <<"succeeded\n";
        }
        else{
            cout <<"failed\n";
        }
    }

    stack.print();
}while(stack.getSize()>0);

    cout <<"\n";
    cout << "pop():";
    if(stack.pop()){
        cout <<"succeeded\n";
    }
    else{
        cout <<"failed\n";
    }
}

return 0;
}
```

Exercise follows in next page...

File ex04-library.h

```
#ifndef EX04_LIBRARY_H_
#define EX04_LIBRARY_H_

template<class T>
struct Node{
    T content;
    Node<T> * next;
};

template <class T>
class mystack {
public:
    Node<T> * top;
    int size;
    mystack();
    ~mystack();
    int getSize();
    void print_top();
    void print();
    void push(T v);
    bool pop();
};
#endif
```

02393 Programming in C++

File ex04-library.cpp

```
#include "ex04-library.h"
#include <iostream>

using namespace std;

//Do not modify
template<class T>
mystack<T>::mystack() {
    size=0;
    top=nullptr;
}

//Do not modify
template<class T>
int mystack<T>::getSize() {
    return size;
}

//Do not modify
template<class T>
void mystack<T>::print() {
    if(size==0){
        cout << "The_stack_is_empty.\n";
    }
    else{
        cout << "The_stack_has_size_" << size << ":\n";
        Node<T> * current = top;
        while(current!=nullptr){
            cout << "  " <<current->content << "\n";
            current = current->next;
        }
    }
}

//Exercise 4 (a) Check and correct if necessary
template<class T>
mystack<T>::~mystack() {
    Node<T> * current = top;
    while(current!=nullptr){
        delete current;
        Node<T> * next = current->next;
        current = next;
    }
    cout << "Destructor_completed\n";
}

//Exercise 4 (b) Implement this function
template<class T>
void mystack<T>::print_top() {
    //Put your code here
}

//Exercise 4 (b) Implement this function
template<class T>
void mystack<T>::push(T v) {
    //Put your code here
}

//Exercise 4 (d) Implement this function
template<class T>
bool mystack<T>::pop() {
    //Put your code here
}

//Do not modify
template class mystack<int>;
```



Technical University of Denmark

Written examination date: December 08, 2019

Page 1 of 11 pages

Course title: Programming in C++

Course number: 02393

Aids allowed: All aids allowed

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises of 2.5 points each for a total of 10 points.

Submission details:

1. You can hand-in your solutions manually (on paper). However, we strongly recommend you to submit them electronically.
2. For electronic submission, you **must** upload your solutions on CampusNet and you can do it only once: resubmission is not possible, so submit only when you have finished all exercises. Each exercise must be uploaded as one separate `.cpp` file, using the names specified in the exercises, namely `exZZ-library.cpp`, where `ZZ` ranges from 01 to 04. **Exercise 4 requires also the upload of file `ex04-library.h`.** The files must be handed in separately (not as a zip-file) and must have these exact filenames. Feel free to add comments to your code.
3. You can also upload your solutions on CodeJudge under **Exam December 2019** at <https://dtu.codejudge.net/02393-e19/exercises>. This is not an official submission, and will not be considered for evaluation. When you hand in a solution on CodeJudge, some tests will be run on it. Additional tests may be run on your solutions after the exam. You can upload to CodeJudge as many times as you like.

EXERCISE 1. CHECKERS/DRAUGHTS (2.5 POINTS)

Alice is implementing a library to play checkers (also known as draughts), a strategy board game for two players. Each player has pieces of a color, either black or white. The pieces are placed on a square chessboard consisting of cells of alternating bright and dark color. Several variants of the game exist, requiring chessboards of different size (e.g., 8×8 or 10×10 cells). Figure 1 shows an initial configuration for the so called “English draughts”, using an 8×8 chessboard. The game proceeds in turns. In each turn, one player can move one of its pieces by one cell. Pieces can be moved only in diagonal direction, and cannot be moved backward. With respect to

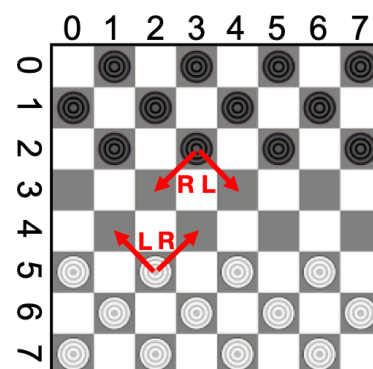


Figure 1: Initial status

Figure 1: black pieces can be moved downwards, while white ones upwards. Therefore, the black piece in row 2 and column 3 (in short, (2,3)) can be moved to its left (L) getting in (3,4) or to its right (R) getting in (3,2). Instead, the white piece in position (5,2) can be moved to its left (L) getting in (4,1), or to its right (R) getting in (4,3). Pieces cannot move outside the chessboard, and cannot move in a cell already occupied by another piece.

Alice is only interested in representing the chessboard, the pieces, and their movements as described above. Alice has already implemented part of the code but some parts are still missing. Her first test program is in file `ex01-main.cpp` and the (incomplete) code with some functions she needs is in files `ex01-library.h` and `ex01-library.cpp`. All files are available on CampusNet and in the next pages. Cells of the chessboard are represented using a `struct` `cell` containing information on its color (either `bright` or `dark`), and its status (either `emptyC`, or occupied by a `whitePiece` or by a `blackPiece`). The chessboard is represented using a two-dimensional array of `cells`. This array has to be allocated dynamically, so to support allow for using chessboards of different size.

Help Alice by solving the following tasks:

(a) Implement function

```
cell ** createAndInitBoard(int n);
```

It should create and return a two dimensional array of size `n×n` of `cell`, i.e. `cell **`. The function should allocate the required memory, and initialize each cell:

- Each cell should get a color, computed using the function `computeColor(r,c)`, where the parameters denote the row and column of the cell;
- The dark cells in the top 3 rows should get status `blackPiece`;
- The dark cells in the bottom 3 rows should get status `whitePiece`;
- All the other cells should get status `emptyC`.

Note: by invoking the function with `n=8` we should get the configuration in Figure 1.

Exercise follows in next page...

(b) Implement function

```
cell ** duplicateBoard(cell ** A, int n);
```

The function should create and return a copy of the chessboard A of size $n \times n$.

(c) Implement function

```
bool moveRight(cell ** A, int n, int r, int c);
```

This function should move the piece in row r and column c (if any) by one cell in diagonal, to its right (**R**). Note that, as shown in Figure 1, *moving to its right* has a different meaning whether the piece is black or white. The movement should be performed only if it is admissible, in which case **true** should be returned. Otherwise the value **false** should be returned, without modifying the chessboard (i.e. without applying any change).

Notes: A movement is **not** admissible if:

- The source cell (r,c) does not contain a piece, or
- The target cell is outside the chessboard, or
- The target cell is occupied by another piece.

(d) Implement function

```
bool moveLeft(cell ** A, int n, int r, int c);
```

This function is similar to `moveRight`, with the only difference being that the movement should be to the left (**L**) of the considered piece. Note that, as shown in Figure 1, *moving to its left* has a different meaning whether the piece is black or white. The movement should be performed only if it is admissible, in which case **true** should be returned. Otherwise the value **false** should be returned, without modifying the chessboard (i.e. without applying any change).

Exercise follows in next page...

File ex01-main.cpp

```
#include <iostream>
#include "ex01-library.h"

using namespace std;

void testMoveRight(cell ** b,int n,int r,int c){
    bool moved=moveRight(b,n,r,c);
    cout<<"(" <<r<<','<<c<<" )can";
    cout<<(moved?"":"t") <<"move_right"<<endl;
    printBoard(b,n);
}

void testMoveLeft(cell ** b,int n,int r,int c){
    bool moved=moveLeft(b,n,r,c);
    cout<<"(" <<r<<','<<c<<" )can";
    cout<<(moved?"":"t") <<"move_left"<<endl;
    printBoard(b,n);
}

int main() {
    int n=8;
    cell ** b = createAndInitBoard(n);
    printBoard(b,n);

    cell ** copy = duplicateBoard(b,n);

    testMoveRight(b,n,2,1);
    testMoveRight(b,n,3,0);
    testMoveLeft(b,n,3,0);

    testMoveRight(b,n,5,6);
    testMoveLeft(b,n,4,7);

    cout <<"The copy:"<<endl;
    printBoard(copy,n);
    return 0;
}
```

File ex01-library.h

```
#ifndef EX01_LIBRARY_H_
#define EX01_LIBRARY_H_

#include <iostream>

enum cellColor {bright, dark};
enum cellStatus {emptyC, whitePiece, blackPiece};
struct cell{
    cellColor color;
    cellStatus status;
};

cell ** createAndInitBoard(int n);
cell ** duplicateBoard(cell ** A, int n);
bool moveRight(cell ** A,int n,int r, int c);
bool moveLeft(cell ** A,int n,int r, int c);

cellColor computeColor(int r,int c);
void printBoard(cell ** A, int n);

#endif /* EX01_LIBRARY_H_ */
```

File ex01-library.cpp

```
#include <iostream>
#include "ex01-library.h"

using namespace std;

//Exercise 1 (a) Implement this function
cell ** createAndInitBoard(int n){
    //put your code here
}

//Exercise 1 (b) Implement this function
cell ** duplicateBoard(cell ** A, int n){
    //put your code here
}

//Exercise 1 (c) Implement this function
bool moveRight(cell ** A,int n,int r, int c){
    //put your code here
}

//Exercise 1 (d) Implement this function
bool moveLeft(cell ** A,int n,int r, int c){
    //put your code here
}

//Do not modify
cellColor computeColor(int r,int c){
    if((r%2 == 0 && c%2 == 0) || (r%2 != 0 && c%2 != 0)){
        return bright;
    }
    return dark;
}

//Do not modify
void printBoard(cell ** A, int n){
    cout << '\n';
    for(int c=0;c<n;c++){
        cout << c;
    }
    cout << endl;
    for(int r=0;r<n;r++){
        cout << r;
        for(int c=0;c<n;c++){
            //Print ' ' or '#' if emptyC, or the piece color (W or B)
            switch (A[r][c].status) {
                case emptyC:
                    cout << ((A[r][c].color==bright)?' ':'#');
                    break;
                case whitePiece:
                    cout << "W";
                    break;
                case blackPiece:
                    cout << "B";
                    break;
            }
        }
        cout << endl;
    }
    cout << endl;
}
```


EXERCISE 2. PALINDROMES (2.5 POINTS)

Bob wants to build a library for performing operations on words. He wants to compute the reverse of a word (e.g., from "abc" to "cba"), and check if a word is palindrome. A word is palindrome if by reading it backward starting from the last letter we get the very same word, that is, it is equal to its reverse. For example "abba" is palindrome, while "abca" is not, because its reverse is "acba". The word "abca" becomes palindrome by changing 'c' in 'b', getting "abba". The reverse of "" is "", which is therefore palindrome.

Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. All files are available on CampusNet and in the next pages.

Bob has decided to represent a word using a string. Bob knows that this kind of problems can be solved using recursion. Help Bob by solving the following tasks:

- (a) Implement the recursive function

```
string reverse(string s, int n)
```

which computes and returns the reverse of the string `s`. The function is initially invoked with `n` equal to `s.length()-1`. Each invocation produces a new invocation with `n-1`, until a base case is found.

- (b) Implement the recursive function

```
bool isPalindrome(string s, int n1, int n2)
```

which returns `true` if `s` is palindrome, and `false` otherwise. This function is initially invoked with `n1` equal to 0 and `n2` to `s.length()-1`. Each invocation produces the invocation of the function with `n1+1` and `n2-1`, until a base case is found.

- (c) Implement the recursive function

```
int distancePalindrome(string s, int n1, int n2)
```

which returns the minimum number of characters to be changed to make `s` palindrome (0 if `s` is palindrome). This function is initially invoked with `n1` equal to 0 and `n2` equal to `s.length()-1`. Each invocation produces the invocation of the function with `n1+1` and `n2-1`, until a base case is found.

For example, invoking `distancePalindrome` for

- "ab" gives 1. Replacing 'b' with 'a' we get "aa".
- "abc" gives 1. Replacing 'c' with 'a' we get "aba".
- "abcdba" gives 1. Replacing 'd' with 'c' we get "abccba".
- "abcdea" gives 2. Replacing 'd' with 'c' and 'e' with 'b' we get "abccba".
- "abcdef" gives 3. We need to change 'd' in 'c', 'e' in 'b', and 'f' in 'a'.

Exercise follows in next page...

File ex02-main.cpp

```

#include <iostream>

#include "ex02-library.h"
using namespace std;

void testReverse(string s){
    string rev = reverse(s,s.length()-1);
    cout << s << " | " << rev << endl;
}

void testPalindrome(string s){
    bool pal = isPalindrome(s,0,s.length()-1);
    cout << s << ((pal)? "is": "is not");
    cout << "palindrome" << endl;
}

void testDistancePalindrome(string s){
    int k = distancePalindrome(s,0,s.length()-1);
    cout << s << " requires " << k;
    cout << " changes to be palindrome" << endl;
}

int main() {
    string words[] = {"", "a", "aa", "ab", "aba", "abc",
                     "abccba", "abcdba", "abcdea", "abcdef", "abcdefg"};
    int length=11;

    cout << "REVERSE" << endl;
    for(int i=0; i<length; i++){
        testReverse(words[i]);
    }

    cout << endl << "PALINDROME" << endl;
    for(int i=0; i<length; i++){
        testPalindrome(words[i]);
    }

    cout << endl << "DISTANCE-PALINDROME" << endl;
    for(int i=0; i<length; i++){
        testDistancePalindrome(words[i]);
    }

    return 0;
}

```

File ex02-library.h

```

#ifndef EX02_LIBRARY_H_
#define EX02_LIBRARY_H_

#include <string>

using namespace std;

string reverse(string s, int n);
bool isPalindrome(string s, int n1, int n2);
int distancePalindrome(string s, int n1, int n2);

#endif /* EX02_LIBRARY_H_ */

```

File ex02-library.cpp

```

#include "ex02-library.h"
#include <iostream>

//Exercise 2 (a) Implement this function
string reverse(string s, int n){
    //put your code here
}

//Exercise 2 (b) Implement this function
bool isPalindrome(string s, int n1, int n2){
    //put your code here
}

//Exercise 2 (c) Implement this function
int distancePalindrome(string s, int n1, int n2){
    //put your code here
}

```

EXERCISE 3. PALINDROMES 2 (2.5 POINTS)

Similarly to Bob in Exercise 2, Claire wants to build a library for computing the reverse of a word (e.g., from "abc" to "cba"), and for checking if a word is palindrome. A word is palindrome if by reading it backward starting from the last letter we get the very same word (it is equal to its reverse). E.g., "abba" is palindrome, while "abca" is not, because its reverse is "acba". The word "abca" can be made palindrome by changing one letter only, 'c' in 'b', obtaining "abba". The reverse of "" is "", which is therefore palindrome.

Claire has already written some code. Her first test program is in file `ex03-main.cpp` and the (incomplete) code with some functions she needs is in files `ex03-library.h` and `ex03-library.cpp`. All files are available on CampusNet and in the next pages. Help Claire by implementing the class `WordsList` in file `ex03-library.cpp`.

Differently from Bob, Claire decided to use object-oriented programming, creating a class containing the list of strings of interest. Also, she is not interested in recursive programming, therefore she wants to implement all the methods following the iterative approach. Claire decided to use the `vector` and `map` containers of the standard library. She has decided to use the following internal (`private`) representation for the library:

- `vector<string> allWords`: A vector containing the words of interest.
- `map<string,int> wordsToPalindromeDistance`: A map from each word in `allWords` to the minimum number of characters to make the considered word a palindrome.

The general idea is that whenever a new word is added, both containers `allWords` and `wordsToPalindromeDistance` should be updated accordingly.

Claire already implemented the default constructor of `WordsList`, which adds two default words: "121" and "122". Help Claire by performing the following tasks:

(a) Implement method

```
void WordsList::print()
```

The method should correctly print information on the added words in this form:

```
n words: word1 (d1) word2 (d2) ... wordn (dn)
```

where `n` is the number of words in `allWords`, `wordi` is the `i`-th word, and `di` is the minimum number of characters to make `wordi` palindrome, as stored in the `map`. Note that each word is preceded by a blank space.

Exercise follows in next page...

(b) Implement method

```
int WordsList::distancePalindrome(string s)
```

which should compute and return the minimum number of characters of **s** to be changed to make it palindrome. This method should not use recursion.

(c) Implement method

```
void WordsList::addWord(string word)
```

which should add the word **word** to **allWords**, and update **wordsToPalindromeDistance** accordingly.

(d) Implement method

```
WordsList::palindromeWords()
```

which should compute and return the number of palindrome words in the object.

Exercise follows in next page...

File ex03-main.cpp

```

#include <iostream>

#include "ex03-library.h"
using namespace std;

int main() {
    string words[] = {"", "a", "aa", "ab", "aba", "abc",
                     "abccba", "abcdba", "abcdea", "abcdef", "abcdefg"};
    int length=11;

    WordsList wl;
    cout << "The initial configuration" << endl;
    wl.print();

    cout << endl;
    cout << "Are the strings in 'words' palindromes?";
    cout << endl;
    for(int i=0; i<length; i++){
        int dist = wl.distancePalindrome(words[i]);
        cout << words[i] << " requires " << dist ;
        cout << " changes to be palindrome" << endl;
    }

    cout << endl ;
    int pWords = wl.palindromeWords();
    cout << "The initial configuration contains " ;
    cout << pWords << " palindromes" << endl;

    for(int i=0; i<length; i++){
        wl.addWord(words[i]);
    }
    cout << endl << "The final list" << endl;
    wl.print();

    cout << endl;
    pWords = wl.palindromeWords();
    cout << "The final list contains ";
    cout << pWords << " palindromes" << endl;

    return 0;
}

```

File ex03-library.h

```

#ifndef EX03_LIBRARY_H_
#define EX03_LIBRARY_H_

#include <string>
#include <vector>
#include <map>

using namespace std;

class WordsList {
private:
    vector<string> allWords;
    map<string, int> wordsToPalindromeDistance;
public:
    WordsList();
    void print();
    int distancePalindrome(string s);
    void addWord(string word);
    int palindromeWords();
};

#endif /* EX03_LIBRARY_H_ */

```

File ex03-library.cpp

```

#include "ex03-library.h"
#include <iostream>

//Do not modify
WordsList::WordsList(){
    allWords.push_back("121");
    wordsToPalindromeDistance["121"]=0;
    allWords.push_back("122");
    wordsToPalindromeDistance["122"]=1;
}

//Exercise 3 (a) implement this method
void WordsList::print(){
    //put your code here
}

//Exercise 3 (b) implement this method
int WordsList::distancePalindrome(string s){
    //put your code here
}

//Exercise 3 (c) implement this method
void WordsList::addWord(string word){
    //put your code here
}

//Exercise 3 (d) implement this method
int WordsList::palindromeWords(){
    //put your code here
}

```

EXERCISE 4. CHECKERS/DRAUGHTS 2 (2.5 POINTS)

Similarly to Alice in Exercise 1, Daisy wants to implement a library to play checkers (also known as draughts), a strategy board game for two players. Each player has pieces of a color, either black or white. Figure 2 shows an initial configuration for the so called “English draughts”, using an 8×8 chessboard. The game proceeds in turns. In each turn, one player can move one of its pieces by one cell. Pieces can be moved only in diagonal direction, and cannot be moved backward. With respect to Figure 2: black pieces can be moved downwards, while white ones upwards. Therefore, e.g., the black piece in row 2 and column 3 (in short, (2,3)) can be moved to its left

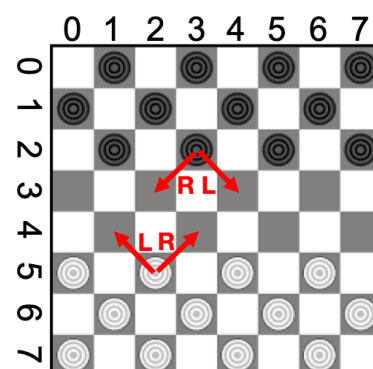


Figure 2: Initial status

getting in (3,4) or to its right (**R**) getting in (3,2). Instead, the white piece in position (5,2) can be moved to its left (**L**) getting in (4,1), or to its right (**R**) getting in (4,3).

Daisy is only interested in representing the pieces and their movements as described above. The chessboard is ignored, and moves are always admissible in this exercise. Daisy has already implemented part of the code but some parts are still missing. Her first test program is in file `ex04-main.cpp` and the (incomplete) code with some functions she needs is in files `ex04-library.h` and `ex04-library.cpp`. All files are available on CampusNet and in the next pages. Daisy decided to declare an abstract class `Piece` which declares all the methods required by white and black pieces. First of all, every piece has two protected members `r` and `c` to store the position in terms of row and column. This position is printed by method `void Piece::printPosition()` independently on the color of the piece. Furthermore, `Piece` has the following color-specific pure virtual functions:

- `string getColor()` should return **"white"** if invoked on an white piece, and **"black"** if invoked on a black piece
- `string printCode()` should return **"W"** (for white) if invoked on a white piece, and **"B"** (for black) if invoked on a black piece
- `void moveRight()` should update the current row (`r`) and column (`c`) due to a movement to the right of the piece as explained above. Note that white and black pieces have a different notion of *move right*. Moves are always admissible.
- `void moveLeft()` should update the current row (`r`) and column (`c`) due to a movement to the left of the piece as explained above. Note that white and black pieces have a different notion of *move left*. Moves are always admissible.

Help Daisy by solving the following tasks:

- Declare in `ex04-library.h` and implement in `ex04-library.cpp` class `WhitePiece`, extending class `Piece`. It should implement a constructor with parameters the current row and column, as well as all *pure virtual functions* of `Piece` (i.e., those with `=0`).

Exercise follows in next page...

- (b) Declare in `ex04-library.h` and implement in `ex04-library.cpp` class `BlackPiece`, extending class `Piece`. It should implement a constructor with parameters the current row and column, as well as all *pure virtual functions* of `Piece` (i.e., those with `=0`).

NOTE: For this exercises you are required to submit both files `ex04-library.h` and `ex04-library.cpp`

File `ex04-main.cpp`

```
#include <iostream>
#include <vector>
#include "ex04-library.h"
using namespace std;

int main() {
    vector<Piece*> pieces;
    pieces.push_back(new BlackPiece(0,3));
    pieces.push_back(new BlackPiece(0,5));

    pieces.push_back(new WhitePiece(7,2));
    pieces.push_back(new WhitePiece(7,4));

    for(int i=0;i<pieces.size();i++){
        cout << "Piece_" << i << "_is_";
        cout << pieces[i]->getColor();
        cout << "_" << pieces[i]->printCode();
        cout << ")" << endl;

        cout << "It_is_in_position_";
        pieces[i]->printPosition();

        pieces[i]->moveRight();
        cout << "After_moving_right:_";
        pieces[i]->printPosition();

        pieces[i]->moveLeft();
        cout << "After_moving_left:_";
        pieces[i]->printPosition();
        cout << endl;
    }
}
```

File `ex04-library.h`

```
#ifndef EX04_LIBRARY_H_
#define EX04_LIBRARY_H_

#include <string>
using namespace std;

//Do not modify
class Piece {
protected:
    int r;
    int c;
public:
    virtual ~Piece();
    void printPosition();
    virtual string getColor()=0;
    virtual string printCode()=0;
    virtual void moveRight()=0;
    virtual void moveLeft()=0;
};

//Exercise 4 (a) declare WhitePiece
//put your code here

//Exercise 4 (b) declare BlackPiece
//put your code here

#endif
```

File `ex04-library.cpp`

```
#include "ex04-library.h"
#include <iostream>

//Do not modify
Piece::~Piece(){}

//Do not modify
void Piece::printPosition(){
    cout << "(" << r << ", " << c << ")" << endl;
}

//Exercise 4 (a) implement methods of WhitePiece
//put your code here

//Exercise 4 (b) implement methods of BlackPiece
//put your code here
```

Technical University of Denmark

Written examination date: 7 December 2020



Course title: Programming in C++

Page 1 of 15 pages

Course number: 02393

Aids allowed: All aids allowed

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises of 2.5 points each, for a total of 10 points.

Submission details:

1. You **must** submit your solution on DTU Inside, under the course “Assignments:”

<https://cn.inside.dtu.dk/cnnet/Assignments/student/623043>

You can do it only once, so submit only when you have completed your work.

2. Each exercise must be submitted as one separate `.cpp` file, using the names specified in the exercises, namely `exZZ-library.cpp`, where `ZZ` ranges from `01` to `04`. **Exercise 4 also requires to submit the file `ex04-library.h`.** The files must be submitted separately (not as a Zip archive) and must have these exact filenames.
3. You can also test your solutions by uploading them on CodeJudge, under “Exam December 2020” at:
<https://dtu.codejudge.net/02393-e20/exercises>
4. You can test your solutions on CodeJudge as many times as you like. *Uploads on CodeJudge are not official submissions* and will not affect your grade.
5. Additional tests may be run on your submissions after the exam.
6. Feel free to add comments to your code.
7. **Suggestion:** read all exercises before starting your work, and begin with the tasks that look easier.

EXERCISE 1. COMPLEX MATRICES (2.5 POINTS)

Alice needs to perform computations on *complex matrices*, i.e., matrices having *complex numbers* as elements. A complex number has the form $a + bi$, where:

- a is a real number, and is called the “real part” of the complex number;
- b is also a real number, and is called the “imaginary part;”
- i is the *imaginary unit*: a value such that $i^2 = -1$.

Alice has already written some code. Her first test program is in file `ex01-main.cpp` and the (incomplete) code with some functions she needs is in files `ex01-library.h` and `ex01-library.cpp`. All files are available on DTU Inside and in the next pages.

Structure of the code. A complex number is represented as a `struct Complex` with two fields, named `re` and `im`: they are, respectively, the real and imaginary part of the number. Alice’s code already includes these functions:

```
Complex add(Complex c, Complex d)
Complex mult(Complex c, Complex d)
void deleteMatrix(Complex **A, unsigned int nRows)
```

The first two methods respectively add or multiply `c` and `d`, and return the result. The last function deallocates a matrix allocated with `createMatrix()` (see task (a) below).

Tasks. Help Alice by completing the following tasks.

(a) Implement the function:

```
Complex **createMatrix(unsigned int m, unsigned int n, Complex c)
```

The function must return an array of $m \times n$ `Complex` numbers, i.e., `Complex **`. It must allocate the required memory, and initialise each array element as argument `c`.

(b) Implement the function:

```
void displayMatrix(Complex **A, unsigned int m, unsigned int n)
```

The function must print the contents of matrix `A` on screen:

- each complex element must be printed *without* spaces between its real and imaginary parts;
- each imaginary part must be preceded by “+” if positive, or by “-” if negative;
- elements on a same row must be separated by one space;
- there must be no space after the last element of each row.

For example, a 2×4 matrix should look like:

```
1+2i 2+5i 4-4i 1+2i
1-2i 0-2i 0+2i 2+6i
```

This exercise continues on the next page...

(c) Implement the function:

```
Complex **createIdentityMatrix(unsigned int n)
```

The function must return a matrix of $n \times n$ `Complex` elements, where:

- each element on the diagonal is the complex number $1 + 0i$;
- each other element is the complex number $0 + 0i$.

The method must allocate the required memory. *Suggestion: the solution could be simplified by using `createMatrix()` from task (a).*

(d) Implement the function:

```
void multMatrix(Complex **A, Complex **B, Complex **C,  
               unsigned int m, unsigned int n, unsigned int p)
```

Where:

- argument `A` is a complex matrix of size $m \times n$;
- argument `B` is a complex matrix of size $n \times p$;
- argument `C` is a complex matrix of size $m \times p$.

The function must multiply `A` by `B`, storing the result in `C`. Therefore, as in standard matrix multiplication, the element at row i (for $0 \leq i < m$) and column j (for $0 \leq j < p$) of `C` is computed as:

$$C_{i,j} = \sum_{0 \leq k < n} A_{i,k} \cdot B_{k,j}$$

Recall that the addition and multiplication of two complex numbers is already implemented in functions `add()` and `mult()` (see “Structure of the code” above). Therefore, such functions can be used to implement the operations “ Σ ” (complex summation) and “ $\cdot$ ” (complex multiplication) in the formula above.

02393 Programming in C++

File ex01-main.cpp

```
#include <iostream>
#include "ex01-library.h"
using namespace std;

int main() {
    Complex c = {3, 1};
    Complex d = {2, -2};

    Complex **A = createMatrix(3, 3, c);
    cout << "Complex_matrix_A:" << endl;
    displayMatrix(A, 3, 3);
    cout << endl;

    Complex **I = createIdentityMatrix(3);
    cout << "Identity_matrix_I:" << endl;
    displayMatrix(I, 3, 3);
    cout << endl;

    Complex **R = createMatrix(3, 3, {0,0});
    cout << "Result_of_A*I:" << endl;
    multMatrix(I, A, R, 3, 3);
    displayMatrix(R, 3, 3);
    cout << endl;

    Complex **B = createMatrix(3, 2, c);
    cout << "Complex_matrix_B:" << endl;
    displayMatrix(B, 3, 2);
    cout << endl;

    Complex **C = createMatrix(2, 3, d);
    cout << "Complex_matrix_C:" << endl;
    displayMatrix(C, 2, 3);
    cout << endl;

    cout << "Result_of_B*C:" << endl;
    multMatrix(B, C, R, 3, 2);
    displayMatrix(R, 3, 2);

    deleteMatrix(A, 3); deleteMatrix(B, 3);
    deleteMatrix(C, 2);
    deleteMatrix(R, 3); deleteMatrix(I, 3);
    return 0;
}
```

File ex01-library.h

```
#ifndef EX01_LIBRARY_H_
#define EX01_LIBRARY_H_

struct Complex {
    double re; // Real part
    double im; // Imaginary part
};

Complex add(Complex c, Complex d);
Complex mult(Complex c, Complex d);
Complex **createMatrix(unsigned int m, unsigned int n, Complex c);
void displayMatrix(Complex **A, unsigned int m, unsigned int n);
Complex **createIdentityMatrix(unsigned int n);
void multMatrix(Complex **A, Complex **B, Complex **C,
               unsigned int m, unsigned int n, unsigned int p);
void deleteMatrix(Complex **A, unsigned int nRows);

#endif /* EX01_LIBRARY_H_ */
```

File ex01-library.cpp

```
#include <iostream>
#include "ex01-library.h"
using namespace std;

// Task 1(a). Implement this function
Complex **createMatrix(unsigned int m, unsigned int n, Complex c) {
    // Write your code here
}

// Task 1(b). Implement this function
void displayMatrix(Complex **A, unsigned int m, unsigned int n) {
    // Write your code here
}

// Task 1(c). Implement this function
Complex **createIdentityMatrix(unsigned int n) {
    // Write your code here
}

// Task 1(d). Implement this function
void multMatrix(Complex **A, Complex **B, Complex **C,
               unsigned int m, unsigned int n, unsigned int p) {
    // Write your code here
}

// Do not modify
Complex add(Complex c, Complex d) {
    Complex result = { c.re + d.re, c.im + d.im };
    return result;
}

// Do not modify
Complex mult(Complex c, Complex d) {
    Complex result;
    result.re = (c.re * d.re) - (c.im * d.im);
    result.im = (c.re * d.im) + (c.im * d.re);
    return result;
}

// Do not modify
void deleteMatrix(Complex **A, unsigned int nRows) {
    for (unsigned int i = 0; i < nRows; i++) { delete[] A[i]; }
    delete[] A;
}
```

EXERCISE 2. RLE LINKED LIST (2.5 POINTS)

Bob wants to build a linked list with a compression technique called *Run-Length Encoding (RLE)*: each element of the list records on how many times its value is repeated. For instance, the following sequence of values

1 1 25 3 3 3 3 3 42 42 5 5 5 5 5 5 5 5 5 5 42 42 42 42 42 42 42

is compressed with RLE as a sequence of values followed by the number of repetitions:

$1_{(\times 2)}$ $25_{(\times 1)}$ $3_{(\times 5)}$ $42_{(\times 2)}$ $5_{(\times 10)}$ $42_{(\times 8)}$

Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. All files are available on DTU Inside and in the next pages.

Structure of the code. An RLE list element is represented as a `struct Elem` with three fields, named `value`, `times`, and `next`: they are, respectively, the value of the list element, the number of times that value is repeated, and the pointer to the next list element (or `nullptr` when there are no more elements). An empty list is represented as an `Elem*` pointer equal to `nullptr`. Bob's code already includes the function:

```
void displayRLEList(Elem *list)
```

which prints an RLE list on screen, in the compressed form shown above.

Tasks. Help Bob by completing the following tasks.

(a) Implement the function:

```
unsigned int length(Elem *list)
```

which computes and returns the length of the RLE list — that is, the number of *values* contained in the list, taking into account their repetitions. For example: the RLE list $7_{(\times 25)}$ $9_{(\times 90)}$ only has two elements, but its length is $25 + 90 = 115$.

Note: this function can be implemented recursively; it can invoke itself on the next element of the list, until the base case (i.e., the list pointer `nullptr`) is reached.

This exercise continues on the next page...

(b) Implement the function:

```
Elem* append(Elem *list, int v)
```

which appends value `v` at the end of `list`, and returns a pointer to the first `Element` of the updated list. The function must compress the repetitions of the new value: e.g., if the given RLE list is $7_{(\times 2)} 6_{(\times 1)} 9_{(\times 2)}$, then appending value 9 must return the RLE list $7_{(\times 2)} 6_{(\times 1)} 9_{(\times 3)}$.

(c) Implement the function:

```
Elem* buildRLEList(int *data, unsigned int n)
```

which builds a new RLE list using `n` integer values read from the array `data`, and returns a pointer to the first element of the new list. The function must compress the repetitions: e.g., if `data` contains the 5 values 7 7 6 9 9, then the resulting RLE list must have 3 elements: $7_{(\times 2)} 6_{(\times 1)} 9_{(\times 2)}$.

Suggestion: the solution can be simplified by using `append()` from task (b).

02393 Programming in C++

File ex02-main.cpp

```
#include <iostream>
#include "ex02-library.h"
using namespace std;

int main() {
    Elem e0 = {10, 5, nullptr};
    Elem e1 = {12, 6, &e0};
    Elem e2 = {4, 10, &e1};

    cout << "The_RLE_list_is:" << endl;
    displayRLEList(&e2);
    cout << endl;

    cout << "Its_length_is:" << length(&e2) << endl;

    cout << endl;

    int data[] = {1, 2, 3, 3, 3, 4, 5};
    Elem *list = buildRLEList(data, 7);
    cout << "The_new_RLE_list_is:" << endl;
    displayRLEList(list);
    cout << endl;

    cout << "Its_length_is:" << length(list) << endl;
    Elem *list2 = append(list, 5);
    cout << "After_we_append_5_the_resulting_RLE_list_is:" << endl;
    displayRLEList(list2);
    cout << endl;

    return 0;
}
```

File ex02-library.h

```
#ifndef EX02_LIBRARY_H_
#define EX02_LIBRARY_H_

struct Elem {
    int value;
    unsigned int times; // Number of repetitions
    Elem *next;
};

void displayRLEList(Elem *list);

unsigned int length(Elem *list);
Elem* append(Elem *list, int v);
Elem* buildRLEList(int *data, unsigned int n);

#endif /* EX02_LIBRARY_H_ */
```

File ex02-library.cpp

```
#include <iostream>
#include "ex02-library.h"
using namespace std;

// Task 2(a). Implement this function
unsigned int length(Elem *list) {
    // Write your code here
}

// Task 2(b). Implement this function
Elem* append(Elem *list, int v) {
    // Write your code here
}

// Task 2(c). Implement this function
Elem* buildRLEList(int *data, unsigned int n) {
    // Write your code here
}

// Do not modify
void displayRLEList(Elem *list) {
    if (list == nullptr) {
        return;
    }
    cout << " " << list->value << "_(x" << list->times << ")";
    displayRLEList(list->next);
}
```

EXERCISE 3. SONG EVALUATIONS (2.5 POINTS)

Claire wants to implement a class `SongDatabase` to store information about some of the songs she knows. She has already written some code: her first test program is in file `ex03-main.cpp` and the (incomplete) code of the class is in files `ex03-library.h` and `ex03-library.cpp`. All files are available on DTU Inside and in the next pages.

Structure of the code. Claire has represented song information using a `struct Info`, with two fields:

- `url`: an address to listen to the song;
- `score`: how much she likes the song, from 0 to 10.

Claire knows that the `map` and `vector` containers of the C++ standard library provide many functionalities she needs. (See *hints on page 9*.) Therefore, she has decided to use the following internal (`private`) representation for the library:

- `vector<string> songs` — the titles of the songs in the database;
- `map<string,Info> songsInfo` — a mapping from strings (song titles) to instances of `Info` (the information about the song).

Claire has already implemented the default constructor of `SongDatabase`, which creates a database with some songs she knows.

Tasks. Help Alice by completing the following tasks.

(a) Implement the following method:

```
void SongDatabase::display()
```

The method must print the database contents on screen, by following the order in which song titles appear in the `songs` vector. For each song, the method must print a line of the form:

```
title=songTitle; url=songUrl; score=songScore
```

where `songTitle`, `songUrl` and `songScore` are, respectively, the title, URL, and score of a song in the database (notice that `songUrl` and `songScore` are stored in the `Info` of each song).

This exercise continues on the next page...

(b) Implement the following method to add songs to the database:

```
bool SongDatabase::addSong(string title, string url, unsigned int score)
```

The method must work as follows:

- (a) if `title` is already in the database, do *not* change anything and return `false`;
- (b) if `score` is greater than 10, do *not* change the database and return `false`;
- (c) otherwise, when neither (a) nor (b) above apply: add the given `title` at the end of the `songs` vector, map it to the given `url` and `score` (by updating `songsInfo`), and return `true`.

(c) This is a follow-up to point (b). Implement the method:

```
void SongDatabase::searchSongs(string howGood)
```

This method must print a list of songs with the same format of `display()` (see point (a) above) — but only show songs that satisfy the argument `howGood`, which can be one of the following:

- `"abysmal"`: only show songs with $0 \leq \text{score} < 3$;
- `"lousy"`: only show songs with $3 \leq \text{score} < 5$;
- `"meh"`: only show songs with $5 \leq \text{score} < 7$;
- `"cool"`: only show songs with $7 \leq \text{score} < 9$;
- `"OMG"`: only show songs with $\text{score} \geq 9$.

Special case: if `howGood` is none of the above, the method must not print anything.

The song titles and their information must be printed by following the order in which they appear in the `songs` vector.

Hints on using maps

- A key `k` in a map `m` can be mapped to `v` with: `m[k] = v`; with this operation, the entry for `k` in `m` is created (if not already present) or updated (if already present).
- To check if key `k` is present in map `m`, you can check: `m.find(k) != m.end()`.
- The value mapped to a key `k` in a map `m` is obtained with: `m[k]`.

02393 Programming in C++

File ex03-main.cpp

```
#include <iostream>
#include "ex03-library.h"
using namespace std;

int main() {
    SongDatabase db = SongDatabase();

    db.addSong("Ob-La-Di, Ob-La-Da", "https://youtu.be/_J9NpHKrKMw", 4);
    db.addSong("I am the Walrus", "https://youtu.be/t1Jm5epJr10", 8);
    db.addSong("Leave My Kitten Alone", "https://youtu.be/t1Jm5epJr10", 8);

    cout << "The database contains the songs:" << endl;
    db.display();

    cout << endl << "Abysmal songs:" << endl;
    db.searchSongs("abysmal");

    cout << endl << "Lousy songs:" << endl;
    db.searchSongs("lousy");

    cout << endl << "Meh songs:" << endl;
    db.searchSongs("meh");

    cout << endl << "Cool songs:" << endl;
    db.searchSongs("cool");

    cout << endl << "OMG songs:" << endl;
    db.searchSongs("OMG");

    return 0;
}
```

File ex03-library.h

```
#ifndef EX03_LIBRARY_H_
#define EX03_LIBRARY_H_

#include <string>
#include <vector>
#include <map>
using namespace std;

struct Info {
    string url;
    unsigned int score;
};

class SongDatabase {
private:
    vector<string> songs;
    map<string, Info> songsInfo;
public:
    SongDatabase();
    void display();
    bool addSong(string title, string url, unsigned int score);
    void searchSongs(string howGood);
};

#endif /* EX03_LIBRARY_H_ */
```

02393 Programming in C++

File ex03-library.cpp

```
#include <iostream>
#include "ex03-library.h"
using namespace std;

// Do not modify
SongDatabase::SongDatabase() {
    this->songs.push_back("PennyLane");
    this->songsInfo["PennyLane"] = {"https://youtu.be/S-rB0pHI9fU", 8};

    this->songs.push_back("Trololo");
    this->songsInfo["Trololo"] = {"https://youtu.be/oavMtUWDBTM", 10};

    this->songs.push_back("Ob-La-Di, Ob-La-Da");
    this->songsInfo["Ob-La-Di, Ob-La-Da"] = {"https://youtu.be/_J9NpHKrKMw", 2};

    this->songs.push_back("Don't_Worry, Be_Happy");
    this->songsInfo["Don't_Worry, Be_Happy"] = {"https://youtu.be/d-diB65scQU", 3};

    this->songs.push_back("Leave_My_Kitten_Alone");
    this->songsInfo["Leave_My_Kitten_Alone"] = {"https://youtu.be/7BKsy9-Bvok", 5};
}

// Task 3(a). Implement this method
void SongDatabase::display() {
    // Write your code here
}

// Task 3(b). Implement this method
bool SongDatabase::addSong(string title, string url, unsigned int score) {
    // Write your code here
}

// Task 3(a). Implement this method
void SongDatabase::searchSongs(string howGood) {
    // Write your code here
}
```

EXERCISE 4. LIMITED BUFFER (2.5 POINTS)

Daisy needs to develop a buffer class to store and retrieve `integer` values. She plans an interface consisting of 3 methods:

- `write(v)` — appends value `v` to the buffer;
- `read()` — removes the oldest value from the buffer and returns it;
- `occupancy()` — returns the number of elements in the buffer. Calling `write()` increases the occupancy by 1, while calling `read()` decreases the occupancy by 1.

Therefore, the buffer works in FIFO (First-In-First-Out) order: e.g., if `write()` is invoked to append 1, and then invoked again to append 2, then a subsequent call to `read()` must return 1, and a further call must return 2.

For her application, Alice needs to implement a *limited* buffer that accumulates values up-to a maximum capacity; when the maximum capacity is reached, then further calls to `write()` have no effect, until some value is removed using `read()`.

Her first test program is in file `ex04-main.cpp` and the (incomplete) code of the class is in files `ex04-library.h` and `ex04-library.cpp`. All files are available on DTU Inside and in the next pages.

Structure of the code. Daisy has defined a high-level abstract class `Buffer` with the pure virtual methods `write()`, `read()`, and `occupancy()`.

Tasks. Help Alice by completing the following tasks. **IMPORTANT:** for these tasks you are required to submit *both* files `ex04-library.h` and `ex04-library.cpp`.

- (a) Declare in `ex04-library.h` and sketch in `ex04-library.cpp` a class `LimitedBuffer` that extends `Buffer`. This task is completed (and passes CodeJudge tests) when `ex04-main.cpp` compiles without errors. To achieve this, you will need to:
1. define a constructor for `LimitedBuffer` that takes 2 parameters:
 - (i) an `unsigned integer` representing the maximum buffer capacity; and
 - (ii) a value of type `int` representing a default (it is used in point (c) below);
 2. in `LimitedBuffer`, override the *pure virtual methods* of `Buffer` (i.e., those with “=0”), and write (possibly non-working) placeholder implementations.

This exercise continues on the next page...

- (b) This is a follow-up to point (a) above. In `ex04-library.cpp`, write a working implementation of the methods:

```
void LimitedBuffer::write()
unsigned int LimitedBuffer::occupancy()
```

Their intended behaviour is that each time `write()` is invoked, the value returned by `occupancy()` increases by 1, until the maximum capacity is reached (such maximum capacity is specified in the constructor — see point (a)1(i) above). While the occupancy is at its maximum, calls to `write()` have no effect.

- (c) This is a follow-up to points (a) and (b) above. In `ex04-library.cpp`, write a working implementation of the method:

```
int LimitedBuffer::read()
```

When `read()` is invoked, it removes the oldest value previously added by `write()`, and returns it; correspondingly, the value returned by `occupancy()` decreases by 1.

Special case: if the buffer is empty, then `read()` must return the default value specified in the constructor (see point (a)1(ii) above).

NOTE: you are free to define the `private` members of `LimitedBuffer` however you see fit. For instance, you might choose to store the values in a `vector<int>`, or in a linked list. The tests will only consider the behaviour of the public methods `write()`, `read()`, and `occupancy()`.

02393 Programming in C++

File ex04-main.cpp

```
#include <iostream>
#include "ex04-library.h"
using namespace std;

int main() {
    Buffer *b = new LimitedBuffer(5, -999);

    cout << "Current_buffer_occupancy:" << b->occupancy() << endl;
    cout << "Reading_from_the_buffer_returns:" << b->read() << endl;

    for (unsigned int i = 0; i < 10; i++) {
        b->write(i * 10);
    }
    cout << "Current_buffer_occupancy:" << b->occupancy() << endl;

    for (unsigned int i = 0; i < 3; i++) {
        cout << "Reading_from_the_buffer_returns:" << b->read() << endl;
    }
    cout << "Current_buffer_occupancy:" << b->occupancy() << endl;

    for (unsigned int i = 0; i < 10; i++) {
        b->write((i+1) * 100);
    }
    cout << "Current_buffer_occupancy:" << b->occupancy() << endl;
    for (unsigned int i = 0; i < 6; i++) {
        cout << "Reading_from_the_buffer_returns:" << b->read() << endl;
    }

    delete b;
    return 0;
}
```

02393 Programming in C++

File ex04-library.h

```
#ifndef EX04_LIBRARY_H_
#define EX04_LIBRARY_H_

#include <vector>
using namespace std;

class Buffer {
public:
    virtual void write(int v) = 0;
    virtual int read() = 0;
    virtual unsigned int occupancy() = 0;
    virtual ~Buffer();
};

// Task 4(a). Declare the class LimitedBuffer, by extending Buffer
// Write your code here

#endif /* EX04_LIBRARY_H_ */
```

File ex04-library.cpp

```
#include "ex04-library.h"

// Task 4(a). Write a placeholder implementation of LimitedBuffer's
// constructor and methods

// Task 4(b). Write a working implementation of write() and occupancy()

// Task 4(c). Write a working implementation of read()

// Do not modify
Buffer::~Buffer() {
    // Empty destructor
}
```

Technical University of Denmark

Written examination date: 21 December 2021



Course title: Programming in C++

Page 1 of 16 pages

Course number: 02393

Aids allowed: All aids allowed

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises with 3 or 4 tasks each, for a total of 14 tasks

Submission details:

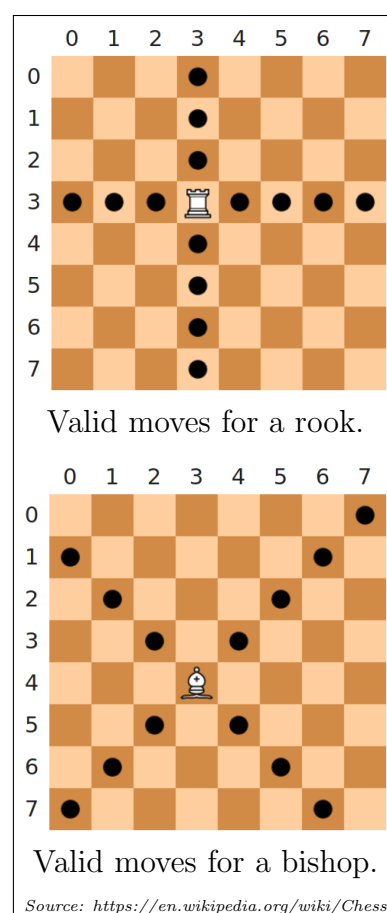
1. You must **submit your solution on DTU Digital Eksamen**. You can do it **only once**, so submit only when you have completed your work.
2. You must submit your solution as **one ZIP archive** containing the following files, with these exact names:
 - `exZZ-library.cpp`, where ZZ ranges from 01 to 04 (i.e., one per exercise);
 - `ex04-library.h` (additionally required for exercise 4).
3. You can test your solutions by uploading them on CodeJudge, under “Exam December 2021” at:
<https://dtu.codejudge.net/02393-e21/exercises>
4. You can test your solutions on CodeJudge as many times as you like. *Uploads on CodeJudge are not official submissions* and will not affect your grade.
5. Additional tests may be run on your submissions after the exam.
6. Feel free to add comments to your code.
7. **Suggestion:** read all exercises before starting your work; start by solving the tasks that look easier, even if they belong to different exercises.

EXERCISE 1. MINI CHESS

Alice wants to implement a variant of the game of chess:

- there are two opposing teams: black and white;
- the chessboard can have any size of $n \times n$ squares ($n \geq 2$);
- each team has two kinds of pieces on the chessboard:
 - rooks, who move horizontally and vertically by any number of squares (see figure on the right);
 - bishops, who move diagonally by any number of squares (see figure on the right);
- pieces can traverse already-occupied squares while moving (unlike standard chess rules);
- a piece can only end its movement on a square that is empty, or occupied by an opponent's piece. In the second case, the opponent's piece is *captured* (i.e. removed).

Alice has already written some code. Her first test program is in file `ex01-main.cpp` and the (incomplete) code with some functions she needs is in files `ex01-library.h` and `ex01-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and they are also reported in the next pages.



Structure of the code. A square on the board is represented as a `struct Square` with two fields named `piece` and `team`, which are two `enums` representing, respectively:

- which piece (if any) is occupying the square: `rook`, `bishop`, or `none`;
- which team (if any) owns the piece: `black`, `white`, or `nobody` (if the piece is `none`).

Alice's code already includes the function:

```
void deleteChessboard(Square **c, unsigned int n)
```

which deallocates a chessboard `c` allocated with `createChessboard()` (see task [\(a\)](#) below).

Tasks. Help Alice by completing the following tasks. You need to edit and submit the file `ex01-library.cpp`.

(a) Implement the function:

```
Square** createChessboard(unsigned int n)
```

The function must return an array of $n \times n$ `Squares`, i.e., `Square**`. It must allocate the required memory, and initialise each square to be empty (i.e. having `none` as piece and `nobody` as team).

This exercise continues on the next page...

(b) Implement the function:

```
void displayChessboard(Square **c, unsigned int n)
```

The function must print on screen the contents of the chessboard `c` of size $n \times n$:

- each empty square must be displayed as `_` (underscore);
- if a square is occupied by a piece, it must be displayed as either:
 - `R` (if occupied by a black rook) or `r` (if occupied by a white rook);
 - `B` (if occupied by a black bishop) or `b` (if occupied by a white bishop);
- adjacent squares on a same row must be separated by one space.

Example. A 3×3 chessboard might look as follows: (position (0,0) on the top-left)

```
b _ r
_ _ _
R B _
```

(c) Implement the function:

```
bool move(Square **c, unsigned int n, int r1, int c1, int r2, int c2)
```

- Argument `c` is a chessboard of size $n \times n$;
- Arguments `r1` and `c1` are a row and column position on the chessboard;
- Arguments `r2` and `c2` are a row and column position on the chessboard.

The function attempts to move a piece on the chessboard `c` from position `(r1,c1)` into position `(r2,c2)`. The function must check whether the move is valid, i.e.:

- there is a piece at position `(r1,c1)`;
- positions `(r1,c1)` and `(r2,c2)` are different and not occupied by the same team;
- the move respects the game rules (see beginning of the exercise).

If the move is valid, the function must update the chessboard `c` and return `true`; otherwise, it must return `false` without altering the chessboard.

You can assume that positions `(r1,c1)` and `(r2,c2)` are within the chessboard bounds.

Example. Assume that `c` is the chessboard shown in Task [\(b\)](#) above:

- `move(c, 3, 0, 0, 1, 0)` must return `false` (bishops only move diagonally);
- `move(c, 3, 2, 0, 2, 1)` must return `false` (same team on both positions);
- `move(c, 3, 2, 0, 0, 0)` must return `true` (black rook captures white bishop);
- `move(c, 3, 2, 1, 1, 2)` must return `true` (the bishop move is valid).

Hints. Positions `(r1,c1)` and `(r2,c2)` are on the same diagonal if they differ by equal (absolute) numbers of rows and columns. For example: positions (4,5) and (6,3) are on the same diagonal, because $|4 - 6| = |5 - 3| = 2$. This check (and others) are also needed in Task [\(d\)](#) below: with a bit of planning, you can avoid code duplication...

This exercise continues on the next page...

(d) Implement the function:

```
bool threatened(Square **c, unsigned int n, int row, int col)
```

Where:

- argument `c` is a chessboard of size $n \times n$;
- arguments `row` and `col` are a row and column position on the chessboard.

The function must return `true` if at position `(row, col)` of chessboard `c` there is a piece that can be captured by another piece on the same chessboard (see game rules at the beginning of the exercise). Otherwise, the function must return `false`. In both cases, the function must *not* change the chessboard.

You can assume that the position `(row, col)` is within the chessboard bounds.

Example. Assume that `c` is the chessboard shown in Task [\(b\)](#) above:

- `threatened(c, 3, 0, 0)` must return `true` (can be captured by rook at (2,0));
- `threatened(c, 3, 0, 2)` must return `false` (no piece can capture at (0,2));
- `threatened(c, 3, 1, 1)` must return `false` (there is no piece at (1,1)).

Hint: this function must perform several checks that are also needed in Task [\(c\)](#) above (see also its hints). With a bit of planning, you can avoid code duplication...

02393 Programming in C++

File ex01-main.cpp

```
#include <iostream>
#include "ex01-library.h"
using namespace std;

int main() {
    Square **c = createChessboard(3);
    c[0][0] = {bishop, white};
    c[0][2] = {rook, white};
    c[2][0] = {rook, black};
    c[2][1] = {bishop, black};
    cout << "Chessboard:" << endl;
    displayChessboard(c, 3);

    cout << "Is the piece in (0,2) threatened?"
    if (threatened(c, 3, 0, 2)) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    cout << "Can we move from (0,0) to (1,0)?"
    if (move(c, 3, 0, 0, 1, 0)) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    cout << "Can we move from (2,0) to (2,1)?"
    if (move(c, 3, 2, 0, 2, 1)) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    cout << "Can we move from (2,0) to (0,0)?"
    if (move(c, 3, 2, 0, 0, 0)) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    cout << "Can we move from (2,1) to (1,2)?"
    if (move(c, 3, 2, 1, 1, 2)) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    cout << endl << "The chessboard is now:" << endl;
    displayChessboard(c, 3);

    cout << "Is the piece in (0,2) threatened?"
    if (threatened(c, 3, 0, 2)) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    deleteChessboard(c, 3);
    return 0;
}
```

File ex01-library.h

```
#ifndef EX01_LIBRARY_H_
#define EX01_LIBRARY_H_

enum Piece { rook, bishop, none };
enum Team { black, white, nobody };

struct Square {
    Piece piece;
    Team team;
};

Square **createChessboard(unsigned int n);
void displayChessboard(Square **c, unsigned int n);
bool move(Square **c, unsigned int n, int r1, int c1, int r2, int c2);
bool threatened(Square **c, unsigned int n, int row, int col);
void deleteChessboard(Square **c, unsigned int n);

#endif /* EX01_LIBRARY_H_ */
```

File ex01-library.cpp

```
#include <iostream>
#include "ex01-library.h"

using namespace std;

// Task 1(a). Implement this function
Square **createChessboard(unsigned int n) {
    // Replace the following with your code
    return nullptr;
}

// Task 1(b). Implement this function
void displayChessboard(Square **c, unsigned int n) {
    // Write your code here
}

// Task 1(c). Implement this function
bool move(Square **c, unsigned int n,
          int r1, int c1, int r2, int c2) {
    // Replace the following with your code
    return false;
}

// Task 1(d). Implement this function
bool threatened(Square **c, unsigned int n,
                int row, int col) {
    // Replace the following with your code
    return false;
}

// Do not modify
void deleteChessboard(Square **c, unsigned int n) {
    for (unsigned int i = 0; i < n; i++) {
        delete[] c[i];
    }
    delete[] c;
}
```

EXERCISE 2. AIRLINE PASSENGERS QUEUE

Bob is writing a program to manage airline passengers boarding a flight. Passengers queue in order of arrival, but they may board the flight in a different order, depending on whether they bought a Priority add-on. Bob decides to represent the queue as a linked list.

Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and they are also reported in the next pages.

Structure of the code. A queue element is represented as a `struct Passenger` with 4 fields: `name`, `ticket`, `priority`, and `next`. Such fields represent, respectively, the passenger name, the ticket number, whether the passenger has Priority (`true` or `false`), and a pointer to the next passenger in the queue (or `nullptr` when there are no more passengers). An empty queue is represented as a `Passenger*` pointer equal to `nullptr`. Bob's code already includes a function to print the passengers queue on screen:

```
void displayQueue(Passenger *q)
```

Tasks. Help Bob by completing the following tasks. You need to edit and submit the file `ex02-library.cpp`. **NOTE:** some tasks may be easier to solve using recursion, but you can use iteration if you prefer.

(a) Implement the function:

```
Passenger* find(Passenger *q, unsigned int ticket)
```

which returns a pointer to the `Passenger` in the queue `q` that has the given `ticket`. You can assume that at most one passenger has that `ticket`; if no passenger has it, the function must return `nullptr`. **Important:** the function must *not* modify `q`.

(b) Implement the function:

```
Passenger* remove(Passenger *q, unsigned int ticket)
```

which returns a passenger queue including all passengers in `q` (in the same order) *except* the passenger with the given `ticket`. You can assume that at most one passenger has that `ticket`. If no passenger owns that ticket, the returned list must have the same passengers of `q`, in the same order.

Important: the function must *not* use `delete` on any element of `q`. Besides this, you can choose to implement the function by either creating and returning a new list of passengers, or modifying `q`.

This exercise continues on the next page...

(c) Implement the function:

```
Passenger* priority(Passenger *q)
```

which returns a *new* passenger queue containing a *copy* of all passengers in the queue **q** who have bought Priority, and in the same order they have in **q**.

Important: the function must *not* modify **q**.

02393 Programming in C++

File ex02-library.h

```
#ifndef EX02_LIBRARY_H_
#define EX02_LIBRARY_H_

#include <string>

struct Passenger {
    std::string name;
    unsigned int ticket;
    bool priority;
    Passenger *next;
};

void displayQueue(Passenger *q);

Passenger* find(Passenger *q, unsigned int ticket);
Passenger* remove(Passenger *q, unsigned int ticket);
Passenger* priority(Passenger *q);

#endif /* EX02_LIBRARY_H_ */
```

File ex02-main.cpp

```
#include <iostream>
#include <string>
#include "ex02-library.h"
using namespace std;

int main() {
    Passenger p0 = {"AlfredA.", 123, false, nullptr};
    Passenger p1 = {"BarbaraB.", 321, true, &p0};
    Passenger p2 = {"CharlieC.", 456, true, &p1};
    Passenger p3 = {"DariaD.", 654, false, &p2};
    Passenger p4 = {"EmilE.", 789, false, &p3};
    Passenger p5 = {"FionaF.", 987, true, &p4};

    Passenger *q = &p5;

    cout << "The passengers queue is:" << endl;
    displayQueue(q);
    cout << endl;

    cout << "The passenger with ticket 654 is:";
    Passenger *pp = find(q, 654);
    if (pp == nullptr) {
        cout << "nobody!" << endl;
    } else {
        cout << pp->name << endl;
    }

    Passenger* q2 = remove(q, 654);
    cout << "After removing the passenger with ticket 654, the queue is:" << endl;
    displayQueue(q2);
    cout << endl;

    Passenger *qp = priority(q2);
    cout << "The queue of priority passengers is:" << endl;
    displayQueue(qp);

    return 0;
}
```

File ex02-library.cpp

```
#include <iostream>
#include "ex02-library.h"
using namespace std;

// Task 2(a). Implement this function
Passenger* find(Passenger *q, unsigned int ticket) {
    // Replace the following with your code
    return nullptr;
}

// Task 2(b). Implement this function
Passenger* remove(Passenger *q, unsigned int ticket) {
    // Replace the following with your code
    return nullptr;
}

// Task 2(c). Implement this function
Passenger* priority(Passenger *q) {
    // Replace the following with your code
    return nullptr;
}

// Do not modify
void displayQueue(Passenger *q) {
    if (q == nullptr) {
        return;
    }
    cout << q->name << " ticket:" << q->ticket;
    if (q->priority) {
        cout << " (priority)";
    }
    cout << endl;
    displayQueue(q->next);
}
```

EXERCISE 3. HOTEL MANAGEMENT

Claire owns a fancy hotel where every room has the name of a flower. She is writing a class `Hotel` to manage the information about the rooms and guests. She has already written some code: her first test program is in file `ex03-main.cpp` and the (incomplete) code of the class is in files `ex03-library.h` and `ex03-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and they are also reported in the next pages.

Structure of the code. Claire has represented the information about a guest using a `struct Guest`, with two fields:

- `name`: the full name of the guest;
- `id`: the document id provided by the guest.

Claire knows that the `map` and `vector` containers of the C++ standard library provide many functionalities she needs. (*See hints on page 10.*) Therefore, she has decided to use the following internal (`private`) representation for the library:

- `vector<string> roomNames` — the names of the fancy hotel rooms;
- `map<string, Guest> roomOccupancy` — a mapping from `strings` (room names) to instances of `Guest` (info about the guest occupying the room, if any). When a room name does not appear in this mapping, it means that the room is empty and available.

Claire has already implemented the default constructor of `Hotel`, which creates a database with all rooms. She has also implemented the method `display()`, which shows which guest occupies which room.

Tasks. Help Claire by completing the following tasks. You need to edit and submit the file `ex03-library.cpp`.

(a) Implement the following method to add a room name:

```
void Hotel::addRoom(string name)
```

The method must work as follows:

- (a) if `name` is *not* in `roomNames`, add the given `name` at the end of `roomNames`;
- (b) if `name` is already in `roomNames`, do nothing.

This exercise continues on the next page...

(b) Implement the following method to add a guest:

```
void Hotel::addGuest(string roomName, string guestName, string guestId)
```

The method must work as follows:

- (a) if `roomName` is *not* in `roomNames`, do nothing;
- (b) otherwise:
 - if the room `roomName` is already occupied by some guest, do nothing;
 - if the room `roomName` is available, then:
 - if there is already a room with a guest having the given `guestId`, do nothing (document ids cannot be duplicated!);
 - otherwise, update the map `roomOccupancy` to assign the given room to the given guest.

(c) Implement the method:

```
void Hotel::findRoomByGuest(string guestName, string guestId)
```

This method displays the name(s) of the room(s) with an occupant who has the given guest name and id.

The room names must be displayed one-per-line, by following their order in `roomNames`.

The string `"*"` can be passed as `guestName` or `guestId` to match anything. Therefore:

- `hotel.findRoomByGuest("*", "*")` must display all hotel rooms having a guest;
- `hotel.findRoomByGuest("Ed_Wood", "*")` must display all hotel rooms occupied by a guest called Ed Wood;
- `hotel.findRoomByGuest("Ed_Wood", "123abc")` must display all hotel rooms occupied by a guest called Ed Wood with id `"123abc"`.

Hints on using maps (See also: <https://www.cplusplus.com/reference/map/map/>)

- A key `k` in a map `m` can be mapped to `v` with: `m[k] = v`; with this operation, the entry for `k` in `m` is created (if not already present) or updated (if already present).
- To check if key `k` is present in map `m`, you can check: `m.find(k) != m.end()`.
- The value mapped to a key `k` in a map `m` is obtained with: `m[k]`.
- To loop on all (key, value) pairs in a map `m`, you can use: `for (auto p: m) { ... }`. The loop variable `p` is a pair with the map key as `p.first`, and the corresponding value as `p.second` (see <https://www.cplusplus.com/reference/utility/pair/>)

02393 Programming in C++

File ex03-main.cpp

```
#include <iostream>
#include "ex03-library.h"
using namespace std;

int main() {
    Hotel hotel = Hotel();

    cout << "Initial hotel occupancy:" << endl;
    hotel.display();

    hotel.addRoom("Waterlily");
    cout << endl << "After adding room 'Waterlily':" << endl;
    hotel.display();

    hotel.addGuest("Waterlily", "Taika Waititi", "pqr567");
    cout << endl << "After adding a guest:" << endl;
    hotel.display();

    cout << endl << "Room(s) occupied by someone called Alan Smithee:" << endl;
    hotel.findRoomByGuest("Alan Smithee", "*");

    return 0;
}
```

File ex03-library.h

```
#ifndef EX03_LIBRARY_H_
#define EX03_LIBRARY_H_

#include <string>
#include <vector>
#include <map>
using namespace std;

struct Guest {
    string name;
    string id;
};

class Hotel {
private:
    vector<string> roomNames;
    map<string, Guest> roomOccupancy;
public:
    Hotel();
    void addRoom(string name);
    void addGuest(string roomName, string guestName, string guestId);
    void findRoomByGuest(string guestName, string guestId);
    void display();
};

#endif /* EX03_LIBRARY_H_ */
```

02393 Programming in C++

File ex03-library.cpp

```
#include <iostream>
#include "ex03-library.h"
using namespace std;

// Do not modify
Hotel::Hotel() {
    this->roomNames.push_back("Daisy");
    this->roomOccupancy["Daisy"] = {"Alan_Smithee", "xyz890"};

    this->roomNames.push_back("Geranium");

    this->roomNames.push_back("Lotus");
    this->roomOccupancy["Lotus"] = {"Kathryn_Bigelow", "456abc"};

    this->roomNames.push_back("Orchid");
    this->roomOccupancy["Orchid"] = {"Alan_Smithee", "abc123"};

    this->roomNames.push_back("Tulip");
    this->roomOccupancy["Tulip"] = {"Denis_Villeneuve", "123xyz"};
}

// Task 3(a). Implement this method
void Hotel::addRoom(string name) {
    // Write your code here
}

// Task 3(b). Implement this method
void Hotel::addGuest(string roomName, string guestName, string guestId) {
    // Write your code here
}

// Task 3(c). Implement this method
void Hotel::findRoomByGuest(string guestName, string guestId) {
    // Write your code here
}

// Do not modify
void Hotel::display() {
    for (auto it = this->roomNames.begin(); it != this->roomNames.end(); it++) {
        cout << "Room_" << *it << "_is_";
        if (this->roomOccupancy.find(*it) == this->roomOccupancy.end()) {
            cout << "empty" << endl;
        } else {
            cout << "occupied_by_" << this->roomOccupancy[*it].name;
            cout << "_id:" << this->roomOccupancy[*it].id << ")" << endl;
        }
    }
}
```

EXERCISE 4. SENSOR DATA BUFFER

Daisy is writing a program that reads `integer` values from a sensor. Her program may need to read either the most recent value obtained from the sensor, or the average of the most recent values. Therefore, she plans a `SensorBuffer` class with the following interface:

- `write(v)`: appends value `v` (obtained from the sensor) into the buffer;
- `read()`: returns the most recent value written in the buffer;
- `readAvg()`: returns the average of the most recent values written in the buffer. The average is computed using up to n values, where n is a parameter given to the `SensorBuffer` constructor (see below);
- `writeCount()`: returns how many times the method `write()` has been invoked since the buffer was created.

Daisy's first test program is in the file `ex04-main.cpp` and the (incomplete) code of the class is in files `ex04-library.h` and `ex04-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and they are also reported in the next pages.

Structure of the code. Daisy has defined a high-level abstract class `Buffer` with the pure virtual methods `write()` and `read()`. She wants to implement `SensorBuffer` as a subclass of `Buffer`, with the additional methods `readAvg()` and `writeCount()`.

Example. Once completed, the class `SensorBuffer` must work as follows:

- suppose that we create `buf = SensorBuffer(4)` (i.e. `buf` uses up to 4 values to compute an average);
- suppose that `buf.write(1)` is invoked, followed by `buf.write(2)`. Then, a call to `buf.read()` must return 2, and a further call to `buf.read()` must still return 2; (therefore, `read()` does *not* remove the returned value from the buffer)
- then, suppose that `buf.write(4)` is invoked, followed by `write(5)`. Then, a subsequent call to `buf.readAvg()` must return 3 (i.e. $(1 + 2 + 4 + 5) \div 4$);
- finally, suppose that `buf.write(5)` is invoked. Then, `buf.read()` must return 5, and `buf.readAvg()` must return 4 (i.e. $(2 + 4 + 5 + 5) \div 4$).

Tasks. Help Daisy by completing the following tasks. You need to edit and submit **two files**: `ex04-library.h` and `ex04-library.cpp`.

NOTE: you are free to define the `private` members of `SensorBuffer` however you see fit. For instance, you might choose to store the values in a `vector<int>`, or in a linked list. The tests will only consider the behaviour of the public methods `write()`, `read()`, `readAvg()`, and `writeCount()`.

This exercise continues on the next page...

(a) Declare in `ex04-library.h` and sketch in `ex04-library.cpp` a class `SensorBuffer` that extends `Buffer`. This task is completed (and passes CodeJudge tests) when `ex04-main.cpp` compiles without errors. To achieve this, you will need to:

1. define a `SensorBuffer` constructor that takes one parameter: an `unsigned int` representing the number of values used to compute averages (used in Task (d));
2. in `SensorBuffer`, override the *pure virtual methods* of `Buffer` (i.e., those with “=0”), and add the following `public` methods to the class interface:
 - `int readAvg()`
 - `unsigned int writeCount()`
3. finally, write a placeholder implementation of all the `SensorBuffer` methods above (e.g. they may do nothing and/or just return 0 when invoked).

(b) This is a follow-up to task (a) above. In `ex04-library.cpp`, write a working implementation of the methods:

```
void SensorBuffer::write(int v)
unsigned int SensorBuffer::writeCount()
```

The intended behaviour of `write(v)` is to store value `v` (so it can be used by `read()` and `readAvg()`) and increment an internal counter used by `writeCount()`. The method `writeCount()` returns how many times the method `write()` has been invoked since the buffer was created.

(c) This is a follow-up to tasks (a) and (b) above. In `ex04-library.cpp`, write a working implementation of the method:

```
int SensorBuffer::read()
```

When invoked, `read()` returns the latest value written in the buffer using `write()`.

Special case: if `write()` was never used before, then `read()` must return 0.

This exercise continues on the next page...

- (d) This is a follow-up to task (a) and (b) above. In `ex04-library.cpp`, write a working implementation of the method:

```
int SensorBuffer::readAvg()
```

When invoked, `readAvg()` returns the average of latest n values written in the buffer using `write()` — where n is the constructor parameter described in task (a)1 above.

Special cases:

- if `write()` was never used before, then `readAvg()` must return 0;
- if `write()` has only been called m times with $0 < m < n$, then the returned average must be computed using the latest m values.

02393 Programming in C++

File ex04-main.cpp

```
#include <iostream>
#include "ex04-library.h"
using namespace std;

int main() {
    SensorBuffer *sb = new SensorBuffer(4);
    Buffer *b = sb; // Just an alias for 'sb' above, but using the superclass

    cout << "Current_write_count:_\n" << sb->writeCount() << endl;
    cout << "Reading_from_the_buffer_returns:_\n" << b->read() << endl;

    b->write(1); b->write(2);
    cout << "Current_write_count:_\n" << sb->writeCount() << endl;
    cout << "Reading_from_the_buffer_now_returns:_\n" << b->read() << endl;

    b->write(4); b->write(5);
    cout << "Current_write_count:_\n" << sb->writeCount() << endl;
    cout << "Reading_from_the_buffer_now_returns:_\n" << b->read() << endl;
    cout << "The_buffer_average_is_now:_\n" << sb->readAvg() << endl;

    b->write(5);
    cout << "Current_write_count:_\n" << sb->writeCount() << endl;
    cout << "Reading_from_the_buffer_now_returns:_\n" << b->read() << endl;
    cout << "The_buffer_average_is_now:_\n" << sb->readAvg() << endl;

    delete sb;
    return 0;
}
```

File ex04-library.h

```
#ifndef EX04_LIBRARY_H_
#define EX04_LIBRARY_H_

class Buffer {
public:
    virtual void write(int v) = 0;
    virtual int read() = 0;
    virtual ~Buffer();
};

// Task 4(a). Declare the class SensorBuffer, by extending Buffer
// Write your code here

#endif /* EX04_LIBRARY_H_ */
```

File ex04-library.cpp

```
#include "ex04-library.h"

// Task 4(a). Write a placeholder implementation of SensorBuffer's
// constructor and methods

// Task 4(b). Write a working implementation of write() and writeCount()

// Task 4(c). Write a working implementation of read()

// Task 4(d). Write a working implementation of readAvg()

// Do not modify
Buffer::~Buffer() {
    // Empty destructor
}
```

Technical University of Denmark

Written examination date: 17 May 2021



Course title: Programming in C++

Page 1 of 15 pages

Course number: 02393

Aids allowed: All aids allowed

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises of 2.5 points each, for a total of 10 points.

Submission details:

1. You must **submit your solution on DTU Digital Eksamen**. You can do it **only once**, so submit only when you have completed your work.
2. You must submit your solution as **one ZIP archive** containing the following files, with these exact names:
 - `exZZ-library.cpp`, where ZZ ranges from 01 to 04 (i.e., one per exercise);
 - `ex04-library.h` (additionally required for exercise 4).
3. You can test your solutions by uploading them on CodeJudge, under “Reexam May 2021” at:
<https://dtu.codejudge.net/02393-e20/exercises>
4. You can test your solutions on CodeJudge as many times as you like. *Uploads on CodeJudge are not official submissions* and will not affect your grade.
5. Additional tests may be run on your submissions after the exam.
6. Feel free to add comments to your code.
7. **Suggestion:** read all exercises before starting your work, and begin with the tasks that look easier.

EXERCISE 1. VECTOR FIELDS (2.5 POINTS)

Alice needs to perform computations on *vector fields*, i.e., matrices having bidimensional geometric vectors as elements. Alice has already written some code. Her first test program is in file `ex01-main.cpp` and the (incomplete) code with some functions she needs is in files `ex01-library.h` and `ex01-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and they are also reported in the next pages.

Structure of the code. A geometric vector is represented as a `struct Vector` with two fields, named `x` and `y`: they are, respectively, the x and y component of the vector. Alice's code already includes the function:

```
void deleteField(Vector **A, unsigned int nRows)
```

which deallocates a vector field allocated with `createField()` (see task (a) below).

Tasks. Help Alice by completing the following tasks. You need to edit and submit the file `ex01-library.cpp`.

(a) Implement the function:

```
Complex **createField(unsigned int m, unsigned int n, Vector v)
```

The function must return an array of $m \times n$ Vectors, i.e., `Vector **`. It must allocate the required memory, and initialise each array element as argument `v`.

(b) Implement the function:

```
void displayField(Vector **A, unsigned int m, unsigned int n)
```

The function must print on screen the contents of the vector field `A` of size $m \times n$:

- each vector must be printed as (x,y) *without* spaces between the x,y field values;
- elements on a same row must be separated by one space;
- there must be no space after the last element of each row.

For example, a 2×4 vector field should look like:

```
(1,2) (2,5) (4,4) (1,2)
(1,2) (0,2) (0,2) (2,6)
```

This exercise continues on the next page...

(c) Implement the function:

```
void addFields(Vector **A, Vector **B, Vector **C,
               unsigned int m, unsigned int n)
```

Where:

- argument **A** is a vector field of size $m \times n$;
- argument **B** is a vector field of size $m \times n$;
- argument **C** is a vector field of size $m \times n$.

The function must add the corresponding elements of **A** by **B**, storing the result in **C**. Therefore, as in standard matrix addition, the element at row i and column j of **C** is computed as:

$$C_{i,j} = A_{i,j} + B_{i,j}$$

where “+” is the standard vector addition: the addition of two **Vectors** **u** and **v** is a **Vector** whose fields have values $u.x + v.x$ and $u.y + v.y$.

(d) Implement the function:

```
void scaleField(Vector **A, double c, unsigned int m, unsigned int n)
```

Where:

- argument **A** is a vector field of size $m \times n$;
- argument **c** is a scalar value.

The function must multiply each element of **A** by **c**, storing the result in **A** itself. More precisely, the element at row i and column j of **A** must be updated as follows:

$$A_{i,j} = A_{i,j} \times c$$

where “ $\times$ ” is the standard vector scalar multiplication: to multiply a **Vector** **v** by a scalar **c**, we multiply both $v.x$ and $v.y$ by **c**.

02393 Programming in C++

File ex01-main.cpp

```
#include <iostream>
#include "ex01-library.h"
using namespace std;

int main() {
    Vector c = {1, 2};
    Vector d = {2, -2};

    Vector **A = createField(3, 3, c);
    A[1][1] = {2, 2};
    cout << "Vector_field_A:" << endl;
    displayField(A, 3, 3);
    cout << endl;

    Vector **B = createField(3, 3, d);
    B[0][0] = B[2][2] = {9, 8};
    cout << "Vector_field_B:" << endl;
    displayField(B, 3, 3);
    cout << endl;

    Vector **R = createField(3, 3, {0,0});
    cout << "Result_of_A+B:" << endl;
    addFields(A, B, R, 3, 3);
    displayField(R, 3, 3);
    cout << endl;

    cout << "Result_of_scaling_A_by_2:" << endl;
    scaleField(A, 2, 3, 3);
    displayField(A, 3, 3);

    deleteField(A, 3); deleteField(B, 3);
    deleteField(R, 3);
    return 0;
}
```

File ex01-library.cpp

```
#include <iostream>
#include "ex01-library.h"

using namespace std;

// Task 1(a). Implement this function
Vector **createField(unsigned int m, unsigned int n, Vector v) {
    // Write your code here
}

// Task 1(b). Implement this function
void displayField(Vector **A, unsigned int m, unsigned int n) {
    // Write your code here
}

// Task 1(c). Implement this function
void addFields(Vector **A, Vector **B, Vector **C,
               unsigned int m, unsigned int n) {
    // Write your code here
}

// Task 1(d). Implement this function
void scaleField(Vector **A, double c,
                unsigned int m, unsigned int n) {
    // Write your code here
}

// Do not modify
void deleteField(Vector **A, unsigned int nRows) {
    for (unsigned int i = 0; i < nRows; ++i) {
        delete[] A[i];
    }
    delete[] A;
}
```

File ex01-library.h

```
#ifndef EX01_LIBRARY_H_
#define EX01_LIBRARY_H_

struct Vector {
    double x;
    double y;
};

Vector **createField(unsigned int m, unsigned int n, Vector v);
void displayField(Vector **A, unsigned int m, unsigned int n);
void addFields(Vector **A, Vector **B, Vector **C,
               unsigned int m, unsigned int n);
void scaleField(Vector **A, double c, unsigned int m, unsigned int n);
void deleteField(Vector **A, unsigned int nRows);

#endif /* EX01_LIBRARY_H_ */
```

EXERCISE 2. RLE LINKED LIST (2.5 POINTS)

Bob wants to build a linked list with a compression technique called *Run-Length Encoding (RLE)*: each element of the list records on how many times its value is repeated. For instance, the following sequence of values

1 1 25 3 3 3 3 3 42 42 5 5 5 5 5 5 5 5 5 5 42 42 42 42 42 42 42 42

is compressed with RLE as a sequence of values with their respective number of repetitions:

$1_{(\times 2)}$ $25_{(\times 1)}$ $3_{(\times 5)}$ $42_{(\times 2)}$ $5_{(\times 10)}$ $42_{(\times 8)}$

Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and they are also reported in the next pages.

Structure of the code. An RLE list element is represented as a `struct Elem` with three fields, named `value`, `times`, and `next`: they are, respectively, the value of the list element, the number of times that value is repeated, and the pointer to the next list element (or `nullptr` when there are no more elements). An empty list is represented as an `Elem*` pointer equal to `nullptr`. Bob's code already includes the function:

```
void displayRLEList(Elem *list)
```

which prints an RLE list on screen, in the compressed form shown above.

Tasks. Help Bob by completing the following tasks. You need to edit and submit the file `ex02-library.cpp`.

(a) Implement the function:

```
Elem* reverse(Elem *list);
```

which reverses the RLE list `list` *in place*, that is, by updating the pointers of its elements. The function returns a pointer to the first element of the reversed list (which corresponds to the last element of the original `list`). For example: if the RLE list $7_{(\times 25)} 9_{(\times 90)}$ is reversed, the result is $9_{(\times 90)} 7_{(\times 25)}$.

This exercise continues on the next page...

(b) Implement the function:

```
Elem* concatenate(Elem *list1, Elem *list2)
```

which concatenates the lists `list1` and `list2`, and returns a pointer to the first `Element` of the resulting list. The function must compress the repetitions resulting from the concatenation. For example, if the arguments of the function are:

- `list1 = 7(×2) 6(×1) 9(×2)`
- `list2 = 9(×3) 10(×3)`

then the resulting list must be:

$$7_{(\times 2)} \ 6_{(\times 1)} \ 9_{(\times 5)} \ 10_{(\times 3)}$$

Notice that the last element of `list1` and the first element of `list2` have been compressed into one.

Important: the function must *not* use `delete` on any element of `list1` nor `list2`. Besides this, you can choose to implement the function by either creating and returning a new list, or modifying `list1` and `list2`.

(c) Implement the function:

```
int sum(Elem *list)
```

which returns the sum of the elements of `list`, taking into account their repetitions. For example, if `list` is `7(×2) 6(×1) 9(×2)`, then the function must return 38.

File ex02-library.h

```
#ifndef EX02_LIBRARY_H_
#define EX02_LIBRARY_H_

struct Elem {
    int value;
    unsigned int times; // Number of repetitions
    Elem *next;
};

void displayRLEList(Elem *list);

Elem* reverse(Elem *list);
Elem* concatenate(Elem *list1, Elem *list2);
int sum(Elem *list);

#endif /* EX02_LIBRARY_H_ */
```

File ex02-main.cpp

```
#include <iostream>
#include "ex02-library.h"
using namespace std;

int main() {
    Elem e0 = {10, 5, nullptr};
    Elem e1 = {12, 6, &e0};
    Elem e2 = {4, 10, &e1};

    Elem e4 = {100, 7, nullptr};
    Elem e5 = {4, 3, &e4};
    Elem e6 = {101, 9, &e5};

    cout << "The RLE list is:" << endl;
    displayRLEList(&e2);
    cout << endl;

    cout << "The reversed list is:" << endl;
    Elem *r = reverse(&e2);
    displayRLEList(r);

    cout << endl;

    cout << "After concatenation, the list is:" << endl;
    Elem *l = concatenate(r, &e6);
    displayRLEList(l);
    cout << endl;

    cout << "The sum of its elements is:" << sum(l) << endl;

    return 0;
}
```

File ex02-library.cpp

```
#include <iostream>
#include "ex02-library.h"
using namespace std;

// Task 2(a). Implement this function
Elem* reverse(Elem *list) {
    // Write your code here
}

// Task 2(b). Implement this function
Elem* concatenate(Elem *list1, Elem *list2) {
    // Write your code here
}

// Task 2(c). Implement this function
int sum(Elem *list) {
    // Write your code here
}

// Do not modify
void displayRLEList(Elem *list) {
    if (list == nullptr) {
        return;
    }
    cout << "␣" << list->value << "␣(x" << list->times << ")";
    displayRLEList(list->next);
}
```

EXERCISE 3. GROCERY LIST (2.5 POINTS)

Claire wants to implement a class `GroceryList` to store and update her grocery list. She has already written some code: her first test program is in file `ex03-main.cpp` and the (incomplete) code of the class is in files `ex03-library.h` and `ex03-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and they are also reported in the next pages.

Structure of the code. Claire has represented the information about each entry in the grocery list using a `struct Info`, with two fields:

- **quantity**: how much to buy of a certain item;
- **notes**: any remark about the item.

Claire knows that the `map` and `vector` containers of the C++ standard library provide many functionalities she needs. (See *hints on page 9*.) Therefore, she has decided to use the following internal (`private`) representation for the library:

- `vector<string> items` — the names of the items to buy;
- `map<string,Info> itemsInfo` — a mapping from `strings` (item names) to instances of `Info` (the information about the item to buy).

Claire has already implemented the default constructor of `GroceryList`, which creates a database with some needed items. She has also implemented the method `display()`, which shows the contents of the grocery list.

Tasks. Help Claire by completing the following tasks. You need to edit and submit the file `ex03-library.cpp`.

(a) Implement the following method to add an entry to the grocery list:

```
void GroceryList::add(string name, unsigned int quantity, string notes)
```

The method must work as follows:

- if `name` is *not* in the grocery list, add the given `name` at the end of the `items` vector, and map it to the given `quantity` and `notes` (by updating `itemsInfo`);
- if `name` is already in the grocery list, update its information in `itemsInfo` as follows:
 - increase the original quantity by the given `quantity`. For example: if the original quantity is 100 and the method is invoked with `quantity=200`, the updated quantity must be 300;
 - extend the original notes by adding `";"` and the given `notes`. For example: if the original notes are `"A"` and the method is invoked with `notes="B"`, the updated notes must be `"A;B"`.

This exercise continues on the next page...

(b) Implement the method:

```
bool GroceryList::remove(string name, unsigned int quantity)
```

This method tries to remove the given `quantity` from the grocery list item with the given `name`; it returns `true` if the operation succeeds, and `false` otherwise. The method must work as follows:

- (a) if the grocery list does *not* contain an item with the given `name`, then the method returns `false` without changing the grocery list;
- (b) if the grocery list *does* contain an item with the given `name`, then:
 - if the item's quantity is lower than the given `quantity`, then the method must return `false` without changing the grocery list.
 - otherwise, the method must reduce the item's quantity by subtracting the given `quantity`; then, if the updated item quantity becomes 0, then the method must remove the item from the shopping list. (*See hints below.*) In either case, the method must return `true`.

(c) Implement the method:

```
bool GroceryList::copyEntry(string name, string newName)
```

This method creates a new grocery list entry named `newName`, by copying the information of the item called `name`; it returns `true` if the operation succeeds, and `false` otherwise. The method must work as follows:

- (a) if the grocery list does *not* contain an item with the given `name`, *or* it already contains an item called `newName`, then the method returns `false` without changing the grocery list;
- (b) otherwise, the method must add `newName` at the end of the `items` vector, and update `itemsInfo` to map `newName` to the same information of `name`.

Hints on using maps and vectors

- A key `k` in a map `m` can be mapped to `v` with: `m[k] = v`; with this operation, the entry for `k` in `m` is created (if not already present) or updated (if already present).
- To check if key `k` is present in map `m`, you can check: `m.find(k) != m.end()`.
- The value mapped to a key `k` in a map `m` is obtained with: `m[k]`.
- To remove an element from a map or a vector, you can use their `erase(...)` methods.

02393 Programming in C++

File ex03-main.cpp

```
#include <iostream>
#include "ex03-library.h"
using namespace std;

int main() {
    GroceryList gl = GroceryList();

    cout << "Initial grocery list:" << endl;
    gl.display();

    cout << endl << "After adding cheddar:" << endl;
    gl.add("Cheddar", 500, "Not too mature");
    gl.display();

    cout << endl << "After removing some spinach:" << endl;
    if (gl.remove("Spinach", 200)) {
        gl.display();
    } else {
        cout << "FAILED! (this should not happen)" << endl;
    }

    cout << endl << "After copying salmon into haddock:" << endl;
    if (gl.copyEntry("Salmon", "Haddock")) {
        gl.display();
    } else {
        cout << "FAILED! (this should not happen)" << endl;
    }

    return 0;
}
```

File ex03-library.h

```
#ifndef EX03_LIBRARY_H_
#define EX03_LIBRARY_H_

#include <string>
#include <vector>
#include <map>
using namespace std;

struct Info {
    unsigned int quantity;
    string notes;
};

class GroceryList {
private:
    vector<string> items;
    map<string, Info> itemsInfo;
public:
    GroceryList();
    void add(string name, unsigned int quantity, string notes);
    bool remove(string name, unsigned int quantity);
    bool copyEntry(string name, string newName);
    void display();
};

#endif /* EX03_LIBRARY_H_ */
```


File ex03-library.cpp

```
#include <iostream>
#include "ex03-library.h"
using namespace std;

// Do not modify
GroceryList::GroceryList() {
    this->items.push_back("Lasagne");
    this->itemsInfo["Lasagne"] = {1, "With_eggs_if_available"};

    this->items.push_back("Salmon");
    this->itemsInfo["Salmon"] = {500, "Smoked_if_available"};

    this->items.push_back("Spinach");
    this->itemsInfo["Spinach"] = {300, "Fresh"};

    this->items.push_back("Dessert");
    this->itemsInfo["Dessert"] = {8, "Maybe_lagkage?"};
}

// Task 3(a). Implement this method
void GroceryList::add(string name, unsigned int quantity, string notes) {
    // Write your code here
}

// Task 3(b). Implement this method
bool GroceryList::remove(string name, unsigned int quantity) {
    // Write your code here
}

// Task 3(c). Implement this method
bool GroceryList::copyEntry(string name, string newName) {
    // Write your code here
}

// Do not modify
void GroceryList::display() {
    // Write your code here
    for (auto it = this->items.begin(); it != this->items.end(); it++) {
        Info &item = this->itemsInfo[*it];
        cout << "name=" << *it << " ";
        cout << "quantity=" << item.quantity << " ";
        cout << "notes=" << item.notes << " " << endl;
    }
}
```

EXERCISE 4. FILTERING BUFFER (2.5 POINTS)

Daisy needs to develop a buffer class to store and retrieve `integer` values. She plans an interface consisting of 4 methods:

- `write(v)` — appends value `v` to the buffer;
- `read()` — removes the oldest value from the buffer and returns it;
- `occupancy()` — returns the number buffered values;
- `reset()` — empties the buffer.

Therefore, the buffer works in FIFO (First-In-First-Out) order: e.g., if `write()` is invoked to append 1, and then invoked again to append 2, then a subsequent call to `read()` must return 1, and a further call must return 2.

For her application, Alice needs to implement a *filtering* buffer that accumulates *unique* values, by remembering which values it has contained during its lifecycle. For example, assume that a `FilteringBuffer b` has never contained the value 42:

- the first time `b.write(42)` is called, the value 42 is appended to the buffer contents. From now on, `b` remembers that it has contained 42 — even after 42 is removed by `b.read()`. If `b.write(42)` is executed again, the operation has no effect;
- if `b.reset()` is called, then the buffer `b` is emptied, and it also “forgets” which values it has contained in the past. Therefore, the first call `b.write(42)` after the reset will append 42 to the buffer contents.

Daisy’s first test program is in the file `ex04-main.cpp` and the (incomplete) code of the class is in files `ex04-library.h` and `ex04-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and they are also reported in the next pages.

Structure of the code. Daisy has defined a high-level abstract class `Buffer` with the pure virtual methods `write()`, `read()`, `occupancy()`, and `reset()`.

Tasks. Help Daisy by completing the following tasks. You need to edit and submit **two files**: `ex04-library.h` and `ex04-library.cpp`.

- (a) Declare in `ex04-library.h` and sketch in `ex04-library.cpp` a class `FilteringBuffer` that extends `Buffer`. This task is completed (and passes CodeJudge tests) when `ex04-main.cpp` compiles without errors. To achieve this, you will need to:
1. define a constructor for `FilteringBuffer` that takes one parameter: a value of type `int` representing a default (it is used in point [\(c\)](#) below);
 2. in `FilteringBuffer`, override the *pure virtual methods* of `Buffer` (i.e., those with “=0”), and write (possibly non-working) placeholder implementations.

This exercise continues on the next page...

- (b) This is a follow-up to point (a) above. In `ex04-library.cpp`, write a working implementation of the methods:

```
void FilteringBuffer::write(int v)
unsigned int FilteringBuffer::occupancy()
```

The method `occupancy()` returns the number of values currently stored in the buffer. The intended behaviour of `write(v)` is to check the value `v`, and:

- if the buffer has already contained `v` in the past, then the method has no effect;
- otherwise, the method appends `v` to the buffer contents, and remembers that it has contained `v` (hence, invoking `write(v)` again will have no effect). Correspondingly, the buffer occupancy increases by 1.

- (c) This is a follow-up to points (a) and (b) above. In `ex04-library.cpp`, write a working implementation of the method:

```
int FilteringBuffer::read()
```

When `read()` is invoked, it removes the oldest value previously added by `write()`, and returns it; correspondingly, the value returned by `occupancy()` decreases by 1. Crucially, `read()` must *not* cause the buffer to “forget” which values it has contained in the past: for example, if `b.read()` returns 42, then invoking `b.write(42)` afterwards must have no effect — because the buffer `b` must remember that it has contained the value 42 (although it might not *currently* contain 42).

Special case: if the buffer is empty, then `read()` must return the default value specified in the constructor (see point (a)¹ above).

- (d) This is a follow-up to points (a), (b), and (c) above. In `ex04-library.cpp`, write a working implementation of the method:

```
void FilteringBuffer::reset()
```

When `reset()` is invoked, the buffer becomes empty (hence, its occupancy becomes 0), and it also forgets which values it has contained in the past.

For example: if buffer `b` contains (or has contained) the value 42, then invoking `b.write(42)` has no effect; however, invoking `b.reset()` and then `b.write(42)` causes 42 to be appended to the (empty) buffer.

NOTE: you are free to define the `private` members of `FilteringBuffer` however you see fit. For instance, you might choose to store the values in a `vector<int>`, or in a linked list. Similarly, you are free to choose how to remember which values have been already contained in the buffer. The tests will only consider the behaviour of the public methods `write()`, `read()`, `occupancy()`, and `reset()`.

02393 Programming in C++

File ex04-main.cpp

```
#include <iostream>
#include "ex04-library.h"
using namespace std;

int main() {
    Buffer *b = new FilteringBuffer(-999);

    cout << "Current_buffer_occupancy:" << b->occupancy() << endl;
    cout << "Reading_from_the_buffer_returns:" << b->read() << endl;

    for (unsigned int i = 0; i < 10; i++) {
        b->write(i * 10);
    }
    cout << "Current_buffer_occupancy:" << b->occupancy() << endl;

    for (unsigned int i = 0; i < 10; i++) {
        b->write(20);
    }
    cout << "Current_buffer_occupancy:" << b->occupancy() << endl;

    for (unsigned int i = 0; i < 3; i++) {
        cout << "Reading_from_the_buffer_returns:" << b->read() << endl;
    }
    cout << "Current_buffer_occupancy:" << b->occupancy() << endl;

    b->reset();
    cout << "Current_buffer_occupancy:" << b->occupancy() << endl;
    cout << "Reading_from_the_buffer_returns:" << b->read() << endl;

    delete b;
    return 0;
}
```

02393 Programming in C++

File ex04-library.h

```
#ifndef EX04_LIBRARY_H_
#define EX04_LIBRARY_H_

class Buffer {
public:
    virtual void write(int v) = 0;
    virtual int read() = 0;
    virtual unsigned int occupancy() = 0;
    virtual void reset() = 0;
    virtual ~Buffer();
};

// Task 4(a). Declare the class FilteringBuffer, by extending Buffer
// Write your code here

#endif /* EX04_LIBRARY_H_ */
```

File ex04-library.cpp

```
#include "ex04-library.h"

// Task 4(a). Write a placeholder implementation of FilteringBuffer's
// constructor and methods

// Task 4(b). Write a working implementation of write() and occupancy()

// Task 4(c). Write a working implementation of read()

// Task 4(d). Write a working implementation of reset()

// Do not modify
Buffer::~Buffer() {
    // Empty destructor
}
```

Technical University of Denmark

Written examination date: 22 December 2022



Course title: Programming in C++

Page 1 of 17 pages

Course number: 02393

Aids allowed: All aids allowed

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises with 3 or 4 tasks each, for a total of 14 tasks

Submission details:

1. You must **submit your solution on DTU Digital Eksamen**. You can do it **only once**, so submit only when you have completed your work.
2. You must submit your solution as **one ZIP archive** containing the following files, with **these exact names**:
 - `exZZ-library.cpp`, where `ZZ` ranges from `01` to `04` (i.e., one per exercise);
 - `ex04-library.h` (additionally required for exercise 4).
3. You can test your solutions by uploading them on CodeJudge, under “Exam December 2022” at:
<https://dtu.codejudge.net/02393-e22/exercises>
4. You can test your solutions on CodeJudge as many times as you like. *Uploads on CodeJudge are not official submissions* and will not affect your grade.
5. Additional tests may be run on your submissions after the exam.
6. Feel free to add comments to your code.
7. **Suggestion:** read all exercises before starting your work; start by solving the tasks that look easier, even if they belong to different exercises.

EXERCISE 1. THE WUMPUS

Alice is implementing a variant of the old game *Hunt the Wumpus*: the player explores a randomly-generated cavern with a map made of tiles. A map of size 6×16 might look like **Fig. 1** on the right:

- 'X' is the current position of the player;
- ' ' is an empty tile: the player can move there;
- '#' is a rock: the player cannot move onto that tile;
- 'W' is the Wumpus: a monster living in the cavern.

During the game, the player explores the cavern in the dark, with a torch that only illuminates the nearby tiles. The dark cavern is drawn as shown in **Fig. 2** on the right:

- when a tile is close enough to the player, it is lit by the torch, hence its content is visible and appears as described above. Also, the tile is marked as “explored”;
- when a tile is too far from the player, it is *not* lit by the torch. Therefore:
 - if the tile was *not* explored before, its content is unknown to the player, and it is shown as '?';
 - if the tile was previously explored by the player, then:
 - * if the tile contains a rock, it is shown as '#';
 - * otherwise, it is shown as '-'.

In other words: the game map always shows already-explored rocks '#', even when they are not lit; instead, other explored but unlit tiles are shown as '-' (such tiles may be empty, or contain the Wumpus).

Alice has written some code: a test program is in the file `ex01-main.cpp`, and an (incomplete) implementation is in `ex01-library.h` and `ex01-library.cpp`. Such files are available in the ZIP archive for this exam, and in the next pages.

Structure of the code. A tile of the cavern is represented as a `struct Tile` with two fields named `content` and `explored`, which represent, respectively:

- the content of the tile: and `enum` value between `nothing`, `player`, `rock`, or `wumpus`;
- whether the tile has been already explored by the player.

Alice's code already includes two functions (that you must not modify):

- `void setupCavern(Tile **cav, unsigned int m, unsigned int n, unsigned int seed)`
- `void deleteCavern(Tile **c, unsigned int m)`

These functions respectively insert random contents in a cavern `cav` with `m` rows and `n` columns (created with `createCavern()`, see task **(a)**), or deallocate the cavern.

Tasks. Help Alice by completing the following tasks. You need to edit and submit the file `ex01-library.cpp`.

This exercise continues on the next page...

```
# ##  # #
##      W# ##
      # # ##
      # # #
##      X #
###      ### #
```

Fig. 1. Randomly-generated 6×16 cavern with all tiles visible.

```
----#-##????????
-##--      W??????
----  #  # ??????
---#      #  ??????
##--      X # ??????
###-      ##??????
```

Fig. 2. The cavern above in the dark, with some explored tiles.

(a) Implement the function:

```
Tile** createCavern(unsigned int m, unsigned int n)
```

It must return an array of $m \times n$ Tiles, i.e., `Tile**`. It must allocate the required memory, and initialise each tile with `nothing` as content, and `explored` set to `false`.

(b) Implement the function:

```
void revealCavern(Tile **cav, unsigned int m, unsigned int n){
```

The function must print on screen the contents of all the tiles of the cavern `cav` of size $m \times n$, according to the explanation at the beginning of this exercise: see page [2](#) and **Fig. 1**. Spaces must only be used to represent empty tiles.

Hint. This function just prints all the tiles contents *without* considering the player position and light. Still, some operations are similar to Task [\(d\)](#) below: with a bit of planning, you can avoid code duplication...

(c) Implement the function:

```
bool movePlayer(Tile **cav, int m, int n, int r, int c)
```

where `cav` is a cavern of size $m \times n$, and `r` and `c` are a row and column position.

The function tries to change the player position to `(r, c)` in the cavern `cav`. The function must find the position of the player in `cav`, and check whether:

- the target position `(r,c)` is within the boundaries of the cavern;
- the distance between the player position and coordinate `(r,c)` is *less* than 5;
- the tile at position `(r,c)` is empty, or already contains the player.

If any of these conditions is *not* satisfied, the function must return `false` without altering the cavern. If all conditions are satisfied, the function must update the cavern `cav` and return `true`: the old player tile becomes `empty`, and the tile at position `(r, c)` contains the player. You can assume that there is one player in the cavern.

Example. Assume that `cav` is the cavern shown on page [2](#), **Fig. 1**: it has size 6×16 and the player is at position `(4, 7)`.

- `movePlayer(cav, 6, 16, 6, 7)` must return `false` (out of boundary);
- `movePlayer(cav, 6, 16, 0, 0)` must return `false` (destination too far);
- `movePlayer(cav, 6, 16, 4, 7)` must return `true` (after updating the cavern);
- `movePlayer(cav, 6, 16, 3, 7)` must return `false` (destination tile not empty).

Hints. Remember that the distance between two points at coordinates (x_1, y_1) and (x_2, y_2) is $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. A similar operation is also needed in Task [\(d\)](#) below: with a bit of planning, you can avoid code duplication...

This exercise continues on the next page...

(d) Implement the function:

```
void drawCavern(Tile **cav, unsigned int m, unsigned int n)
```

The function must print on screen the cavern `cav` of size $m \times n$ taking into account the position of the player and the torch light, according to the explanation at the beginning of this exercise: see page [2](#) and **Fig. 2**.

More in detail, the function must find the position of the player in the cavern `cav`, and then print the tiles on screen. When printing a tile on screen, the function must check its distance from the player's position:

- if the distance between the player position and the tile position is *less* than 4, then the tile is lit by the player's torch, hence the function must:
 - print the tile content as in Task [\(b\)](#) above, and
 - mark the tile as “explored,” by setting its `explored` field to `true`;
- otherwise, the tile is *not* lit by the player's torch. Therefore, the function must:
 - if the tile has *not* been already explored, print it as '?';
 - if the tile has been already explored, print it as '#' if it contains a rock, or '-' otherwise.

Spaces must only be used to represent empty tiles.

You can assume that there is exactly one player in the cavern.

Hint: this function must perform several checks that are also needed in Tasks [\(b\)](#) and [\(c\)](#) above (see also their hints). With a bit of planning, you can avoid code duplication...

02393 Programming in C++

File ex01-main.cpp

```
#include <iostream>
#include "ex01-library.h"
using namespace std;

int main() {
    unsigned int rows = 6, cols = 16;
    Tile **cav = createCavern(rows, cols);
    setupCavern(cav, rows, cols, 5);

    cout << "The_cavern_is:" << endl;
    drawCavern(cav, rows, cols);

    cout << endl << "Moving_to_row_3,column_0..." << endl;
    if (!movePlayer(cav, rows, cols, 3, 0)) {
        cout << "Cannot_move_to_row_3,column_0!" << endl;
    }
    drawCavern(cav, rows, cols);

    cout << endl << "Moving_to_row_3,column_7..." << endl;
    if (!movePlayer(cav, rows, cols, 3, 7)) {
        cout << "Cannot_move_to_row_3,column_7!" << endl;
    }
    drawCavern(cav, rows, cols);

    cout << endl << "Moving_to_row_2,column_4..." << endl;
    if (!movePlayer(cav, rows, cols, 2, 4)) {
        cout << "Cannot_move_to_row_2,column_4!" << endl;
    }
    drawCavern(cav, rows, cols);

    cout << endl << "Moving_to_row_4,column_7..." << endl;
    if (!movePlayer(cav, rows, cols, 4, 7)) {
        cout << "Cannot_move_to_row_4,column_7!" << endl;
    }
    drawCavern(cav, rows, cols);

    cout << endl << "Revealed_cavern:" << endl;
    revealCavern(cav, rows, cols);

    deleteCavern(cav, rows);
    return 0;
}
```

File ex01-library.h

```
#ifndef EX01_LIBRARY_H_
#define EX01_LIBRARY_H_

enum Content { nothing, player, rock, wumpus };

struct Tile {
    Content content;
    bool explored;
};

Tile **createCavern(unsigned int m, unsigned int n);
void revealCavern(Tile **cav, unsigned int m, unsigned int n);
bool movePlayer(Tile **cav, int m, int n, int r, int c);
void drawCavern(Tile **cav, unsigned int m, unsigned int n);

void setupCavern(Tile **cav, unsigned int m, unsigned int n,
                unsigned int seed);
void deleteCavern(Tile **cav, unsigned int m);

#endif /* EX01_LIBRARY_H_ */
```

File ex01-library.cpp

```
#include <iostream>
#include <random>
#include "ex01-library.h"

using namespace std;

// Task 1(a). Implement this function
Tile **createCavern(unsigned int m, unsigned int n) {
    // Replace the following with your code
    return nullptr;
}

// Task 1(b). Implement this function
void revealCavern(Tile **cav, unsigned int m, unsigned int n) {
    // Write your code here
}

// Task 1(c). Implement this function
bool movePlayer(Tile **cav, int m, int n, int r, int c) {
    // Replace the following with your code
    return false;
}

// Task 1(d). Implement this function
void drawCavern(Tile **cav, unsigned int m, unsigned int n) {
    // Write your code here
}

// Do not modify the following function.
// This code (that you don't need to read) places the player
// at location (0,0) and pseudo-randomly places some rocks
// and a Wumpus. The pseudo-random placement depends on the
// value of 'seed'.
void setupCavern(Tile **cav, unsigned int m, unsigned int n,
                unsigned int seed) {
    mt19937 e; // Pseudo-random number generator
    e.seed(seed);

    // 1/3rd of the tiles are rocks
    unsigned int rocks = (m * n) / 3;
    for (unsigned int i = 0; i < rocks; i++) {
        cav[e() % m][e() % n].content = rock;
    }

    // We never place the Wumpus on row 0 or column 0
    unsigned int row = (e() % (m-1)) + 1;
    unsigned int col = (e() % (n-1)) + 1;
    cav[row][col].content = wumpus;

    cav[0][0].content = player;
}

// Do not modify the following function.
void deleteCavern(Tile **c, unsigned int m) {
    for (unsigned int i = 0; i < m; i++) {
        delete[] c[i];
    }
    delete[] c;
}
```

EXERCISE 2. DTU HANDBALL TOURNAMENT

Bob is writing a program to manage the DTU Handball Tournament. Each DTU department can form a team, and play a match against another team; the winner of a match proceeds to the next match. Bob's plan is to represent the DTU Handball Tournament data as a tree, where the leaves are the teams, and each internal node represent a handball match between a "left" and "right" team, and its winner.

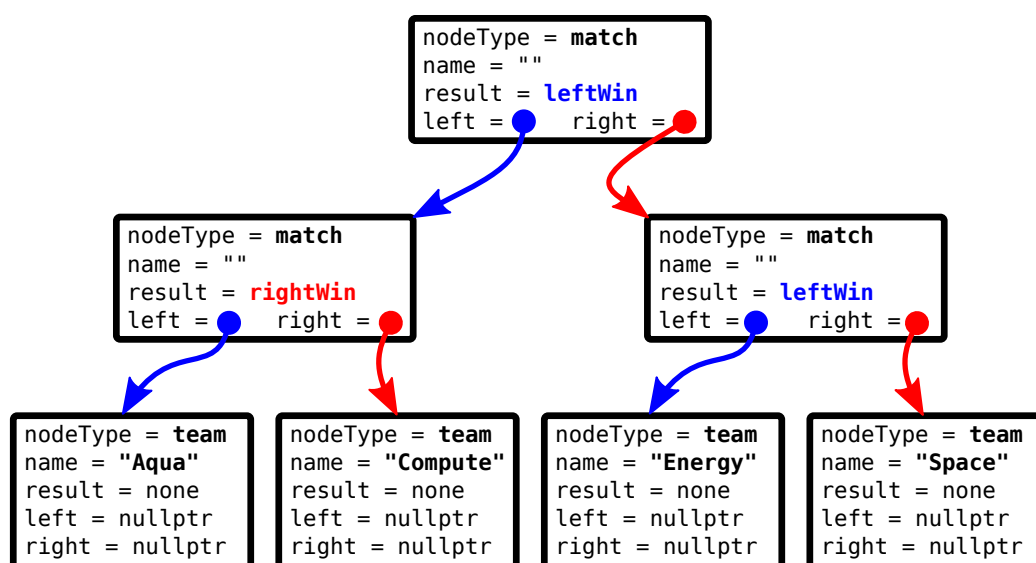
Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and they are also reported in the next pages.

Structure of the code. A tournament tree node is represented with a `struct TournamentNode` having 5 fields:

- `nodeType` can be either `team` or `match`;
- `name` (*only meaningful if `nodeType` is `team`*) is the name of the team;
- `result` (*only meaningful if `nodeType` is `match`*) can be `leftWin` or `rightWin` or `none`;
- `left` and `right` (*only meaningful if `nodeType` is `match`*) point to two subtrees.

An empty tournament is represented as a `TournamentNode*` pointer equal to `nullptr`.

Example. A small tournament with 4 teams is represented as follows:



The leaves contain the 4 teams: Aqua, Compute, Energy, and Space. The node with "`nodeType = match`" pointing to Aqua and Compute means: the result of the match was a win for the team on the "right" (i.e. Compute), which proceeds to the next match. Instead, the match between Energy and Space was won by the team on the "left" (i.e. Energy) which proceeds to the next match. The topmost node is the final match between the winners of the two matches below (i.e. Compute on the left, and Energy on the right); the result was a win for the team on the "left," so DTU Compute won the handball tournament.

Tasks. Help Bob by completing the following tasks. You need to edit and submit the file `ex02-library.cpp`. **NOTE:** some tasks may be easier to solve using recursion, but you can use iteration if you prefer.

(a) Implement the function:

```
void displayTeams(TournamentNode *t)
```

which shows the teams participating in the tournament `t`. The function must traverse the tree `t` in-order, from left to right, printing on screen one team name per line.

Special case. If `t` is `nullptr`, the function must not print anything on screen.

Example. Consider the tournament tree on page [6](#), and assume that `t` points to its topmost node. Then, `displayTeams(t)` must print:

```
Aqua
Compute
Energy
Space
```

(b) Implement the function:

```
unsigned int matches(TournamentNode *t)
```

which returns the number of matches that took place in the tournament tree `t`.

Special cases. If `t` is `nullptr` or `t` points to a team node, then the function must return 0.

Example. Consider the tournament tree on page [6](#), and assume that `t` points to its topmost node. Then, `matches(t)` must return 3.

(c) Implement the function:

```
string winner(TournamentNode *t)
```

which returns the name of the team that won the tournament `t`.

The function must traverse the match nodes of the tournament tree `t` by descending either on the “left” or “right” (depending on who won the match), until it reaches a team node (which is the winner of `t`).

Special cases. If `t` is `nullptr`, the function must return the empty string `""`.

Example. Consider the tournament tree on page [6](#), and assume that `t` points to its topmost node. Then, `winner(t)` must return `"Compute"`.

This exercise continues on the next page...

(d) Implement the function:

```
bool wonAnyMatch(TournamentNode *t, string teamName)
```

which returns `true` if the team named `teamName` won *at least* one match in the tournament `t`. Otherwise (i.e. if the team did not participate in the tournament, or lost its very first match and was eliminated) the function returns `false`.

Special cases. If `t` is `nullptr` or `t` points to a team node, then the function must return `false`.

Example. Consider the tournament tree on page 6, and assume that `t` points to its topmost node. Then:

- `wonAnyMatch(t, "Energy")` must return `true`;
- `wonAnyMatch(t, "Space")` must return `false`;
- `wonAnyMatch(t, "Physics")` must return `false`.

Hint. To find out whether a team called "`X`" has won at least one match in a tournament tree `t`, you can traverse the match nodes of `t`, looking for one where the winner between `left` and `right` directly points to a team node for "`X`". For example, consider the tournament tree on page 6: we can determine that "`Energy`" has won at least one match by traversing the tree until we encounter the match node on the right, whose `left` field (i.e. the winner) directly points to the "`Energy`" team.

File ex02-library.h

```

#ifndef EX02_LIBRARY_H_
#define EX02_LIBRARY_H_

#include <string>

enum NodeType { match, team };
enum Result { leftWin, rightWin, none };

struct TournamentNode {
    NodeType nodeType;

    std::string name; // Only used if nodeType == team

    Result result; // Only used if nodeType == match;
    TournamentNode *left; // Only used if nodeType == match;
    TournamentNode *right; // Only used if nodeType == match;
};

void displayTeams(TournamentNode *t);
unsigned int matches(TournamentNode *t);
std::string winner(TournamentNode *t);
bool wonAnyMatch(TournamentNode *t, std::string teamName);

#endif /* EX02_LIBRARY_H_ */

```

File ex02-main.cpp

```

#include <iostream>
#include "ex02-library.h"
using namespace std;

int main() {
    TournamentNode t0 = {team, "Aqua", none, nullptr, nullptr};
    TournamentNode t1 = {team, "Compute", none, nullptr, nullptr};
    TournamentNode t2 = {team, "Energy", none, nullptr, nullptr};
    TournamentNode t3 = {team, "Space", none, nullptr, nullptr};

    // Initial matches
    TournamentNode m0 = {match, "", rightWin, &t0, &t1};
    TournamentNode m1 = {match, "", leftWin, &t2, &t3};

    // Final match
    TournamentNode t = {match, "", leftWin, &m0, &m1};

    cout << "The teams in the tournament are:" << endl;
    displayTeams(&t);

    cout << endl << "The number of matches is:" << matches(&t) << endl;

    cout << "The winner of the tournament is:" << winner(&t) << endl;

    cout << "Did DTU Energy win any match?";
    if (wonAnyMatch(&t, "Energy")) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    cout << "Did DTU Space win any match?";
    if (wonAnyMatch(&t, "Space")) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    cout << "Did DTU Physics win any match?";
    if (wonAnyMatch(&t, "Physics")) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    return 0;
}

```

File ex02-library.cpp

```

#include <iostream>
#include "ex02-library.h"
using namespace std;

// Task 2(a). Implement this function
void displayTeams(TournamentNode *t) {
    // Write your code here
}

// Task 2(b). Implement this function
unsigned int matches(TournamentNode *t) {
    // Replace the following with your code
    return 0;
}

// Task 2(c). Implement this function
string winner(TournamentNode *t) {
    // Replace the following with your code
    return "";
}

// Task 2(d). Implement this function
bool wonAnyMatch(TournamentNode *t, string teamName) {
    // Replace the following with your code
    return false;
}

```

EXERCISE 3. SMART LOCKERS

Claire works in a logistics company, which has recently installed several smart lockers. She is writing a class `Locker` to manage the information about the lockers and the packages they contain. She has already written some code: her first test program is in file `ex03-main.cpp` and the (incomplete) code of the class is in files `ex03-library.h` and `ex03-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and they are also reported in the next pages.

Structure of the code. Claire has represented the information about a package using a `struct Package`, with 3 fields:

- `sender`: the name of the sender;
- `recipient`: the name of the recipient;
- `unlockCode`: a numeric code used by the recipient to unlock the locker and retrieve the package.

Claire knows that the `map` and `vector` containers of the C++ standard library provide many functionalities she needs. (See hints on page 11.) Therefore, she has decided to use the following internal (`private`) representation for the library:

- `vector<string> lockerIDs` — the identifiers of the installed lockers;
- `map<string,Package> lockerOccupancy` — a mapping from strings (locker IDs) to instances of `Package` (info about the package occupying the locker, if any). When a locker ID does not appear in this mapping, it means that the locker is empty and available.

Claire has already implemented the default constructor of `Locker`, which creates an internal database with all the lockers installed. She has also implemented the method `display()`, which shows whether a locker is occupied, and what package it contains.

Tasks. Help Claire by completing the following tasks. You need to edit and submit the file `ex03-library.cpp`.

(a) Implement the following method to put a package inside a locker:

```
void Locker::putPackage(string lockerID, string sender, string recipient, int unlockCode)
```

The method must work as follows:

- (a) if `lockerID` is *not* in `lockerIDs`, do nothing;
- (b) otherwise:
 - if the locker `lockerID` is already occupied by some package, do nothing;
 - if the locker `lockerID` is available, then update the map `lockerOccupancy` to assign the given locker to the given package.

This exercise continues on the next page...

(b) Implement the following method to retrieve a package:

```
void Locker::retrievePackage(string lockerID, int unlockCode)
```

The method must check whether `lockerID` is occupied by a package with the given `unlockCode`; if so, remove the occupancy (so the locker is available again); otherwise, do nothing.

(c) Implement the method:

```
void Locker::findPackagesByRecipient(vector<string> recipients)
```

This method displays the ID(s) of the locker(s) with a package whose recipient is contained in the given collection `recipients`.

The locker IDs must be displayed one-per-line, by following their order in `lockerIDs`.

For example, suppose that we have a vector `v` containing the strings `"Alice"` and `"Bob"`. Then, `locker.findPackagesByRecipient(v)` will display the IDs of all lockers whose package has recipient equal to either `"Alice"` or `"Bob"`.

Hints on using maps and vectors (See also: <https://en.cppreference.com/w/cpp/container/map> and <https://en.cppreference.com/w/cpp/container/vector>)

- To remove an element from a map or a vector, you can use their `erase(...)` methods:
 - <https://en.cppreference.com/w/cpp/container/map/erase>
 - <https://en.cppreference.com/w/cpp/container/vector/erase>
- A key `k` in a map `m` can be mapped to `v` with: `m[k] = v`; with this operation, the entry for `k` in `m` is created (if not already present) or updated (if already present).
- To check if key `k` is present in map `m`, you can check: `m.find(k) != m.end()`.
- The value mapped to a key `k` in a map `m` is obtained with: `m[k]`.
- To loop on all (key, value) pairs in a map `m`, you can use: `for (auto p: m) { ... }`. The loop variable `p` is a `pair` with the map key as `p.first`, and the corresponding value as `p.second` (see <https://en.cppreference.com/w/cpp/utility/pair>).

02393 Programming in C++

File ex03-main.cpp

```
#include <iostream>
#include "ex03-library.h"
using namespace std;

int main() {
    Locker locker = Locker();

    cout << "Initial_locker_occupancy:" << endl;
    locker.display();

    locker.putPackage("LYNGBY04", "Alice", "Bob", 951);
    cout << endl << "After_putting_a_package_from_Alice_to_Bob_inside_an_empty_locker:" << endl;
    locker.display();

    locker.retrievePackage("LYNGBY01", 1234);
    cout << endl << "After_retrieving_a_package:" << endl;
    locker.display();

    cout << endl << "Package(s)_addressed_to_Bob_or_Daisy:" << endl;
    vector<string> v;
    v.push_back("Bob");
    v.push_back("Daisy");
    locker.findPackagesByRecipient(v);

    return 0;
}
```

File ex03-library.h

```
#ifndef EX03_LIBRARY_H_
#define EX03_LIBRARY_H_

#include <string>
#include <vector>
#include <map>
using namespace std;

struct Package {
    string sender;
    string recipient;
    int unlockCode;
};

class Locker {
private:
    vector<string> lockerIDs;
    map<string,Package> lockerOccupancy;
public:
    Locker();
    void putPackage(string lockerID, string sender, string recipient, int unlockCode);
    void retrievePackage(string lockerID, int unlockCode);
    void findPackagesByRecipient(vector<string> recipients);
    void display();
};

#endif /* EX03_LIBRARY_H_ */
```

02393 Programming in C++

File ex03-library.cpp

```
#include <iostream>
#include "ex03-library.h"
using namespace std;

// Do not modify
Locker::Locker() {
    this->lockerIDs.push_back("LYNGBY01");
    this->lockerOccupancy["LYNGBY01"] = {"Alice", "Daisy", 1234};

    this->lockerIDs.push_back("LYNGBY02");
    this->lockerOccupancy["LYNGBY02"] = {"Claire", "Alice", 567};

    this->lockerIDs.push_back("LYNGBY03");
    this->lockerOccupancy["LYNGBY03"] = {"Daisy", "Bob", 890};

    this->lockerIDs.push_back("LYNGBY04");

    this->lockerIDs.push_back("LYNGBY05");
    this->lockerOccupancy["LYNGBY05"] = {"Bob", "Daisy", 159};

    this->lockerIDs.push_back("LYNGBY06");
}

// Task 3(a). Implement this method
void Locker::putPackage(string lockerID, string sender, string recipient, int unlockCode) {
    // Write your code here
}

// Task 3(b). Implement this method
void Locker::retrievePackage(string lockerID, int unlockCode) {
    // Write your code here
}

// Task 3(c). Implement this method
void Locker::findPackagesByRecipient(vector<string> recipients) {
    // Write your code here
}

// Do not modify
void Locker::display() {
    for (auto it = this->lockerIDs.begin(); it != this->lockerIDs.end(); it++) {
        cout << "Locker_" << *it << "_";
        if (this->lockerOccupancy.find(*it) == this->lockerOccupancy.end()) {
            cout << "is_empty" << endl;
        } else {
            cout << "contains_a_package_from" << this->lockerOccupancy[*it].sender;
            cout << "_to_" << this->lockerOccupancy[*it].recipient;
            cout << "_(unlock_code:_" << this->lockerOccupancy[*it].unlockCode << ")" << endl;
        }
    }
}
```

EXERCISE 4. COUNTING SENSOR DATA BUFFER

Daisy is writing a program that reads `integer` values from a sensor.

Her program may need to read either the most frequent value obtained from the sensor, or the frequency (i.e., the number of occurrences being observed by the sensor) of a specific value. Therefore, she plans a `CountingBuffer` class with the following interface:

- `write(v)`: appends value `v` (obtained from the sensor) into the buffer;
- `frequency(v)`: returns the frequency of value `v`;
- `mostFrequent()`: returns the value with highest frequency. If two or more values meet this condition, it returns the newest value. If no value is in the buffer, it returns the default value specified in the `CountingBuffer` constructor (see below).
- `clear()`: empties the buffer, and resets the frequency counters to 0.

Daisy's first test program is in the file `ex04-main.cpp` and the (incomplete) code of the class is in files `ex04-library.h` and `ex04-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and they are also reported in the next pages.

Structure of the code. Daisy has defined a high-level abstract class `Buffer` with the pure virtual methods `write(v)` and `clear()`. She wants to implement `CountingBuffer` as a subclass of `Buffer`, with the additional methods `frequency(v)` and `mostFrequent()`.

Example. Once completed, the class `CountingBuffer` must work as follows:

- suppose that we create `buf = CountingBuffer(-1)` (i.e. `buf` has a default value `-1`);
- suppose that `buf.write(9)` is invoked, followed by `buf.write(7)`. Then, a call to `buf.mostFrequent()` must return 7. Also, calls to `buf.frequency(9)` and `buf.frequency(7)` must return 1;
- then, suppose that `buf.write(3)` is invoked, followed by `buf.write(9)`. Since the frequency of 9 is now greater than the frequency of 7, a call to `buf.mostFrequent()` must now return 9. Also, a call to `buf.frequency(9)` must now return 2;
- finally, suppose that `buf.clear()` is invoked. Then, `buf.mostFrequent()` must now return the default value `-1`, and `buf.frequency(9)` must now return 0.

Tasks. Help Daisy by completing the following tasks. You need to edit and **submit two files**: `ex04-library.h` and `ex04-library.cpp`.

NOTE: you are free to define the `private` members of `CountingBuffer` however you see fit. For instance, you may choose to use a `map<int,int>` to keep track of the frequencies of values written in the buffer. (*See hints on page 11.*) Also, you are free to (internally) compute the frequencies and the most frequent value either when new values are appended into the buffer, or when the `frequency(v)` and `mostFrequent()` methods are invoked. The tests will only consider the behaviour of the public methods `write()`, `frequency(v)`, `mostFrequent()`, and `clear()`.

This exercise continues on the next page...

(a) Declare in `ex04-library.h` and sketch in `ex04-library.cpp` a class `CountingBuffer` that extends `Buffer`. This task is completed (and passes CodeJudge tests) when `ex04-main.cpp` compiles without errors. To achieve this, you will need to:

1. define a `CountingBuffer` constructor that takes one parameter: an `int` value representing the default value (returned by `mostFrequent()` when the buffer is empty);
2. in `CountingBuffer`, override the *pure virtual methods* of `Buffer` (i.e., those with “=0”), and add the following `public` methods to the class interface:
 - `unsigned int frequency(int v)`
 - `int mostFrequent()`
3. finally, write a placeholder implementation of all the `CountingBuffer` methods above (e.g. they may do nothing and/or just return 0 when invoked).

(b) This is a follow-up to task (a) above. In `ex04-library.cpp`, write a working implementation of the methods:

```
void CountingBuffer::write(int v)
unsigned int CountingBuffer::frequency(int v)
```

The intended behaviour of `write(v)` is to store value `v` (so it can be used by `frequency(v)` and `mostFrequent()`). The method `frequency(v)` returns the frequency of value `v` (i.e., the number of times `v` was stored in the buffer).

(c) This is a follow-up to tasks (a) and (b) above. In `ex04-library.cpp`, write a working implementation of the method:

```
int CountingBuffer::mostFrequent()
```

When invoked, `mostFrequent()` returns the value with the highest frequency. If two (or more) values meet this condition, then the method returns the newest (i.e. the last one inserted using the method `write(...)` above).

Special case: if the buffer is empty, then `mostFrequent()` must return the default value specified in the `CountingBuffer` constructor.

This exercise continues on the next page...

- (d) This is a follow-up to task (a), (b), and (c) above. In `ex04-library.cpp`, write a working implementation of the method:

```
void CountingBuffer::clear()
```

When invoked, `clear()` empties the buffer and resets the frequency counters. Consequently, invoking `clear()` and then `mostFrequent()` returns the default buffer value (specified in the `CountingBuffer` constructor); also, after invoking `clear()`, `frequency(v)` returns 0 for any value `v`, until `write(v)` is invoked.

02393 Programming in C++

File ex04-main.cpp

```
#include <iostream>
#include "ex04-library.h"
using namespace std;

int main() {
    CountingBuffer *cb = new CountingBuffer(-1);
    Buffer *b = cb; // Just an alias for 'cb' above, but using the superclass

    cout << "Most_frequent_value:_ " << cb->mostFrequent() << endl;
    cout << "Finding_the_frequency_for_value_0_returns:_ " << cb->frequency(0) << endl;

    b->write(9); b->write(7);
    cout << "Wrote_9_and_7_Most_frequent_value:_ " << cb->mostFrequent() << endl;
    cout << "Finding_the_frequency_for_value_9_returns:_ " << cb->frequency(7) << endl;
    cout << "Finding_the_frequency_for_value_7_returns:_ " << cb->frequency(9) << endl;

    b->write(3); b->write(9);
    cout << "Wrote_3_and_9_Most_frequent_value:_ " << cb->mostFrequent() << endl;
    cout << "Finding_the_frequency_for_value_3_returns:_ " << cb->frequency(3) << endl;
    cout << "Finding_the_frequency_for_value_9_returns:_ " << cb->frequency(9) << endl;

    b->clear();
    cout << "Buffer_cleared_Most_frequent_value:_ " << cb->mostFrequent() << endl;
    cout << "Finding_the_frequency_for_value_9_now_returns:_ " << cb->frequency(9) << endl;

    delete cb;
    return 0;
}
```

File ex04-library.h

```
#ifndef EX04_LIBRARY_H_
#define EX04_LIBRARY_H_

class Buffer {
public:
    virtual void write(int v) = 0;
    virtual void clear() = 0;
    virtual ~Buffer();
};

// Task 4(a). Declare the class CountingBuffer, by extending Buffer
// Write your code here

#endif /* EX04_LIBRARY_H_ */
```

File ex04-library.cpp

```
#include "ex04-library.h"

// Task 4(a). Write a placeholder implementation of CountingBuffer's
// constructor and methods

// Task 4(b). Write a working implementation of write() and frequency()

// Task 4(c). Write a working implementation of mostFrequent()

// Task 4(d). Write a working implementation of clear()

// Do not modify
Buffer::~Buffer() {
    // Empty destructor
}
```

Technical University of Denmark

Written examination date: 31 May 2022



Course title: Programming in C++

Page 1 of 16 pages

Course number: 02393

Aids allowed: All aids allowed

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises with 3 or 4 tasks each, for a total of 14 tasks

Submission details:

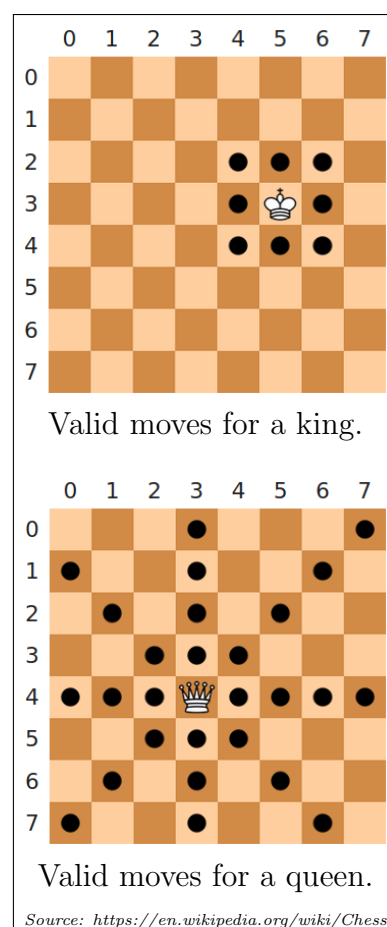
1. You must **submit your solution on DTU Digital Eksamen**. You can do it **only once**, so submit only when you have completed your work.
2. You must submit your solution as **one ZIP archive** containing the following files, with **these exact names**:
 - `exZZ-library.cpp`, where `ZZ` ranges from `01` to `04` (i.e., one per exercise);
 - `ex04-library.h` (additionally required for exercise 4).
3. You can test your solutions by uploading them on CodeJudge, under “Reexam May 2022” at:
<https://dtu.codejudge.net/02393-f22/exercises>
4. You can test your solutions on CodeJudge as many times as you like. *Uploads on CodeJudge are not official submissions* and will not affect your grade.
5. Additional tests may be run on your submissions after the exam.
6. Feel free to add comments to your code.
7. **Suggestion:** read all exercises before starting your work; start by solving the tasks that look easier, even if they belong to different exercises.

EXERCISE 1. MINI CHESS

Alice wants to implement a variant of the game of chess:

- there are two opposing teams: black and white;
- the chessboard can have any size of $m \times n$ squares (with $m \geq 2$ and $n \geq 2$);
- each team has two kinds of pieces on the chessboard:
 - kings, who move horizontally or diagonally by one square (see figure on the right);
 - queens, who move horizontally or diagonally by any number of squares (see figure on the right);
- pieces can traverse already-occupied squares while moving (unlike standard chess rules);
- a piece can only end its movement on a square that is empty, or occupied by an opponent's piece. In the second case, the opponent's piece is *captured* (i.e. removed).

Alice has written some code: her first test program is the file `ex01-main.cpp`, and the (incomplete) code with some functions she needs is in `ex01-library.h` and `ex01-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and in the next pages.



Structure of the code. A square on the board is represented as a `struct Square` with two fields named `piece` and `team`, which are two `enums` representing, respectively:

- which piece (if any) is occupying the square: `king`, `queen`, or `none`;
- which team (if any) owns the piece: `black`, `white`, or `nobody` (if the piece is `none`).

Alice's code already includes the function:

```
void deleteChessboard(Square **c, unsigned int m)
```

which deallocates a chessboard `c` with `m` rows created with `createChessboard()` (task (a)).

Tasks. Help Alice by completing the following tasks. You need to edit and submit the file `ex01-library.cpp`.

(a) Implement the function:

```
Square** createChessboard(unsigned int m, unsigned int n)
```

The function must return an array of $m \times n$ `Squares`, i.e., `Square**`. It must allocate the required memory, and initialise each square to be empty (i.e. having `none` as piece and `nobody` as team).

This exercise continues on the next page...

(b) Implement the function:

```
void displayChessboard(Square **c, unsigned int m, unsigned int n)
```

The function must print on screen the contents of the chessboard `c` of size $m \times n$:

- each empty square must be displayed as `_` (underscore);
- if a square is occupied by a piece, it must be displayed as either:
 - `K` (if occupied by a black king) or `k` (if occupied by a white king);
 - `Q` (if occupied by a black queen) or `q` (if occupied by a white queen);
- adjacent squares on a same row must be separated by one space.

Example. A 3×4 chessboard might look as follows: (position (0,0) on the top-left)

```
Q _ _ K
_ _ q _
k _ _ _
```

(c) Implement the function:

```
bool move(Square **c, unsigned int m, unsigned int n,
          int r1, int c1, int r2, int c2)
```

- Argument `c` is a chessboard of size $m \times n$;
- Arguments `r1` and `c1` are a row and column position on the chessboard;
- Arguments `r2` and `c2` are a row and column position on the chessboard.

The function attempts to move a piece on the chessboard `c` from position $(r1, c1)$ into position $(r2, c2)$. The function must check whether the move is valid, i.e.:

- there is a piece at position $(r1, c1)$;
- positions $(r1, c1)$ and $(r2, c2)$ are different and not occupied by the same team;
- the move respects the game rules (see beginning of the exercise).

If the move is valid, the function must update the chessboard `c` and return `true`; otherwise, it must return `false` without altering the chessboard.

You can assume that positions $(r1, c1)$ and $(r2, c2)$ are within the chessboard bounds.

Example. Assume that `c` is the chessboard shown in Task [\(b\)](#) above:

- `move(c, 3, 4, 0, 0, 0, 3)` must return `false` (same team on both positions);
- `move(c, 3, 4, 0, 0, 0, 2)` must return `true` (the queen move is valid);
- `move(c, 3, 4, 2, 0, 2, 2)` must return `false` (the king move is invalid).
- `move(c, 3, 4, 1, 2, 0, 3)` must return `true` (white queen captures black king);

Hints. Positions $(r1, c1)$ and $(r2, c2)$ are on the same diagonal if they differ by equal (absolute) numbers of rows and columns. For instance, positions (4,5) and (6,3) are on the same diagonal, because $|4 - 6| = |5 - 3| = 2$. This check (and others) are also needed in Task [\(d\)](#) below: with a bit of planning, you can avoid code duplication...

This exercise continues on the next page...

(d) Implement the function:

```
bool threatened(Square **c, unsigned int m, unsigned int n, int row, int col)
```

Where:

- argument `c` is a chessboard of size $m \times n$;
- arguments `row` and `col` are a row and column position on the chessboard.

The function must return `true` if at position `(row, col)` of chessboard `c` there is a piece that can be captured by another piece on the same chessboard (see game rules at the beginning of the exercise). Otherwise, the function must return `false`. In both cases, the function must *not* change the chessboard.

You can assume that the position `(row, col)` is within the chessboard bounds.

Example. Assume that `c` is the chessboard shown in Task [\(b\)](#) above:

- `threatened(c, 3, 4, 0, 0)` must return `false` (no piece can capture at (0,0));
- `threatened(c, 3, 4, 2, 0)` must return `true` (the white piece at (2,0) can be captured by the black queen at (0,0));
- `threatened(c, 3, 4, 1, 1)` must return `false` (there is no piece at (1,1)).

Hint: this function must perform several checks that are also needed in Task [\(c\)](#) above (see also its hints). With a bit of planning, you can avoid code duplication...

02393 Programming in C++

File ex01-main.cpp

```
#include <iostream>
#include "ex01-library.h"
using namespace std;

int main() {
    Square **c = createChessboard(3, 4);
    c[0][0] = {queen, black};
    c[0][3] = {king, black};
    c[2][0] = {king, white};
    c[1][2] = {queen, white};
    cout << "Chessboard:" << endl;
    displayChessboard(c, 3, 4);

    cout << "Is the piece in (0,0) threatened?" << endl;
    if (threatened(c, 3, 4, 0, 0)) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    cout << "Can we move from (0,0) to (0,3)?" << endl;
    if (move(c, 3, 4, 0, 0, 0, 3)) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    cout << "Can we move from (0,0) to (0,2)?" << endl;
    if (move(c, 3, 4, 0, 0, 0, 2)) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    cout << "Can we move from (2,0) to (2,2)?" << endl;
    if (move(c, 3, 4, 2, 0, 2, 2)) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    cout << "Can we move from (1,2) to (0,3)?" << endl;
    if (move(c, 3, 4, 1, 2, 0, 3)) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    cout << endl << "The chessboard is now:" << endl;
    displayChessboard(c, 3, 4);

    cout << "Is the piece in (2,0) threatened?" << endl;
    if (threatened(c, 3, 4, 2, 0)) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    deleteChessboard(c, 3);
    return 0;
}
```

File ex01-library.h

```
#ifndef EX01_LIBRARY_H_
#define EX01_LIBRARY_H_

enum Piece { king, queen, none };
enum Team { black, white, nobody };

struct Square {
    Piece piece;
    Team team;
};

Square **createChessboard(unsigned int m, unsigned int n);
void displayChessboard(Square **c, unsigned int m, unsigned int n);
bool move(Square **c, unsigned int m, unsigned int n,
          int r1, int c1, int r2, int c2);
bool threatened(Square **c, unsigned int m, unsigned int n, int row, int col);
void deleteChessboard(Square **c, unsigned int m);

#endif /* EX01_LIBRARY_H_ */
```

File ex01-library.cpp

```
#include <iostream>
#include "ex01-library.h"

using namespace std;

// Task 1(a). Implement this function
Square **createChessboard(unsigned int m, unsigned int n) {
    // Replace the following with your code
    return nullptr;
}

// Task 1(b). Implement this function
void displayChessboard(Square **c,
                      unsigned int m, unsigned int n) {
    // Write your code here
}

// Task 1(c). Implement this function
bool move(Square **c, unsigned int m, unsigned int n,
          int r1, int c1, int r2, int c2) {
    // Replace the following with your code
    return false;
}

// Task 1(d). Implement this function
bool threatened(Square **c, unsigned int m, unsigned int n,
                int row, int col) {
    // Replace the following with your code
    return false;
}

// Do not modify
void deleteChessboard(Square **c, unsigned int m) {
    for (unsigned int i = 0; i < m; i++) {
        delete[] c[i];
    }
    delete[] c;
}
```

EXERCISE 2. AIRLINE PASSENGERS QUEUE

Bob is writing a program to manage airline passengers boarding a flight. Passengers queue in order of arrival, but they may board the flight in a different order, depending on where they sit. Bob decides to represent the queue as a linked list.

Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and they are also reported in the next pages.

Structure of the code. A queue element is represented as a `struct Passenger` with 4 fields: `name`, `ticket`, `row`, `seat`, and `next`. Such fields represent, respectively, the passenger name, the ticket number, the row number and the seat (a letter between 'A' and 'F') where the passenger sits, and a pointer to the next passenger in the queue (or `nullptr` when there are no more passengers). An empty queue is represented as a `Passenger*` pointer equal to `nullptr`. Bob's code already includes a function to print the passengers queue on screen:

```
void displayQueue(Passenger *q)
```

Tasks. Help Bob by completing the following tasks. You need to edit and submit the file `ex02-library.cpp`. **NOTE:** some tasks may be easier to solve using recursion, but you can use iteration if you prefer.

(a) Implement the function:

```
void shift(Passenger *q, unsigned int n)
```

which modifies the seat of each passenger in the queue `q`, by adding `n` rows to it. For example: if `q` contains a passenger sitting in row 3, a call to `shift(q, 2)` will update that passenger's seat to row $3 + 2 = 5$.

Important: the function must modify the elements of `q` in-place.

(b) Implement the function:

```
Passenger* find(Passenger *q, unsigned int rowMin, unsigned int rowMax)
```

which returns a new queue containing all `Passengers` in the queue `q` that sit between rows `rowMin` and `rowMax` (included), preserving the order in which they appear in `q`. **Important:** the function must return a new queue, where each `Passenger` is a dynamically-allocated copy of its original from `q`; the function must *not* modify `q`.

This exercise continues on the next page...

(c) Implement the function:

```
bool occupied(Passenger *q, unsigned int row, char seat)
```

which checks all passengers in the queue `q` and returns `true` if there is someone sitting in the given `row` and `seat`. If no passenger in the queue is sitting in that position, the function returns `false`.

02393 Programming in C++

File ex02-library.h

```
#ifndef EX02_LIBRARY_H_
#define EX02_LIBRARY_H_

#include <string>

struct Passenger {
    std::string name;
    unsigned int ticket;
    unsigned int row;
    char seat;
    Passenger *next;
};

void displayQueue(Passenger *q);

void shift(Passenger *q, unsigned int n);
Passenger* find(Passenger *q, unsigned int rowMin,
               unsigned int rowMax);
bool occupied(Passenger *q, unsigned int row, char seat);

#endif /* EX02_LIBRARY_H_ */
```

File ex02-main.cpp

```
#include <iostream>
#include <string>
#include "ex02-library.h"
using namespace std;

int main() {
    Passenger p0 = {"AlfredA.", 123, 5, 'A', nullptr};
    Passenger p1 = {"BarbaraB.", 321, 1, 'B', &p0};
    Passenger p2 = {"CharlieC.", 456, 10, 'D', &p1};
    Passenger p3 = {"DariaD.", 654, 22, 'C', &p2};
    Passenger p4 = {"EmilE.", 789, 10, 'E', &p3};
    Passenger p5 = {"FionaF.", 987, 21, 'F', &p4};

    Passenger *q = &p5;

    cout << "The passengers queue is:" << endl;
    displayQueue(q);
    cout << endl;

    shift(q, 2);
    cout << "After shifting the passengers by 2 rows we have:" << endl;
    displayQueue(q);
    cout << endl;

    cout << "The passengers sitting between rows 5 and 12 are:" << endl;
    Passenger *q2 = find(q, 5, 12);
    if (q2 == nullptr) { cout << "nobody!" << endl; }
    else { displayQueue(q2); }
    cout << endl;

    cout << "Is seat 10D occupied?";
    if (occupied(q, 10, 'D')) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    cout << "Is seat 12E occupied?";
    if (occupied(q, 12, 'E')) { cout << "Yes!" << endl; }
    else { cout << "No!" << endl; }

    return 0;
}
```

File ex02-library.cpp

```
#include <iostream>
#include "ex02-library.h"
using namespace std;

// Task 2(a). Implement this function
void shift(Passenger *q, unsigned int n) {
    // Write your code here
}

// Task 2(b). Implement this function
Passenger* find(Passenger *q, unsigned int rowMin,
               unsigned int rowMax) {
    // Replace the following with your code
    return nullptr;
}

// Task 2(c). Implement this function
bool occupied(Passenger *q, unsigned int row, char seat) {
    // Replace the following with your code
    return false;
}

// Do not modify
void displayQueue(Passenger *q) {
    if (q == nullptr) {
        return;
    }
    cout << q->name << " ticket:" << q->ticket;
    cout << ", seat:" << q->row << q->seat << endl;
    displayQueue(q->next);
}
```

EXERCISE 3. HOTEL MANAGEMENT

Claire owns a fancy hotel where every room has the name of a flower. She is writing a class `Hotel` to manage the information about the rooms and guests. She has already written some code: her first test program is in file `ex03-main.cpp` and the (incomplete) code of the class is in files `ex03-library.h` and `ex03-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and they are also reported in the next pages.

Structure of the code. Claire has represented the information about a guest using a `struct Guest`, with two fields:

- `name`: the full name of the guest;
- `id`: the document id provided by the guest.

Claire knows that the `map` and `vector` containers of the C++ standard library provide many functionalities she needs. (See *hints on page 10.*) Therefore, she has decided to use the following internal (`private`) representation for the library:

- `vector<string> roomNames` — the names of the fancy hotel rooms;
- `map<string, Guest> roomOccupancy` — a mapping from `strings` (room names) to instances of `Guest` (info about the guest occupying the room, if any). When a room name does not appear in this mapping, it means that the room is empty and available.

Claire has already implemented the default constructor of `Hotel`, which creates an internal database with all rooms. She has also implemented the method `display()`, which shows which guest occupies which room.

Tasks. Help Claire by completing the following tasks. You need to edit and submit the file `ex03-library.cpp`.

(a) Implement the following method to rename a room:

```
void Hotel::renameRoom(string oldName, string newName)
```

The method must work as follows:

- (a) if `oldName` is *not* in `roomNames`, do nothing;
- (b) if `newName` is already in `roomNames`, do nothing;
- (c) otherwise, update `roomNames` by replacing the element `oldName` with `newName`; also, update `roomOccupancy` so that any guest occupying `oldName` is moved to `newName`.

This exercise continues on the next page...

(b) Implement the following method to remove a guest:

```
void Hotel::removeGuest(string roomName, string guestName, string guestId)
```

The method must check whether `roomName` is occupied by a guest with the given `guestName` and `guestId`; if so, remove the occupancy (so the room is available again); otherwise, do nothing.

(c) Implement the method:

```
void Hotel::findRoomByGuestId(vector<string> guestIds)
```

This method displays the name(s) of the room(s) with an occupant whose id is contained in the given collection `guestIds`.

The room names must be displayed one-per-line, by following their order in `roomNames`.

For example, suppose that we have a vector `v` containing the strings `"123"` and `"abc"`. Then, `hotel.findRoomByGuestId(v)` will display the names of all rooms whose guest has id equal to either `"123"` or `"abc"`.

Hints on using maps and vectors (See also: <https://www.cplusplus.com/reference/map/map/> and <https://www.cplusplus.com/reference/vector/vector/>)

- To remove an element from a map or a vector, you can use their `erase(...)` methods:
 - <https://www.cplusplus.com/reference/map/map/erase/>
 - <https://www.cplusplus.com/reference/vector/vector/erase/>
- A key `k` in a map `m` can be mapped to `v` with: `m[k] = v`; with this operation, the entry for `k` in `m` is created (if not already present) or updated (if already present).
- To check if key `k` is present in map `m`, you can check: `m.find(k) != m.end()`.
- The value mapped to a key `k` in a map `m` is obtained with: `m[k]`.
- To loop on all (key, value) pairs in a map `m`, you can use: `for (auto p: m) { ... }`. The loop variable `p` is a `pair` with the map key as `p.first`, and the corresponding value as `p.second` (see <https://www.cplusplus.com/reference/utility/pair/>)

02393 Programming in C++

File ex03-main.cpp

```
#include <iostream>
#include "ex03-library.h"
using namespace std;

int main() {
    Hotel hotel = Hotel();

    cout << "Initial hotel occupancy:" << endl;
    hotel.display();

    hotel.renameRoom("Lotus", "Waterlily");
    cout << endl << "After renaming room 'Lotus' to 'Waterlily':" << endl;
    hotel.display();

    hotel.removeGuest("Orchid", "Alan_Smithee", "abc123");
    cout << endl << "After removing a guest:" << endl;
    hotel.display();

    cout << endl << "Room(s) with guests with id '123xyz' or '456abc':" << endl;
    vector<string> v;
    v.push_back("123xyz");
    v.push_back("456abc");
    hotel.findRoomByGuestId(v);

    return 0;
}
```

File ex03-library.h

```
#ifndef EX03_LIBRARY_H_
#define EX03_LIBRARY_H_

#include <string>
#include <vector>
#include <map>
using namespace std;

struct Guest {
    string name;
    string id;
};

class Hotel {
private:
    vector<string> roomNames;
    map<string, Guest> roomOccupancy;
public:
    Hotel();
    void renameRoom(string oldName, string newName);
    void removeGuest(string roomName, string guestName, string guestId);
    void findRoomByGuestId(vector<string> guestIds);
    void display();
};

#endif /* EX03_LIBRARY_H_ */
```

02393 Programming in C++

File ex03-library.cpp

```
#include <iostream>
#include "ex03-library.h"
using namespace std;

// Do not modify
Hotel::Hotel() {
    this->roomNames.push_back("Daisy");
    this->roomOccupancy["Daisy"] = {"Alan_Smithee", "xyz890"};

    this->roomNames.push_back("Geranium");

    this->roomNames.push_back("Lotus");
    this->roomOccupancy["Lotus"] = {"Kathryn_Bigelow", "456abc"};

    this->roomNames.push_back("Orchid");
    this->roomOccupancy["Orchid"] = {"Alan_Smithee", "abc123"};

    this->roomNames.push_back("Tulip");
    this->roomOccupancy["Tulip"] = {"Denis_Villeneuve", "123xyz"};
}

// Task 3(a). Implement this method
void Hotel::renameRoom(string oldName, string newName) {
    // Write your code here
}

// Task 3(b). Implement this method
void Hotel::removeGuest(string roomName, string guestName, string guestId) {
    // Write your code here
}

// Task 3(c). Implement this method
void Hotel::findRoomByGuestId(vector<string> guestIds) {
    // Write your code here
}

// Do not modify
void Hotel::display() {
    for (auto it = this->roomNames.begin(); it != this->roomNames.end(); it++) {
        cout << "Room_" << *it << "_is_";
        if (this->roomOccupancy.find(*it) == this->roomOccupancy.end()) {
            cout << "empty" << endl;
        } else {
            cout << "occupied_by_" << this->roomOccupancy[*it].name;
            cout << "_id:" << this->roomOccupancy[*it].id << ")" << endl;
        }
    }
}
```

EXERCISE 4. SENSOR DATA BUFFER

Daisy is writing a program that reads `integer` values from a sensor; the sensor may sometimes yield values that are outside a certain allowed range, so her program must account for this possibility. Therefore, she plans a `SensorBuffer` class with the following interface:

- `write(v)`: appends value `v` (obtained from the sensor) into the buffer;
- `read()`: returns the oldest value written in the buffer, and removes it from the buffer;
- `faults()`: returns the number of time the method `write(v)` has been invoked with a value `v` outside a range specified with the `SensorBuffer` constructor (see below).
- `clear()`: empties the buffer, and resets the number of faults to 0.

Daisy's first test program is in the file `ex04-main.cpp` and the (incomplete) code of the class is in files `ex04-library.h` and `ex04-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive), and they are also reported in the next pages.

Structure of the code. Daisy has defined a high-level abstract class `Buffer` with the pure virtual methods `write()` and `read()`. She wants to implement `SensorBuffer` as a subclass of `Buffer`, with the additional methods `faults()` and `clear()`.

Example. Once completed, the class `SensorBuffer` must work as follows:

- suppose that we create `buf = SensorBuffer(-1, 5, 10)` (i.e. `buf` has a default value `-1`, and expects that written values are between 5 and 10 included);
- suppose that `buf.write(7)` is invoked, followed by `buf.write(9)`. Then, a call to `buf.read()` must return 7, and a further call to `buf.read()` must return 9 (therefore, `read()` removes the returned value from the buffer). A further call to `buf.read()` must return the default value `-1` (since `buf` is now empty);
- then, suppose that `buf.write(3)` is invoked. Since the given value 3 is below the minimum value 5, it must be corrected as 5, so a subsequent call to `buf.read()` must return 5. Moreover, `buf.faults()` must return 1;
- finally, suppose that `buf.clear()` is invoked. Then, `buf.read()` must return the default value `-1`, and `buf.faults()` must return 0.

Tasks. Help Daisy by completing the following tasks. You need to edit and **submit two files**: `ex04-library.h` and `ex04-library.cpp`.

NOTE: you are free to define the `private` members of `SensorBuffer` however you see fit. For instance, you might choose to store the values in a `vector<int>`, or in a linked list. The tests will only consider the behaviour of the public methods `write()`, `read()`, `faults()`, and `clear()`.

This exercise continues on the next page...

- (a) Declare in `ex04-library.h` and sketch in `ex04-library.cpp` a class `SensorBuffer` that extends `Buffer`. This task is completed (and passes CodeJudge tests) when `ex04-main.cpp` compiles without errors. To achieve this, you will need to:
1. define a `SensorBuffer` constructor that takes 3 parameters: 3 `int` values representing, respectively, the default value (returned by `read()` when the buffer is empty), and the minimum and maximum values (checked when using `write()`);
 2. in `SensorBuffer`, override the *pure virtual methods* of `Buffer` (i.e., those with “=0”), and add the following `public` methods to the class interface:
 - `unsigned int faults()`
 - `void clear()`
 3. finally, write a placeholder implementation of all the `SensorBuffer` methods above (e.g. they may do nothing and/or just return 0 when invoked).
- (b) This is a follow-up to task (a) above. In `ex04-library.cpp`, write a working implementation of the methods:

```
void SensorBuffer::write(int v)
unsigned int SensorBuffer::faults()
```

The intended behaviour of `write(v)` is to check whether `v` is between the minimum and maximum values specified in the `SensorBuffer` constructor:

- i)* if `v` is within such values, it is stored (so it can be retrieved by `read()`);
- ii)* otherwise, if `v` is below the minimum value, the minimum value is stored;
- iii)* otherwise (i.e. if `v` is above the maximum value), the maximum value is stored.

In both cases (ii) and (iii), the method `write()` should increment an internal counter used by `faults()`. The method `faults()` returns how many times the method `write(v)` has been invoked with an out-of-range value `v`.

- (c) This is a follow-up to tasks (a) and (b) above. In `ex04-library.cpp`, write a working implementation of the method:

```
int SensorBuffer::read()
```

When invoked, `read()` returns the oldest value stored in the buffer using `write()`, removing the value from the buffer.

Special case: if the buffer is empty, then `read()` must return the default value specified in the `SensorBuffer` constructor.

This exercise continues on the next page...

- (d) This is a follow-up to task (a), (b), and (c) above. In `ex04-library.cpp`, write a working implementation of the method:

```
void SensorBuffer::clear()
```

When invoked, `clear()` empties the buffer and resets the faults counter. Consequently, invoking `clear()` and then `read()` returns the default buffer value (specified in the `SensorBuffer` constructor); also, invoking `clear()` and then `faults()` returns 0.

02393 Programming in C++

File ex04-main.cpp

```
#include <iostream>
#include "ex04-library.h"
using namespace std;

int main() {
    SensorBuffer *sb = new SensorBuffer(-1, 5, 10);
    Buffer *b = sb; // Just an alias for 'sb' above, but using the superclass

    cout << "Current faults:\n" << sb->faults() << endl;
    cout << "Reading from the buffer returns:\n" << b->read() << endl;

    b->write(7); b->write(9);
    cout << "Wrote 7 and 9. Current faults:\n" << sb->faults() << endl;
    cout << "Reading from the buffer now returns:\n" << b->read() << endl;
    cout << "Reading from the buffer now returns:\n" << b->read() << endl;

    b->write(3); b->write(10);
    cout << "Wrote 3 and 10. Current faults:\n" << sb->faults() << endl;
    cout << "Reading from the buffer now returns:\n" << b->read() << endl;
    cout << "Reading from the buffer now returns:\n" << b->read() << endl;

    sb->clear();
    cout << "Buffer cleared. Current faults:\n" << sb->faults() << endl;
    cout << "Reading from the buffer now returns:\n" << b->read() << endl;

    delete sb;
    return 0;
}
```

File ex04-library.h

```
#ifndef EX04_LIBRARY_H_
#define EX04_LIBRARY_H_

class Buffer {
public:
    virtual void write(int v) = 0;
    virtual int read() = 0;
    virtual ~Buffer();
};

// Task 4(a). Declare the class SensorBuffer, by extending Buffer
// Write your code here

#endif /* EX04_LIBRARY_H_ */
```

File ex04-library.cpp

```
#include "ex04-library.h"

// Task 4(a). Write a placeholder implementation of SensorBuffer's
// constructor and methods

// Task 4(b). Write a working implementation of write() and faults()

// Task 4(c). Write a working implementation of read()

// Task 4(d). Write a working implementation of clear()

// Do not modify
Buffer::~Buffer() {
    // Empty destructor
}
```

Course title: Programming in C++

Course number: 02393

Aids allowed: All aids allowed

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises with 3 or 4 tasks each, for a total of 15 tasks

Page 1 of 12 pages

Submission details:

1. You must **submit your solution on DTU Digital Eksamen**. You can do it **only once**, so submit only when you have completed your work.
2. You must submit your solution as **one file per exercise**, with **these exact names**:
 - For exercises 1 to 3, `exZZ-library.cpp`, where `ZZ` ranges from 01 to 03 (i.e., one per exercise);
 - For exercise 4, `ex04-library.zip`, which must contain `ex04-library.cpp` and `ex04-library.h`.
3. You can test your solutions by uploading them on Autolab, under “Exam December 2023” at:
<https://autolab.compute.dtu.dk/courses/02393-E23/assessments>
4. You can test your solutions on Autolab as many times as you like. *Uploads on Autolab are not official submissions* and will not affect your grade.
5. In case Autolab is malfunctioning, you can test your solution by comparing your output with `exZZ-main-output.txt`, which is enclosed in the ZIP archive for each handout.
6. Additional tests may be run on your submissions after the exam.
7. Feel free to add comments to your code.
8. **Suggestion:** read all exercises before starting your work; start by solving the tasks that look easier, even if they belong to different exercises.

EXERCISE 1. BATTLESHIP

Alice is implementing a variant of the old game *Battleship* (also known as Sea Battle): the first player places up to 9 ships in a sea with a map made of tiles. Each ship can occupy up to 4 consecutive tiles, and it can be placed either horizontally or vertically. Each ship must be numbered differently from the other ships. Ships must not overlap. Then, the second player has to sink all the ships by launching missiles targeting a specific tile. A map of size 6×16 might look like **Fig. 1** on the right:

- ' ' is an empty tile;
- '3' is a tile occupied by ship number 3, which has not been hit by a missile;
- 'X' is a tile occupied by a ship, which has been hit by a missile.

After the first player has placed all the ships, they become hidden. The sea with hidden ships is drawn as shown in **Fig. 2** on the right:

- tiles that are not hit by a missile are shown as '?';
- if a tile was hit by a missile, then:
 - if the tile is occupied by a ship, it is shown as 'X';
 - otherwise, it is shown as ' '.

Alice has written some code: a test program is in the file `ex01-main.cpp`, and an (incomplete) implementation is in `ex01-library.h` and `ex01-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive).

Structure of the code. A tile of the sea is represented as a `struct Tile` with two fields named `ship` and `hit`, which represent, respectively:

- the number associated to the ship occupying that tile; 0 indicates that no ship occupies that tile;
- whether the tile has been hit by a missile.

Alice's code already includes a function (that you must not modify):

- `void deleteSea(Tile **sea, unsigned int m)`

This function deallocates the sea.

Tasks. Help Alice by completing the following tasks. You need to edit and submit the file `ex01-library.cpp`.

This exercise continues on the next page...

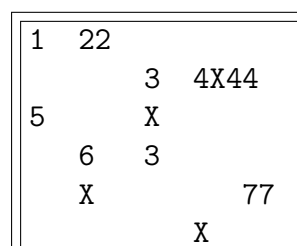


Fig. 1. 6×16 sea with all tiles visible.

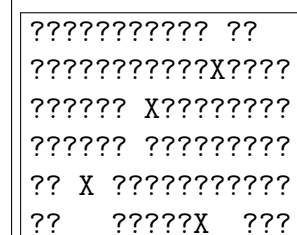


Fig. 2. The sea above with hidden ships, some of which have been hit.

(a) Implement the function:

```
Tile** createSea(unsigned int m, unsigned int n)
```

It must return an array of $m \times n$ Tiles, i.e., `Tile**`. It must allocate the required memory, and initialise each tile with `ship` set to 0 and `hit` set to `false`.

(b) Implement the function:

```
void displaySea(Tile **sea, unsigned int m, unsigned int n,  
    bool reveal)
```

The function must print on screen the contents of all the tiles of the sea `sea` of size $m \times n$, according to the explanation at the beginning of this exercise: when `reveal` is set to `true`, the sea must be displayed as shown in **Fig. 1** of page [2](#); when `reveal` is set to `false`, the sea must be displayed as shown in **Fig. 2** of page [2](#).

(c) Implement the function:

```
bool placeShip(Tile **sea, int m, int n, int r, int c, int number,  
    int size, bool vertical)
```

where `sea` is a sea of size $m \times n$, `r` and `c` are a row and column position, `number` is the number of the ship, `size` the number of tiles it occupies, and `vertical` indicates whether the ship should be placed vertically or horizontally in the sea.

The function tries to place a ship in the sea, starting from the tile at position (r, c) . The function must check whether:

- No other ship in the sea has the same `number`;
- the size of the ship is between 1 and 4 (included);
- the position (r, c) is within the boundaries of the sea;
- if the ship is placed horizontally:
 - The position $(r, c+size)$ is within the boundaries of the sea;
 - Tiles at position $(r, c+i)$, where `i` is between 0 and `size` (included), are not occupied by another ship.
- if the ship is placed vertically:
 - The position $(r+size, c)$ is within the boundaries of the sea;
 - Tiles at position $(r+i, c)$, where `i` is between 0 and `size` (included), are not occupied by another ship.

This exercise continues on the next page...

If any of these conditions is *not* satisfied, the function must return `false` without altering the sea. If all conditions are satisfied, the function must update the sea `sea` and return `true`:

- if the ship is placed horizontally, tiles at position $(r, c+i)$, where i is between 0 and `size` (included), will be associated to the ship `number`;
- if the ship is placed vertically, tiles at position $(r+i, c)$, where i is between 0 and `size` (included), will be associated to the ship `number`.

Example. Assume that `sea` is the sea shown on page 2, **Fig. 1**: it has size 6×16 .

- `placeShip(sea, 6, 16, 5, 0, 1, 2, false)` must return `false` (another ship numbered 1 exists);
- `placeShip(sea, 6, 16, 5, 0, 9, 5, false)` must return `false` (ship size exceeds limit);
- `placeShip(sea, 6, 16, 6, 7, 9, 2, false)` must return `false` (position out of boundary);
- `placeShip(sea, 6, 16, 5, 15, 9, 2, false)` must return `false` (position out of boundary);
- `placeShip(sea, 6, 16, 5, 9, 9, 3, false)` must return `false` (tile already occupied);
- `placeShip(sea, 6, 16, 5, 0, 9, 2, false)` must return `true` (after updating the sea);

(d) Implement the function:

```
bool launchMissile(Tile **sea, int m, int n, int r, int c)
```

where `sea` is a sea of size $m \times n$, and `r` and `c` are a row and column position of the tile targeted by the missile.

The function tries to launch a missile targeting the tile at position (r, c) . The function must check whether:

- The position (r, c) is within the boundaries of the sea;
- the tile has already been hit.

If any of these conditions is *not* satisfied, the function must return `false` without altering the sea. If all conditions are satisfied, the function must update the sea `sea` by marking the tile as hit, and return `true`.

EXERCISE 2. PLAYLIST

Bob is writing a program to manage the songs aired by a radio station. In particular, songs queue in a playlist, where they are removed once they are played. Normally, the playlist works in FIFO (first-in-first-out) order: the oldest song being added to the playlist will be the first one to be played. However, one can decide to reorder the playlist by shifting a listed song before or after the others. Bob decides to represent the playlist as a linked list, where the head of the list represents the song that will be played first, and the tail the song that will be played last.

Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive).

Structure of the code. A playlist element is represented as a `struct Song` with 4 fields: `title`, `artist`, `genre`, `duration`, and `next`. Such fields represent, respectively, the title of the song, the artist, the genre and the duration in seconds, and a pointer to the next song in the playlist (which will be played after the current one) (or `nullptr` when there are no more songs). An empty playlist is represented as a `Song*` pointer equal to `nullptr`. Bob's code already includes a function to print the content of the playlist on screen:

```
void displayPlaylist(Song *s)
```

Tasks. Help Bob by completing the following tasks. You need to edit and submit the file `ex02-library.cpp`. **NOTE:** some tasks may be easier to solve using recursion, but you can use iteration if you prefer.

(a) Implement the function:

```
unsigned int totalDuration(Song *s)
```

which computes the total duration of all songs in the playlist `s` and returns it.

This exercise continues on the next page...

(b) Implement the function:

```
Song* find(Song *s, string genre)
```

which returns a new playlist containing all **Songs** in the playlist **s** that belong to the same **genre**, preserving the order in which they appear in **s**. **Important:** the function must return a new playlist, where each **Song** is a dynamically-allocated copy of its original from **s**; the function must *not* modify **s**.

(c) Implement the function:

```
bool shift(Song *&s, unsigned int pos, unsigned int n)
```

which tries to modify the playlist **s** by moving the song located at position **pos** after the **n** subsequent songs. Note that the position of songs start from 1.

The function must check whether:

- A song exists at position **pos**.
- A song exists at position **pos + n**.

If any of these conditions is *not* satisfied, the function must return **false** without altering **s**. If all conditions are satisfied, the function must update **s** accordingly and return **true**.

Example. Assume that **s** contains the following items (from the head to the tail):

- Billie Jean, Micheal Jackson, Pop, 232
- Sandstorm, Darude, Dance, 225
- Never Gonna Give You Up, Rick Hashley, Pop, 215
- Barbie Girl, Aqua, Dance, 196

Then, `shift(*s, 3, 2)` must return **false** since the element at position 3 does not have 2 successors. Conversely, `shift(*s, 2, 2)` must return **true** and change **s** to reflect the following order:

- Billie Jean, Micheal Jackson, Pop, 232
- Never Gonna Give You Up, Rick Hashley, Pop, 215
- Barbie Girl, Aqua, Dance, 196
- Sandstorm, Darude, Dance, 225

EXERCISE 3. PARKING AREA

Claire works in a municipality, which has recently installed several cameras to monitor a public parking area. She is writing a class `ParkingArea` to manage the information about the parking spaces in that car park, and the vehicles parked in there. She has already written some code: her first test program is in file `ex03-main.cpp` and the (incomplete) code of the class is in files `ex03-library.h` and `ex03-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive).

Structure of the code. Claire has represented the information about a vehicle occupying a parking space using a `struct Vehicle`, with 3 fields:

- **plate**: the license plate of the vehicle;
- **owner**: the name of the owner of the vehicle;
- **category**: the category of vehicle: car, motorbike, or truck.

Claire knows that the `map` and `vector` containers of the C++ standard library provide many functionalities she needs. (See *hints on page 9*.) Therefore, she has decided to use the following internal (`private`) representation for the library:

- `enum Category { CAR, MOTORBIKE, TRUCK }` — an enumeration to specify the category of a vehicle.
- `map<string,Category> parkingSpaces` — a mapping from `strings` (parking space IDs) to `Category` (info about the category of vehicle that should occupy the parking space);
- `map<string,Vehicle> parkingOccupancy` — a mapping from `strings` (parking space IDs) to instances of `Vehicle` (info about the vehicle occupying the parking space, if any). When a parking space ID does not appear in this mapping, it means that the parking space is empty.

Claire has already implemented the default constructor of `ParkingArea`, which creates an internal database with all the parking spaces installed. She has also implemented the methods `display()`, which shows whether a parking space is occupied, and what vehicle occupies it, and `categoryToString()`, which converts a `Category` into a `string`.

Tasks. Help Claire by completing the following tasks. You need to edit and submit the file `ex03-library.cpp`.

This exercise continues on the next page...

- (a) Implement the following method to register a vehicle occupying a parking space:

```
void ParkingArea::park(string parkingSpaceID, string plate,
    string owner, Category category)
```

The method must work as follows:

- (a) if `parkingSpaceID` is *not* in `parkingSpaces`, do nothing;
- (b) otherwise:
 - if the parking space `parkingSpaceID` is already occupied by a vehicle, do nothing;
 - if the parking space `parkingSpaceID` is available, then update the map `parkingOccupancy` to register the vehicle occupying that parking space.

- (b) Implement the following method to register a vehicle leaving a parking space:

```
void ParkingArea::leave(string plate)
```

The method must work as follows:

- (a) if no parking space is occupied by the vehicle having `plate` as license plate, do nothing; otherwise
- (b) update the map `parkingOccupancy` to remove information on the vehicle having `plate` as license plate.

- (c) Implement the following method to identify misplaced vehicles:

```
void ParkingArea::findMisplacedVehicles()
```

The method displays the license plates of the vehicles that occupy a parking space of a different `category` than the one of that vehicle.

The license plates must be displayed one-per-line, and parking spaces must be explored by following their order in `parkingSpaces`.

For example, if parking space “LYNGBY04” should be occupied by a car, but it is occupied by a motorbike, the license plate of the motorbike must be displayed.

This exercise continues on the next page...

(d) Implement the method:

```
void ParkingArea::findVehicles(vector<string> plates)
```

This method displays the ID(s) of the parking space(s) with a vehicle whose license plate is contained in the given collection `plates`.

The parking space ID must be displayed one-per-line, by following their order in `parkingSpaces`.

For example, suppose that we have a vector `v` containing the strings `"AB123XY"` and `"CD987QW"`. Then, `parkingSpaces.findVehicles(v)` will display the IDs of all parking spaces whose vehicle has a license plate equal to either `"AB123XY"` or `"CD987QW"`.

Hints on using maps and vectors (See also: <https://en.cppreference.com/w/cpp/container/map> and <https://en.cppreference.com/w/cpp/container/vector>)

- To remove an element from a map or a vector, you can use their `erase(...)` methods:
 - <https://en.cppreference.com/w/cpp/container/map/erase>
 - <https://en.cppreference.com/w/cpp/container/vector/erase>
- A key `k` in a map `m` can be mapped to `v` with: `m[k] = v`; with this operation, the entry for `k` in `m` is created (if not already present) or updated (if already present).
- To check if key `k` is present in map `m`, you can check: `m.find(k) != m.end()`.
- The value mapped to a key `k` in a map `m` is obtained with: `m[k]`.
- To loop on all (key, value) pairs in a map `m`, you can use: `for (auto p: m) { ... }`. The loop variable `p` is a pair with the map key as `p.first`, and the corresponding value as `p.second` (see <https://en.cppreference.com/w/cpp/utility/pair>).

EXERCISE 4. AUTO-REDUCING BUFFER

Daisy is writing a program that stores and retrieves `integer` values from a sensor.

Her program needs to implement an *auto-reducing* buffer that accumulates values up to a maximum capacity n ; when n is exceeded, the oldest value within the buffer capacity is replaced by the average of the oldest value within the buffer capacity and the oldest values beyond the buffer capacity.

Therefore, she plans a `AutoReducingBuffer` class with the following interface:

- `write(v)`: appends value `v` to the buffer; if, after appending `v`, the buffer exceeds the maximum capacity, the oldest value within the buffer capacity is replaced by the average of the oldest value within the buffer capacity and the oldest values beyond the buffer capacity.
- `read()`: removes the oldest value from the buffer and returns it; if no value is in the buffer, it returns the default value specified in the `AutoReducingBuffer` constructor (see below).
- `occupancy()`: returns the number of elements in the buffer; calling `write()` increases the occupancy by 1 until the buffer capacity is reached, while calling `read()` decreases the occupancy by 1.
- `contains(v)`: `true` if the buffer contains value `v`; otherwise, it returns `false`.

Her first test program is in file `ex04-main.cpp` and the (incomplete) code of the class is in files `ex04-library.h` and `ex04-library.cpp`. All files are available with this exam paper (in a separate ZIP archive).

Structure of the code. Daisy has defined a high-level abstract class `Buffer` with the pure virtual methods `write(v)`, `read()`, and `occupancy()`.

Example. Once completed, the class `AutoReducingBuffer` must work as follows:

- suppose that we create `buf = AutoReducingBuffer(3,-1)` (i.e. `buf` has a capacity of 3 elements and a default value -1);
- suppose that `buf.write(9)` is invoked, followed by `buf.write(1)`. Then, a call to `buf.contains(9)` must return `true`. Also, a call to `buf.contains(1)` must return `true`, and a call to `buf.occupancy()` must return 2;
- now, suppose that `buf.read()` is invoked. Then, a call to `buf.contains(9)` must now return `false`, and a call to `buf.occupancy()` must return 1, since 9 was the oldest value in the buffer;

This exercise continues on the next page...

- now, suppose that `buf.write(3)` is invoked, followed by `buf.write(8)`, `buf.write(12)` and `buf.write(7)`. Then, a call to `buf.occupancy()` must return 3 since the buffer capacity was reached. A call to `buf.contains(1)` must now return `false`, since 1 is beyond the buffer capacity. A call to `buf.contains(8)` must return `false`, and a call to `buf.contains(4)` must return `true`, since 8 has been replaced by 4 (i.e., the average between 1, 3, and 8);
- now, suppose that `buf.write(5)` is invoked. Then, a call to `buf.contains(4)` must now return `false`, and a call to `buf.contains(6)` must return `true`, since the oldest value within the buffer capacity is no longer 4, but 6 (i.e., the average between 1, 3, 8 and 12);
- finally, suppose that `buf.read()` is invoked. Then, a call to `buf.contains(6)` must now return `false`, and a call to `buf.occupancy()` must return 2, since 6 was removed from the buffer when `buf.read()` was invoked.

Tasks. Help Daisy by completing the following tasks. **IMPORTANT:** for these tasks you are required to submit *both* files `ex04-library.h` and `ex04-library.cpp` enclosed in a ZIP archive.

NOTE: you are free to define the `private` members of `AutoReducingBuffer` however you see fit. For instance, you might choose to store the values in a `vector<int>`, or in a linked list. The tests will only consider the behaviour of the public methods `write(v)`, `read()`, `occupancy()` and `contains(v)`. When the occupancy of the buffer is at its maximum, you are also free to compute the average of the oldest value within the buffer capacity and the oldest values beyond the buffer capacity whenever `write()` is invoked, or to do so when `read()` is invoked.

- (a) Declare in `ex04-library.h` and sketch in `ex04-library.cpp` a class `AutoReducingBuffer` that extends `Buffer`. This task is completed (and passes Autolab tests) when `ex04-main.cpp` compiles without errors. To achieve this, you will need to:
1. define a constructor for `AutoReducingBuffer` that takes 2 parameters:
 - (i) an `unsigned integer` representing the maximum buffer capacity; and
 - (ii) a value of type `int` representing a default (it is used in point (c) below);
 2. in `AutoReducingBuffer`, override the *pure virtual methods* of `Buffer` (i.e., those with “=0”), and add the following `public` method to the class interface:
 - `boolean contains(int v)`
 3. finally, write a placeholder implementation of all the `AutoReducingBuffer` methods above (e.g. they may do nothing and/or just return 0 when invoked).

This exercise continues on the next page...

- (b) This is a follow-up to point (a) above. In `ex04-library.cpp`, write a working implementation of the methods:

```
void AutoReducingBuffer::write(int v)
unsigned int AutoReducingBuffer::occupancy()
```

Their intended behaviour is that each time `write()` is invoked, the value returned by `occupancy()` increases by 1, until the maximum capacity is reached (such maximum capacity is specified in the constructor — see point (a)1(i) above).

Special case: if, after invoking `write(v)`, the maximum capacity is exceeded, the oldest value within the buffer capacity must be replaced by the average of that value and the oldest values beyond the buffer capacity.

- (c) This is a follow-up to points (a) and (b) above. In `ex04-library.cpp`, write a working implementation of the method:

```
int AutoReducingBuffer::read()
```

When `read()` is invoked, it removes the oldest value previously added by `write()`, and returns it; correspondingly, the value returned by `occupancy()` decreases by 1.

Special case: if the buffer is empty, then `read()` must return the default value specified in the constructor (see point (a)1(ii) above).

- (d) This is a follow-up to points (a) and (b) above. In `ex04-library.cpp`, write a working implementation of the method:

```
boolean AutoReducingBuffer::contains(int v)
```

When `contains(v)` is invoked, it returns `true` if the buffer contains value `v`. Otherwise, it returns `false`.

Course title: Programming in C++

Course number: 02393

Aids allowed: All aids allowed

Exam duration: 4 hours

Weighting: pass/fail

Exercises: 4 exercises with 3 or 4 tasks each, for a total of 15 tasks

Page 1 of 12 pages

Submission details:

1. You must **submit your solution on DTU Digital Eksamen**. You can do it **only once**, so submit only when you have completed your work.
2. You must submit your solution as **one file per exercise**, with **these exact names**:
 - For exercises 1 to 3, `exZZ-library.cpp`, where `ZZ` ranges from 01 to 03 (i.e., one per exercise);
 - For exercise 4, `ex04-library.zip`, which must contain `ex04-library.cpp` and `ex04-library.h`.
3. You can test your solutions by uploading them on Autolab, under “Exam December 2023” at:
<https://autolab.compute.dtu.dk/courses/02393-E23/assessments>
4. You can test your solutions on Autolab as many times as you like. *Uploads on Autolab are not official submissions* and will not affect your grade.
5. In case Autolab is malfunctioning, you can test your solution by comparing your output with `exZZ-main-output.txt`, which is enclosed in the ZIP archive for each handout.
6. Additional tests may be run on your submissions after the exam.
7. Feel free to add comments to your code.
8. **Suggestion:** read all exercises before starting your work; start by solving the tasks that look easier, even if they belong to different exercises.

EXERCISE 1. BATTLESHIP

Alice is implementing a variant of the old game *Battleship* (also known as Sea Battle): the first player places up to 9 ships in a sea with a map made of tiles. Each ship can occupy up to 4 consecutive tiles, and it can be placed either horizontally or vertically. Each ship must be numbered differently from the other ships. Ships must not overlap. Then, the second player has to sink all the ships by launching missiles targeting a specific tile. A map of size 8×16 might look like **Fig. 1** on the right:

- ' ' is an empty tile;
- '3' is a tile occupied by ship number 3, which has not been hit by a missile;
- 'X' is a tile occupied by a ship, which has been hit by a missile.

After the first player has placed all the ships, they become hidden. The sea with hidden ships is drawn as shown in **Fig. 2** on the right:

- tiles that are not hit by a missile are shown as '?';
- if a tile was hit by a missile, then:
 - if the tile is occupied by a ship, it is shown as 'X';
 - otherwise, it is shown as ' '.

Alice has written some code: a test program is in the file `ex01-main.cpp`, and an (incomplete) implementation is in `ex01-library.h` and `ex01-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive).

Structure of the code. A tile of the sea is represented as a `struct Tile` with two fields named `ship` and `hit`, which represent, respectively:

- the number associated to the ship occupying that tile; 0 indicates that no ship occupies that tile;
- whether the tile has been hit by a missile.

Alice's code already includes a function (that you must not modify):

- `void deleteSea(Tile **sea, unsigned int m)`

This function deallocates the sea.

Tasks. Help Alice by completing the following tasks. You need to edit and submit the file `ex01-library.cpp`.

This exercise continues on the next page...

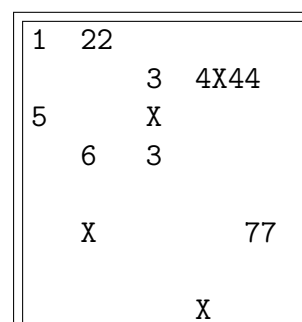


Fig. 1. 8×16 sea with all tiles visible.

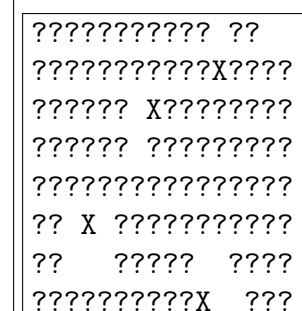


Fig. 2. The sea above with hidden ships, some of which have been hit.

(a) Implement the function:

```
Tile** createSea(unsigned int m)
```

It must return an array of $m \times 2m$ Tiles, i.e., `Tile**`. It must allocate the required memory, and initialise each tile with `ship` set to 0 and `hit` set to `false`.

(b) Implement the function:

```
void displaySea(Tile **sea, unsigned int m,
               bool reveal)
```

The function must print on screen the contents of all the tiles of the sea `sea` of size $m \times 2m$, according to the explanation at the beginning of this exercise: when `reveal` is set to `true`, the sea must be displayed as shown in **Fig. 1** of page [2](#); when `reveal` is set to `false`, the sea must be displayed as shown in **Fig. 2** of page [2](#).

(c) Implement the function:

```
bool launchBigMissile(Tile **sea, int m, int r, int c)
```

where `sea` is a sea of size $m \times 2m$, and `r` and `c` are a row and column position of the tile targeted by the missile.

The function tries to launch a missile targeting the tile at position (r, c) . The function must check whether:

- The position (r, c) is within the boundaries of the sea;
- the tile has not been hit.

If any of these conditions is *not* satisfied, the function must return `false` without altering the sea. If all conditions are satisfied, the function must:

- update the sea `sea` by marking as hit:
 - the tile at position (r, c) ;
 - the tiles that are at most 1 tile far - horizontally or vertically - from position (r, c) , if they exist.
- return `true`.

This exercise continues on the next page...

Example. Assume that `sea` is the sea shown on page [2](#), **Fig. 2**: it has size 8×16 .

- `launchBigMissile(sea, 8, 2, 6)` must return `false` (tile at position (2,6) has already been hit);
- `launchBigMissile(sea, 8, 4, 15)` must return `true` (after updating the sea by marking the tiles at position (4,15), (3,15), (5,14), (4,14) as hit);
- `launchBigMissile(sea, 8, 4, 14)` must return `false` (tile at position (4,14) has been hit when `launchBigMissile(sea, 8, 4, 15)` was invoked);
- `launchBigMissile(sea, 8, 3, 14)` must return `true` (after updating the sea by marking the tiles at position (3,14), (2,14), (3,13) as hit);

(d) Implement the function:

```
bool isGameOver(Tile **sea, int m)
```

where `sea` is a sea of size $m \times 2m$.

The function must check if all tiles occupied by a ship have been hit. If so, the function must return `false`. Otherwise, the function must return `true`.

EXERCISE 2. PLAYLIST

Bob is writing a program to manage the songs aired by a radio station. In particular, songs queue in a playlist, where they are removed once they are played. Normally, the playlist works in FIFO (first-in-first-out) order: the oldest song being added to the playlist will be the first one to be played. However, one can decide to reorder the playlist by shifting a listed song before or after the others. Bob decides to represent the playlist as a linked list, where the head of the list represents the song that will be played first, and the tail the song that will be played last.

Bob has already written some code. His first test program is in file `ex02-main.cpp` and the (incomplete) code with some functions he needs is in files `ex02-library.h` and `ex02-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive).

Structure of the code. A playlist element is represented as a `struct Song` with 4 fields: `title`, `artist`, `genre`, `duration`, and `next`. Such fields represent, respectively, the title of the song, the artist, the genre and the duration in seconds, and a pointer to the next song in the playlist (which will be played after the current one) (or `nullptr` when there are no more songs). An empty playlist is represented as a `Song*` pointer equal to `nullptr`. Bob's code already includes a function to print the content of the playlist on screen:

```
void displayPlaylist(Song *s)
```

Tasks. Help Bob by completing the following tasks. You need to edit and submit the file `ex02-library.cpp`. **NOTE:** some tasks may be easier to solve using recursion, but you can use iteration if you prefer.

(a) Implement the function:

```
unsigned int totalDuration(Song *s, string genre)
```

which computes the total duration of all songs of the specified `genre` in the playlist `s` and returns it.

This exercise continues on the next page...

(b) Implement the function:

```
Song* find(Song *s, unsigned int duration)
```

which returns a new playlist containing all **Songs** in the playlist **s** that are longer than the specified **duration**, preserving the order in which they appear in **s**. **Important:** the function must return a new playlist, where each **Song** is a dynamically-allocated copy of its original from **s**; the function must *not* modify **s**.

(c) Implement the function:

```
Song* split(Song *&s, unsigned int pos)
```

which tries to modify the playlist **s** by moving the song located from position **pos** onwards into a new playlist, which will be returned by that function. Note that the position of songs start from 1.

The function must check if a song exists at position **pos**. If this condition is *not* satisfied, the function must return an empty playlist without altering **s**. If this condition is satisfied, the function must update **s** by removing all the songs from position **pos** onwards. In addition, it must add these songs to a new playlist, which will be returned by the function. Also, the order of the songs in the new playlist must be the same as the one in the original playlist.

Example. Assume that **s** contains the following items (from the head to the tail):

- Billie Jean, Micheal Jackson, Pop, 232
- Sandstorm, Darude, Dance, 225
- Never Gonna Give You Up, Rick Hashley, Pop, 215
- Barbie Girl, Aqua, Dance, 196

Then, `split(*s, 5)` must return `false` since there is no song at position 5. Conversely, `split(*s, 3)` must change **s** to reflect the following order:

- Billie Jean, Micheal Jackson, Pop, 232
- Sandstorm, Darude, Dance, 225

In addition, it must return a new playlist containing the following items:

- Never Gonna Give You Up, Rick Hashley, Pop, 215
- Barbie Girl, Aqua, Dance, 196

EXERCISE 3. PARKING AREA

Claire works in a municipality, which has recently installed several cameras to monitor a public parking area. She is writing a class `ParkingArea` to manage the information about the parking spaces in that car park, and the vehicles parked in there. She has already written some code: her first test program is in file `ex03-main.cpp` and the (incomplete) code of the class is in files `ex03-library.h` and `ex03-library.cpp`. Such files are available with this exam paper (in a separate ZIP archive).

Structure of the code. Claire has represented the information about a vehicle occupying a parking space using a `struct Vehicle`, with 2 fields:

- `plate`: the license plate of the vehicle;
- `owner`: the name of the owner of the vehicle.

Claire knows that the `map` and `vector` containers of the C++ standard library provide many functionalities she needs. (See *hints on page 9*.) Therefore, she has decided to use the following internal (`private`) representation for the library:

- `enum Category { CAR, MOTORBIKE, TRUCK }` — an enumeration to specify the category of a vehicle.
- `map<string,Category> parkingSpaces` — a mapping from `strings` (parking space IDs) to `Category` (info about the category of vehicle that should occupy the parking space);
- `map<string,Vehicle> parkingOccupancy` — a mapping from `strings` (parking space IDs) to instances of `Vehicle` (info about the vehicle occupying the parking space, if any). When a parking space ID does not appear in this mapping, it means that the parking space is empty.

Claire has already implemented the default constructor of `ParkingArea`, which creates an internal database with all the parking spaces installed. She has also implemented the methods `display()`, which shows whether a parking space is occupied, and what vehicle occupies it, and `categoryToString()`, which converts a `Category` into a `string`.

Tasks. Help Claire by completing the following tasks. You need to edit and submit the file `ex03-library.cpp`.

This exercise continues on the next page...

- (a) Implement the following method to register a vehicle occupying a parking space:

```
void ParkingArea::park(string parkingSpaceID, string plate,
    string owner, Category category)
```

The method must work as follows:

- (a) if `parkingSpaceID` is *not* in `parkingSpaces`, do nothing;
- (b) otherwise:
 - if the parking space `parkingSpaceID` is for vehicles belonging to a different `category`, do nothing;
 - otherwise:
 - if the parking space `parkingSpaceID` is already occupied by a vehicle, do nothing;
 - if the parking space `parkingSpaceID` is available, then update the map `parkingOccupancy` to register the vehicle occupying that parking space.

- (b) Implement the following method to register vehicles leaving a parking space:

```
void ParkingArea::leave(vector<string> plates)
```

The method updates the map `parkingOccupancy` to remove information on the vehicles whose license plate is contained in the given collection `plates`.

- (c) Implement the following method to count empty parking spaces:

```
unsigned int ParkingArea::getEmptySpaces(Category category)
```

The method returns how many empty parking spaces are available for a vehicle of the specified `category`.

For example, suppose that there are 4 parking spaces for cars, 3 of which are occupied. Then, `parkingSpaces.getEmptySpaces(CAR)` will return 1.

This exercise continues on the next page...

(d) Implement the method:

```
void ParkingArea::findVehicles(vector<string> owners)
```

This method displays the ID(s) of the parking space(s) with a vehicle whose owner is contained in the given collection `owners`.

The parking space ID must be displayed one-per-line, by following their order in `parkingSpaces`.

For example, suppose that we have a vector `v` containing the strings `"Alice"` and `"Bob"`. Then, `parkingSpaces.findVehicles(v)` will display the IDs of all parking spaces whose vehicle has an owner equal to either `"Alice"` or `"Bob"`.

Hints on using maps and vectors (See also: <https://en.cppreference.com/w/cpp/container/map> and <https://en.cppreference.com/w/cpp/container/vector>)

- To remove an element from a map or a vector, you can use their `erase(...)` methods:
 - <https://en.cppreference.com/w/cpp/container/map/erase>
 - <https://en.cppreference.com/w/cpp/container/vector/erase>

Note that, when calling `erase(...)`, references and iterators to the erased elements are invalidated. An iterator following the last erased element is returned by that method.

- A key `k` in a map `m` can be mapped to `v` with: `m[k] = v`; with this operation, the entry for `k` in `m` is created (if not already present) or updated (if already present).
- To check if key `k` is present in map `m`, you can check: `m.find(k) != m.end()`.
- The value mapped to a key `k` in a map `m` is obtained with: `m[k]`.
- To loop on all (key, value) pairs in a map `m`, you can use: `for (auto p: m) { ... }`. The loop variable `p` is a pair with the map key as `p.first`, and the corresponding value as `p.second` (see <https://en.cppreference.com/w/cpp/utility/pair>).

EXERCISE 4. AUTO-SUMMING BUFFER

Daisy is writing a program that stores and retrieves `integer` values from a sensor.

Her program needs to implement an *auto-summing* buffer that accumulates values up to a maximum capacity n ; when n is exceeded, the oldest value within the buffer capacity is replaced by the sum of the oldest value within the buffer capacity and the oldest values beyond the buffer capacity.

Therefore, she plans a `AutoSummingBuffer` class with the following interface:

- `write(v)`: appends value `v` to the buffer; if, after appending `v`, the buffer exceeds the maximum capacity, the oldest value within the buffer capacity is replaced by the sum of the oldest value within the buffer capacity and the oldest values beyond the buffer capacity.
- `read()`: removes the oldest value from the buffer and returns it; if no value is in the buffer, it returns the default value specified in the `AutoSummingBuffer` constructor (see below).
- `occupancy()`: returns the number of elements in the buffer; calling `write()` increases the occupancy by 1 until the buffer capacity is reached, while calling `read()` decreases the occupancy by 1.
- `occurrences(v)`: returns the number of occurrences of value `v` in the buffer.

Her first test program is in file `ex04-main.cpp` and the (incomplete) code of the class is in files `ex04-library.h` and `ex04-library.cpp`. All files are available with this exam paper (in a separate ZIP archive).

Structure of the code. Daisy has defined a high-level abstract class `Buffer` with the pure virtual methods `write(v)`, `read()`, and `occupancy()`.

Example. Once completed, the class `AutoSummingBuffer` must work as follows:

- suppose that we create `buf = AutoSummingBuffer(3,-1)` (i.e. `buf` has a capacity of 3 elements and a default value `-1`);
- suppose that `buf.write(9)` is invoked, followed by `buf.write(1)`. Then, a call to `buf.occurrences(9)` must return 1, since 9 is present only once in the buffer. Also, a call to `buf.occurrences(1)` must return 1, and a call to `buf.occupancy()` must return 2;
- now, suppose that `buf.read()` is invoked. Then, a call to `buf.occurrences(9)` must now return 0, and a call to `buf.occupancy()` must return 1, since 9 was the oldest value in the buffer and it was removed when `buf.read()` was invoked.;

This exercise continues on the next page...

- now, suppose that `buf.write(3)` is invoked, followed by `buf.write(8)`, `buf.write(6)` and `buf.write(6)`. Then, a call to `buf.occupancy()` must return 3 since the buffer capacity was reached. A call to `buf.occurrences(1)` must now return 0, since 1 is beyond the buffer capacity. A call to `buf.occurrences(8)` must return 0, and a call to `buf.occurrences(12)` must return 1, since 8 has been replaced by 12 (i.e., the sum of 1, 3, and 8). A call to `buf.occurrences(6)` must return 2, since 6 is present twice in the buffer;
- now, suppose that `buf.write(5)` is invoked. Then, a call to `buf.occurrences(12)` must now return 0, and a call to `buf.occurrences(18)` must return 1, since the oldest value within the buffer capacity is no longer 12, but 18 (i.e., the sum between 1, 3, 8 and 6);
- finally, suppose that `buf.read()` is invoked. Then, a call to `buf.occurrences(18)` must now return 0, and a call to `buf.occupancy()` must return 2, since 18 was removed from the buffer when `buf.read()` was invoked.

Tasks. Help Daisy by completing the following tasks. **IMPORTANT:** for these tasks you are required to submit *both* files `ex04-library.h` and `ex04-library.cpp` enclosed in a ZIP archive.

NOTE: you are free to define the `private` members of `AutoSummingBuffer` however you see fit. For instance, you might choose to store the values in a `vector<int>`, or in a linked list. The tests will only consider the behaviour of the public methods `write(v)`, `read()`, `occupancy()` and `occurrences(v)`. When the occupancy of the buffer is at its maximum, you are also free to compute the sum of the oldest value within the buffer capacity and the oldest values beyond the buffer capacity whenever `write()` is invoked, or to do so when `read()` is invoked.

- (a) Declare in `ex04-library.h` and sketch in `ex04-library.cpp` a class `AutoSummingBuffer` that extends `Buffer`. This task is completed (and passes Autolab tests) when `ex04-main.cpp` compiles without errors. To achieve this, you will need to:
1. define a constructor for `AutoSummingBuffer` that takes 2 parameters:
 - (i) an `unsigned integer` representing the maximum buffer capacity; and
 - (ii) a value of type `int` representing a default (it is used in point **(c)** below);
 2. in `AutoSummingBuffer`, override the *pure virtual methods* of `Buffer` (i.e., those with “=0”), and add the following `public` method to the class interface:
 - `unsigned int occurrences(int v)`
 3. finally, write a placeholder implementation of all the `AutoSummingBuffer` methods above (e.g. they may do nothing and/or just return 0 when invoked).

This exercise continues on the next page...

- (b) This is a follow-up to point (a) above. In `ex04-library.cpp`, write a working implementation of the methods:

```
void AutoSummingBuffer::write(int v)
unsigned int AutoSummingBuffer::occupancy()
```

Their intended behaviour is that each time `write()` is invoked, the value returned by `occupancy()` increases by 1, until the maximum capacity is reached (such maximum capacity is specified in the constructor — see point (a)1(i) above).

Special case: if, after invoking `write(v)`, the maximum capacity is exceeded, the oldest value within the buffer capacity must be replaced by the sum of that value and the oldest values beyond the buffer capacity.

- (c) This is a follow-up to points (a) and (b) above. In `ex04-library.cpp`, write a working implementation of the method:

```
int AutoSummingBuffer::read()
```

When `read()` is invoked, it removes the oldest value previously added by `write()`, and returns it; correspondingly, the value returned by `occupancy()` decreases by 1.

Special case: if the buffer is empty, then `read()` must return the default value specified in the constructor (see point (a)1(ii) above).

- (d) This is a follow-up to points (a) and (b) above. In `ex04-library.cpp`, write a working implementation of the method:

```
unsigned int AutoSummingBuffer::occurrences(int v)
```

When `occurrences(v)` is invoked, it returns the number of occurrences of value `v` in the buffer.